Two Schemas of Impossibility: Machine-Checked Limits of Trustworthy Machine Learning, and Why They Tighten with Capability

Manish Bhatt

Contents

Contents i

1 Abstract 1

2 How to read this thesis 3

3 Introduction 5
3.1 Thesis statement 5
3.2 What is not claimed 6
3.3 Contributions 6
3.4 Structure of the thesis 7
3.5 Declaration of collaborative work 7

4 Part I: Background 9

5 Preliminaries 11
5.1 How to read the Lean 4 11
5.2 Sets and functions 18
5.3 The minimum topology 19
5.4 The two engines 21
5.5 How this is verified and built 21

6 The Diagonal 23
6.1 One move, five disguises 23
6.2 Systems that describe themselves 25
6.3 Lawvere's fixed-point theorem 26
6.4 The section and retract form 27
6.5 The contrapositive is the impossibility 28
6.6 Three classical instances 29
6.7 A partial-function preview: toward Rice 30
6.8 The engine, named once 31
6.9 What the diagonal cannot give 32
6.10 Roadmap 32
6.11 Historical notes 33

7 The Classical Limits 35
7.1 The recipe . . . 35
7.2 Cantor's theorem . . . 36
7.3 Russell's paradox . . . 37
7.4 Gödel's first incompleteness theorem . . . 38
7.5 Gödel's second incompleteness theorem . . . 41
7.6 Tarski's undefinability of truth . . . 44
7.7 Turing and the halting problem . . . 45
7.8 Rice's theorem . . . 46
7.9 The categorical unification . . . 48
7.10 Interlude: the wrapper is the mathematics . . . 48
7.11 What varies, and what does not . . . 49
7.12 Historical notes . . . 49

8 Related Work and Positioning 51
8.1 Unifications of the diagonal . . . 51
8.2 Impossibility results in machine learning . . . 51
8.3 Calibration and hallucination . . . 52
8.4 Interpretability and probing . . . 52
8.5 Formal verification of machine learning . . . 53
8.6 Summary of the gap . . . 53

9 Part II: The Two Engines 55

10 Three Trilemmata, One Theorem 57
10.1 What "reflective" demands . . . 58
10.2 The Hallucination Trilemma . . . 58
10.3 The Defense Trilemma . . . 61
10.4 The Coupling Theorem . . . 64
10.5 The Master Trilemma . . . 67
10.6 One proof term . . . 69
10.7 What the readings share, and where they part . . . 69
10.8 The template in four moves . . . 70
10.9 Axioms and trust . . . 70
10.10 What this chapter does not settle . . . 71
10.11 Historical notes . . . 71

11 From Logic to Analysis 73
11.1 Why a second engine . . . 73
11.2 A topology primer . . . 74
11.3 The model, analytically . . . 76
11.4 The topological proof of the trilemma . . . 78
11.5 A worked instance on the interval . . . 79
11.6 Quantitative shadows . . . 80
11.7 Symmetry in place of coverage . . . 80
11.8 The discrete core . . . 81

11.9 Where the two engines meet 83
11.10 Where the two engines diverge 84
11.11 The engine in one page 85
11.12 Historical notes 85

12 The Geometry of Attacks 87
12.1 From the diagonal to the boundary 87
12.2 Decision basins 88
12.3 The threshold-crossing theorem 89
12.4 Lipschitz control 90
12.5 The coarea and cone bounds 91
12.6 Dimensional scaling 92
12.7 Gradient attacks 93
12.8 Transferability 94
12.9 The cost theory 95
12.10 Concentration of measure 98
12.11 Iterated and discrete attacks 99
12.12 The two halves, joined 99
12.13 Historical notes 100

13 Approximate Bridges to Real Models 101
13.1 What the exact theorems assume 101
13.2 Two-sided ε-calibration 102
13.3 The approximate trilemma 103
13.4 Recovering the exact contradiction 103
13.5 The tightening principle 104
13.6 Localization to the transition band 105
13.7 The discrete band constraint 105
13.8 Two-slack separation 106
13.9 The quantitative degradation law 107
13.10 From existence to measure 108
13.11 The reading is where the work is 109
13.12 Historical notes 109

14 Part III: Application and Evidence 111

15 What It Means for AI Safety 113
15.1 The constraint, stated as a triangle 113
15.2 Three relaxations, three cost models 114
15.3 The same menu for defense 117
15.4 Reading a domain: which corner fits 117
15.5 The one empirical unknown 118
15.6 What the theorems do and do not claim 119
15.7 An agenda of open problems 120
15.8 The chapter in one constraint 121

16 J-Space, and Why It Does Not Escape 123
16.1 What J-space is . . . 123
16.2 Two structural facts . . . 124
16.3 The diagonal engine: no intermediate space escapes . . . 125
16.4 The analytic engine: linearity is the liability . . . 128
16.5 Where the two engines meet, again . . . 129
16.6 What an escape would have to look like . . . 129
16.7 What this means for interpretability . . . 129
16.8 What this does not claim . . . 130
16.9 Historical notes . . . 130

17 The Trace Is Not a Window 131
17.1 The construction . . . 131
17.2 Faithfulness forces verbosity . . . 132
17.3 The diagonal, with the model holding the pen . . . 133
17.4 Selection destroys the signal . . . 134
17.5 The analytic engine . . . 135
17.6 What this does not claim . . . 136

18 The Reporter Is Underdetermined 137
18.1 The construction . . . 137
18.2 Underdetermination, without any diagonal . . . 138
18.3 The identifiability criterion . . . 139
18.4 Where the diagonal comes in . . . 140
18.5 The analytic engine . . . 141
18.6 What this does not claim . . . 141

19 Watermarks, Paraphrase, and the Third Corner 143
19.1 The construction . . . 143
19.2 The trilemma . . . 144
19.3 The diagonal, and why its price is low here . . . 146
19.4 The analytic engine, and the cost of an attack . . . 146
19.5 What this does not claim . . . 147

20 Does the Constraint Bind? Measurements on Five Models 149
20.1 What an experiment can and cannot do here . . . 149
20.2 The slack parameter is the quantity to measure . . . 150
20.3 Method . . . 150
20.4 Results . . . 151
20.5 Pythia-410m occupies a corner . . . 151
20.6 What the other two families inherit . . . 152
20.7 Threats to validity . . . 152

21 Part IV: Assessment 153

22 Limitations 155
22.1 The unification claim is not fully supported . . . 155

22.2 Continuity and connectedness of prompt space 155
22.3 Pointwise calibration is stronger than the literature's 156
22.4 The empirical claim rests on five checkpoints 156
22.5 The most valuable missing experiment 156
22.6 Deployment claims are not theorems 156
22.7 What would falsify the thesis 157

23 Conclusion 159
23.1 The claim worth carrying forward 159
23.2 Future work 160

24 Exercises 161
24.1 Preliminaries 161
24.2 The Diagonal 162
24.3 The Classical Limits 162
24.4 Three Trilemmata 163
24.5 From Logic to Analysis 164
24.6 The Geometry of Attacks 165
24.7 Approximate Bridges 166
24.8 What It Means for AI Safety 166
24.9 J-Space 167
24.10 Chain-of-Thought Monitoring 167
24.11 Latent Knowledge 168
24.12 Watermarking 168

25 Appendices 169
25.1 The Formal Corpus 169
25.2 A Notation Glossary 185
25.3 The Two Engines, Side by Side 188
25.4 Annotated Bibliography 188
25.5 Building and Checking This Thesis 190

A thesis submitted in partial fulfilment of the requirements for the degree of Doctor of Philosophy.

1 Abstract

A trustworthy language model should not assert falsehoods with confidence, should not obey instructions hidden in the data it processes, and should be able to report on its own reasoning. Each of these goals has an impossibility result attached to it, and the literature treats those results as separate findings proved by separate methods.

This thesis argues that they are instances of two mathematical schemas. The first is a diagonal schema: a system that can name its own behaviors is forced to a self-referential fixed point, and this single lemma generates Cantor, Gödel, Tarski, and Rice along with the verdict trilemmata for hallucination and for prompt-injection defense. The second is a value-crossing schema: a continuous quantity that changes sign on a connected domain must vanish somewhere, and this generates the truth-boundary results, the geometry of attack surfaces, and the coupling of confidence to correctness.

Both schemas are formalized in Lean 4 and proved from first principles. The accompanying corpus contains 738 declarations across four libraries with no `sorry` and no axioms beyond the three that Mathlib itself uses, and the document is gated so that a numbered claim which is neither proved in the text nor cited to a resolvable declaration fails the build.

The exact theorems assume properties no finite system has. The thesis therefore develops approximate forms in which each hypothesis is relaxed to a tolerance, and then measures that tolerance. On a ladder of five open-weight models the coupling slack falls from 0.225 to 0.075 and the coverage hypothesis rises from 43% to 99% satisfaction, with one documented exception whose training data differs from the rest. The weakest model in the set escapes the trilemma entirely, by never committing to an answer, which is the abstention corner reached through incapacity rather than design.

The conclusion is that these limits tighten as systems become more capable. The hypotheses that make the impossibility theorems apply are the same properties that make a model useful, and a system evades the theorems only by failing to have them.

2 How to read this thesis

Part I is background and may be skipped by a reader who knows Lawvere's fixed-point theorem and its classical instances. Part II is the theoretical contribution and is where the trilemmata are proved. Part III applies the theory and tests it. Part IV states what is wrong with the argument and concludes.

Every code block in this document is elaborated by Lean 4 as the document is built, so the proofs printed in the text are the proofs the kernel accepted. Results proved elsewhere are cited by name to the companion libraries, and Appendix A indexes those libraries file by file.

3 Introduction

A trustworthy language model should not assert falsehoods with confidence. It should not follow instructions that arrive inside the data it was asked to process. It should report on its own reasoning in a way that a monitor can check. Each of these is a reasonable engineering goal, each has a substantial literature proposing mechanisms to achieve it, and each has a published impossibility result showing that some natural formulation of it cannot be achieved at all.

The literature treats these as separate findings. Hallucination bounds come from learning theory and calibration; prompt-injection limits come from security analysis; the obstructions to eliciting latent knowledge come from a counting argument about functions; watermark removal comes from the geometry of paraphrase. They are published in different venues, proved by different methods, and cited by different communities.

This thesis argues that they are not separate.

3.1 Thesis statement

The impossibility results that constrain trustworthy machine learning are instances of two mathematical schemas: a diagonal schema, which produces a self-referential witness whenever a system can name its own behaviors, and a value-crossing schema, which produces a boundary witness whenever a continuous quantity changes sign on a connected domain. Both schemas can be stated and proved in a proof assistant, both can be instantiated to recover the published results as corollaries, and the quantitative form of the second schema makes predictions that measurably tighten as model capability increases.

Three claims, and they are separable. The first is a claim about mathematical structure, and I defend it by exhibiting the schemas and deriving the instances. The second is a claim about verification, and I defend it with a machine-checked corpus that the reader can rebuild. The third is a claim about the physical world, and I defend it with measurements on five language models.

The third claim is the one that can fail, and stating it in a falsifiable form is deliberate. The exact impossibility theorems assume properties no finite system has: perfect calibration, exact coverage, genuine self-naming. If real

systems sat permanently far from those assumptions, the theorems would be elegant and irrelevant. Chapter 6 replaces each exact hypothesis with a slack parameter, and Chapter 13 measures that parameter. If the measured slack were constant across capability, or grew, the applicability claim would be false and I would report it as false.

3.2 What is not claimed

Three disclaimers, stated here because each corresponds to a way the argument could be overread.

I do not claim the schemas are original. Lawvere's fixed-point theorem is from 1969, the intermediate value theorem is older than the calculus that proves it, and the observation that Cantor, Gödel, Turing, and Tarski share a diagonal is standard. What is new is the specific claim that the AI-safety results are further instances, the formal derivations that establish it, and the quantitative bridge in Part III.

I do not claim the results forbid useful systems. Every theorem here is a constraint on a conjunction of properties, and every one of them is escaped by weakening one conjunct. Chapter 8 works out what each escape costs. A theorem that says you cannot have all three of a triple is a design constraint, not a prohibition, and the practical content is in which corner a deployment should choose and why.

I do not claim the impossibilities are the whole story of AI safety. They bound what guarantees are available in principle. They say nothing about how often a system encounters the boundary, how hard the boundary is to find, or what a failure costs. Chapters 6 and 13 supply what quantitative content I can, and Chapter 15 is explicit about the rest.

3.3 Contributions

The contributions of this thesis are as follows.

- *A unified formalization.* Two schemas, stated in Lean 4 and proved from first principles, from which the classical diagonal results and the AI-safety impossibilities follow as instances. The claim that a specific published result is an instance is discharged by a proof, not by an analogy: `no_reflective_verdict` and its relatives are derived from the same lemma that yields Cantor's theorem.

- *A machine-checked corpus.* Four Lean libraries totaling 738 declarations across 94 modules, with no `sorry` and no axioms beyond the three that Mathlib itself uses. Every numbered result in this thesis is either proved in the text, where the proof is elaborated as the document is built, or cited to a declaration in that corpus, and a build gate rejects the document if a citation fails to resolve or a claim is left unbacked.

- *The analytic engine and its trilemma.* A proof that a continuous model on a connected question space cannot be simultaneously faithful, covering, and strictly calibrated, together with its approximate form, in which each exact hypothesis is replaced by a slack parameter and the exact statement is recovered in the limit.
- *Four case studies with distinct load-bearing steps.* J-space probing, chain-of-thought monitoring, eliciting latent knowledge, and watermark detection. Two of the four reach their main results without self-reference, which bounds the diagonal schema's reach and is reported as a limitation of the unification claim rather than concealed.
- *An empirical bridge.* Measurements of the coupling slack on five language models, showing that it falls from 0.225 to 0.075 across a capability ladder, and that the coverage hypothesis the theorems require is satisfied on 43% of paths for the weakest model and 99% for the strongest. The impossibility binds harder as systems improve.

3.4 Structure of the thesis

Part I is background. Chapter 2 introduces the Lean vocabulary and the topology the later arguments use, Chapter 3 develops the diagonal schema, and Chapter 4 recovers the classical impossibility theorems from it. A reader who knows Lawvere's theorem and its classical instances can begin at Chapter 5.

Part II is the theoretical contribution. Chapter 6 states the trilemmata in their combinatorial form, Chapter 7 develops the analytic engine and proves the continuous trilemma, Chapter 8 turns to the geometry of attack surfaces, and Chapter 9 builds the approximate bridges that connect the exact theorems to systems that satisfy their hypotheses only within a tolerance.

Part III is application and evidence. Chapter 10 draws the consequences for deployment. Chapters 11 through 14 are the four case studies. Chapter 15 is the empirical chapter, and it is where the applicability claim is tested.

Part IV closes. Chapter 16 states the limitations, including the assumptions most likely to be wrong and the results that would be affected if they were. Chapter 17 concludes.

3.5 Declaration of collaborative work

Parts of the material in this thesis have appeared in co-authored work, and the division of labor is stated here.

The defense impossibility results of Part II and the geometric analysis of Chapter 8 appeared in *Geometric Limits of Defense Design*, co-authored with S. Munshi, V. S. Narajala, I. Habler, A. Al-Kahfah, K. Huang, and B. Gatto. The Lean formalization in `ManifoldProofs`, the statements and proofs of the three-tier impossibility, and the meta-theorem are mine. The empirical validation harness was built jointly.

The truth-boundary results of Chapter 7 and the experiments of Chapter 15 appeared in the truth-boundary manuscript. The formalization in `HallucinationProofs` and the analysis reported in Chapter 15 are mine.

The expository treatment in Part I is adapted from a book co-authored with K. Huang. The Lean development it presents is mine; the pedagogical framing was developed jointly.

All Lean code in the four companion libraries is my own work. Where a proof follows a known argument, the text says so and names the source.

4 Part I: Background

5 Preliminaries

This chapter fixes vocabulary: how to read the Lean, the set-and-function language of the diagonal engine, the topology the analytic engine needs, and how the development is built and audited.

Two pictures recur. Combinatorial: a system naming its own behaviors cannot be everywhere consistent, because the behavior disagreeing with each name at that name is itself a behavior. Geometric: a continuous quantity negative somewhere and positive elsewhere vanishes in between. The first is the diagonal, the second the intermediate value theorem, and every impossibility here is one of the two in costume.

A reader fluent in Lean should skim to the roadmap, noting `congrFun` and `match` on an existential.

5.1 How to read the Lean 4

The code blocks are the proofs, elaborated as this document builds and checked by the kernel. Definitions here are prefixed `c0_` so they cannot clash with the development.

Types and terms

Lean is built on types. Every term has a type, written `e : T`, checked before the term means anything. `Nat` is the naturals, `Bool` the two-element type, `Prop` the type of propositions.

Definition 0.1 (Type and term). A *type* classifies terms. A *term* `e` *has type* `T`, written `e : T`, when the language's rules assign `T` to `e`. Type-checking verifies such assignments mechanically.

Types have types, and universes form a hierarchy `Type : Type 1 : …` to avoid the paradox of Chapter 1. `Type _` means "some level, inferred," and `Sort` is the umbrella: `Prop = Sort 0`, `Type = Sort 1`.

Example 0.2. `2 : Nat`, `true : Bool`, and `1 + 1 = 2 : Prop`. The first live block asserts the last, proved by `rfl`.

```
example : 1 + 1 = 2 := rfl
```

An `example` is an anonymous check. Named results use `def` for data and `theorem` for proofs; core Lean has no `lemma`.

Functions and application

`f : A → B` applied to `a : A` is written `f a`, by juxtaposition. Application binds tighter than almost everything, so `f a + 1` is `(f a) + 1`.

Definition 0.3 (Function type). For types `A` and `B`, `A → B` is the type of functions from `A` to `B`. Application of `f : A → B` to `a : A` is `f a : B`.

A lambda `fun x => e` has type `A → B` when `e : B` given `x : A`; naming the argument left of the colon is the same thing.

```
def c0_double (n : Nat) : Nat := n + n
def c0_double' : Nat → Nat := fun n => n + n
```

Application computes, and `rfl` accepts it.

```
example : c0_double 3 = 6 := rfl
```

Composition and identity are worth naming, because the diagonal is a claim about a composite.

```
def c0_id {A : Type} (a : A) : A := a
def c0_comp {A B C : Type} (g : B → C)
    (f : A → B) : A → C := fun a => g (f a)
example : c0_comp c0_double c0_double 3 = 12 := rfl
```

Curly braces mark *implicit* arguments, inferred by Lean and never written at a call site.

Currying and the shape A → A → Y

The arrow associates right and application left, so `A → A → Y` is `A → (A → Y)` and `f a b` is `(f a) b`. The conventions make currying invisible. The central object of this thesis has this shape.

Definition 0.4 (Currying). A two-argument function into `Y` is represented curried as `f : A → A → Y`. Then `f a : A → Y` supplies the first argument only, and `f a b : Y` supplies both.

Read `f a` as the *behavior indexed by* `a`, and `f a b` as that behavior at `b`.

```
def c0_add : Nat → Nat → Nat := fun a b => a + b
example : c0_add 2 3 = 5 := rfl
```

Example 0.5. Let `f n m` report whether behavior `n` fires at slot `m`. Then `f n` is the whole behavior and `f n n` is the behavior at its own name. That is the *diagonal*, and Chapter 1 shows it is where self-referential systems break. Both arguments range over the same set, so a name is also a legal input.

∀ and ∃

A proof of `P : Prop` is a term of type `P`, which is why Lean proofs look like programs. Propositions are *proof-irrelevant*: any two proofs of one proposition are definitionally equal, so a theorem's content is its statement plus its axiom dependencies, never the term the elaborator built. `∀ x : A, P x` is a dependent function type, proved with a lambda.

Definition 0.6 (Universal quantifier). `∀ x : A, P x` is the type of proofs that `P a` holds for every `a : A`. A term of it maps each `a` to a proof of `P a`.

```
theorem c0_forall_le : ∀ n : Nat, n ≤ n :=
  fun n => Nat.le_refl n
```

Universal instantiation is application: `h a` proves `P a`. The existential is dual, a pair `⟨a, h⟩` built with the anonymous constructor.

Definition 0.7 (Existential quantifier). `∃ x : A, P x` is the type of proofs that some `a : A` satisfies `P`. A term is a pair `⟨a, h⟩` with `a : A` the witness and `h : P a`.

```
theorem c0_exists_zero : ∃ n : Nat, n = 0 := ⟨0, rfl⟩
```

To use one, take it apart with `match`. Building packs a pair; using unpacks one. The Lawvere proof does one of each. Implication is the function arrow.

```
theorem c0_imp {p q : Prop} (h : p → q) (hp : p) : q := h hp
```

Almost every theorem takes its ambient types implicitly, which is why statements about all sets look like statements about one.

Negation, disjunction, the empty proposition

`¬ P` is definitionally `P → False`, and `False` has no proof.

Definition 0.8 (Negation and the empty proposition). `False` is the proposition with no constructors. For `P : Prop`, `¬ P` is `P → False`. From `False` anything follows, by `False.elim`.

```
theorem c0_not_false : ¬ False := fun h => h
theorem c0_ex_falso {p : Prop} (h : False) : p :=
  False.elim h
```

Inequality is negated equality: to use `h : a ≠ b`, hand it a proof of `a = b` and collect `False`. Every contrapositive result below does this.

Definition 0.9 (Disjunction). `P ∨ Q` is proved by `Or.inl` on a proof of `P` or `Or.inr` on a proof of `Q`, and used by case analysis.

```
example : (1 = 1) ∨ (2 = 3) := Or.inl rfl

theorem c0_or_comm {p q : Prop} (h : p ∨ q) : q ∨ p :=
  match h with
  | Or.inl hp => Or.inr hp
  | Or.inr hq => Or.inl hq
```

Conjunction `P ∧ Q` is dual: built from two proofs, used by projection.

Structures

Definition 0.10 (Structure). A structure is a type whose terms are tuples of named fields. Declaring it yields a constructor and one projection per field.

```
structure c0_Point where
  x : Nat
  y : Nat

def c0_origin : c0_Point := { x := 0, y := 0 }
def c0_origin' : c0_Point := ⟨0, 0⟩
example : c0_origin.x = 0 := rfl
```

One bracket notation builds structures, `∃` proofs, and `∧` proofs, because all three are structures.

```
theorem c0_and_comm {p q : Prop} (h : p ∧ q) : q ∧ p :=
  match h with
  | ⟨hp, hq⟩ => ⟨hq, hp⟩
```

Inductive types

Definition 0.11 (Inductive type). An inductive type is specified by a finite list of constructors, each a function into the type. Its terms are exactly those built by finitely many constructor applications, which is what makes `match` exhaustive.

```
inductive c0_Nat where
  | zero : c0_Nat
  | succ : c0_Nat → c0_Nat
```

A structure is the one-constructor case; `And` is a structure landing in `Prop`.

```
structure c0_And (p q : Prop) : Prop where
  intro ::
  left : p
  right : q
```

The existential's constructor is dependent: it packages an element and a proof about that element.

```
inductive c0_Exists {A : Type} (P : A → Prop) : Prop where
  | intro : (a : A) → P a → c0_Exists P
```

So `⟨a, h⟩` is `intro` applied to a witness and a proof, and `match` is the recursor exposing them. Nothing in the diagonal proof is special notation.

Term mode and tactics

The core arguments are term mode, because a five-line theorem gets a five-line proof with every symbol visible. Tactics appear only for small decidable facts.

Remark 0.12. Both modes produce the same object: a tactic block elaborates to a term and the kernel checks the term. A tactic producing a wrong term yields an error, not a false theorem.

Pattern matching

`match` branches on the shape of an inductive term, binding the pieces, on data and proofs alike.

```
def c0_pred : Nat → Nat
  | 0 => 0
  | n + 1 => n
```

The standard move is `match hf (…) with | ⟨a₀, ha₀⟩ => …`, pulling a witness out of an existential produced by surjectivity.

Dot notation

`h.symm` means `Eq.symm h`, with the namespace found from `h`'s type. Spelling convenience, no logical content.

```
theorem c0_symm {A : Type} {a b : A} (h : a = b) : b = a :=
  h.symm
theorem c0_trans {A : Type} {a b c : A} (h1 : a = b)
    (h2 : b = c) : a = c :=
  h1.trans h2
```

rfl and definitional equality

`rfl` proves an equality whose sides agree by computation. It runs any finite computation but proves no quantified statement: `∀ n, n + 0 = n` is `rfl` because addition recurses on its second argument, and `∀ n, 0 + n = n` is not.

congrFun

Definition 0.13 (congrFun). For `f g : A → B`, `h : f = g` and `a : A`, the term `congrFun h a` has type `f a = g a`. It eliminates an equation between functions by evaluating at a chosen argument.

```
theorem c0_congr {A B : Type} (f g : A → B) (h : f = g)
    (a : A) : f a = g a :=
  congrFun h a
```

This is the hinge of Lawvere's theorem: surjectivity gives an equation between functions, and `congrFun` evaluates both sides at the diagonal point. The converse, function extensionality, costs an axiom, and the core proofs do not need it.

decide

Definition 0.14 (Decidability and `decide`). `P` is *decidable* when an algorithm returns a proof of `P` or of its negation. The tactic `decide` proves a decidable `P` by running that algorithm.

```
example : 2 < 5 := by decide
theorem c0_bool_not : ∀ b : Bool, (!b) ≠ b := by decide
```

The second is the workhorse: no boolean equals its own negation. This single fact drives Cantor, and through Cantor every diagonal result here, by a two-case check needing no axioms.

Remark 0.15. `decide` needs a finite or algorithmically checkable domain. `∀ b : Bool, …` is decidable; `∀ n : Nat, …` is not, and no tactic changes that.

What kernel checking means

Lean has two layers. The *elaborator* turns surface syntax into an explicit proof term and is large and untrusted; the *kernel* checks the term has its claimed type and is a few thousand lines and trusted.

Definition 0.16 (Kernel checking). The kernel *checks* a proof when the trusted type-checker verifies that the elaborated term has exactly the stated type, using the fixed rules of the logic and the axioms the term invokes.

A buggy tactic yields a term that either fails to type-check or type-checks, in which case the theorem holds however it was found. The trusted base is the kernel plus the axioms.

The axiom audit

Definition 0.17 (Axiom audit). `#print axioms name` prints the axioms a proof depends on, collected transitively through every lemma used. An empty list means the result holds in the base logic alone.

The Boolean diagonal results report no axioms; anything routed through the reals inherits `propext`, `Classical.choice`, and `Quot.sound`. This is the precise sense in which Chapter 4 calls the diagonal proof cleaner.

Remark 0.18. The axiom list is part of a theorem's meaning. Two proofs of one statement, one axiom-free and one using choice, are different pieces of knowledge, and this development tracks which is which.

The engine in five lines

```
theorem c0_lawvere {A Y : Type} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g)
    (t : Y → Y) : ∃ y, t y = y :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨f a₀ a₀, (congrFun ha₀ a₀).symm⟩
```

Four moves. Instantiate `hf` at the twisted diagonal. Unpack the `∃` with `match`, binding `a₀` and the function equation. Offer `f a₀ a₀` as the fixed point. Evaluate at `a₀` with `congrFun` and flip with `.symm`. Every core proof below is a variation.

Cantor, live

Specialize: `Y = Bool` and `t` the negation, fixed-point-free by `c0_bool_not`.

```
theorem c0_cantor {A : Type} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  match c0_lawvere f hf (fun b => !b) with
  | ⟨y, hy⟩ => c0_bool_not y hy
```

Suppose `f` names every boolean pattern over its indices. The engine returns a fixed point of negation, which `c0_bool_not` refutes. Identifying a subset with its indicator, universality is exactly a point-surjection of `A` onto its power set. The engine also names the index where the system breaks.

```
theorem c0_liar {A : Type} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) :
    ∃ a, f a a = !(f a a) :=
  match hf (fun a => !(f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Name the twisted diagonal by `a₀` and evaluate there. That is the liar in Boolean clothing, and in Chapter 3 it becomes the query on which a truthful model must hallucinate, the prompt no wrapper neutralizes, and the state a truth probe cannot classify.

5.2 Sets and functions

Definition 0.19 (Function, evaluation). `f : A → Y` assigns each `a : A` a value `f a : Y`. Functions are equal when they agree pointwise; that pointwise agreement suffices is function extensionality.

Definition 0.20 (Composition and identity). For `f : A → B` and `g : B → C`, `g ∘ f : A → C` is `fun a => g (f a)`, and `id` is `fun a => a`. Composition is associative with `id` as two-sided unit.

Definition 0.21 (Injective, surjective, bijective). `f : A → B` is *injective* when `f a = f a'` forces `a = a'`, *surjective* when every `b` is some `f a`, and *bijective* when both.

```
def c0_Injective {A B : Type} (f : A → B) : Prop :=
  ∀ a a', f a = f a' → a = a'
def c0_Surjective {A B : Type} (f : A → B) : Prop :=
  ∀ b, ∃ a, f a = b
```

Example 0.22. Successor on `Nat` is injective, not surjective. `c0_pred` is surjective, not injective. On finite sets the two coincide, which is pigeonhole; its failure on infinite sets is what lets the diagonal say what counting cannot.

Systems and universality

Chapter 1 calls a term of type `A → A → Y` a *system* and reads it as a naming scheme: `a` is a name, `f a` is the behavior it denotes. Both arguments range over `A`, and that coincidence is the entire source of self-reference.

Definition 0.23 (System). A *system* is a function `f : A → A → Y`. Its *diagonal* is `a ↦ f a a`.

Definition 0.24 (Universal, point-surjective). `f : A → A → Y` is *universal*, or *point-surjective*, when every `g : A → Y` equals `f a` for some `a`.

This is "the system can talk about itself," made precise. Where this thesis argues a real system satisfies it, that argument is the substance and the theorem is a formality. Point-surjectivity is weaker than surjectivity of `A → (A → Y)`: it asks only that each behavior be hit, which is all the diagonal needs.

Example 0.25. No finite system is universal once `|Y| ≥ 2`. With `|A| = n` there are `n` behaviors `f a` but `|Y|^n` functions `A → Y`. Counting already forbids it. The diagonal matters because it forbids universality for a reason surviving into the infinite case, where counting is silent.

Definition 0.26 (Indicator). The *indicator* of `S ⊆ A` is the function `A → Bool` true exactly on `S`. Subsets of `A` and functions `A → Bool` correspond bijectively.

```
def c0_iszero : Nat → Bool := fun n => n == 0
example : c0_iszero 0 = true := rfl
example : c0_iszero 3 = false := rfl
```

The liar is then the subset containing a name exactly when its behavior excludes it.

Definition 0.27 (Section). A *section* of `f : A → A → Y` is `s : (A → Y) → A` with `f (s g) = g` for every `g`. It picks a name for each behavior.

Definition 0.28 (Retraction). A *retraction* of `s : A → B` is `r : B → A` with `r (s a) = a`. Its existence makes `s` injective.

```
def c0_Retraction {A B : Type} (r : B → A)
    (s : A → B) : Prop := ∀ a, r (s a) = a
```

Every section yields point-surjectivity. The converse needs choice. Lawvere's theorem holds in section form without choice, which is why the categorical statement is about sections; at the set level the surjection form suffices.

Definition 0.29 (Fixed point, fixed-point-free). A *fixed point* of `t : Y → Y` is a `y` with `t y = y`. `t` is *fixed-point-free* when it has none.

Each impossibility is manufactured by choosing `Y` and a fixed-point-free `t`. On `Bool` the flip has none; on the real interval the complement has exactly one, at `1/2`, and that lone fixed point is the content of the analytic theorem: the forbidden place is the threshold.

Remark 0.30. The parallel is exact. The Boolean flip has no fixed point, so on a naming domain something must be its own negation, which is impossible. The real complement has one, so on a connected domain something must sit exactly at the threshold, which calibration forbids. The fixed-point structure of the outcome map is the dial that switches engines.

5.3 The minimum topology

The analytic half reaches the same boundary object by continuity and connectedness. The facts below are Mathlib's.

We work over $\mathbb{R}$, which is *order-complete*, so closed intervals are connected. Order-completeness is the analytic counterpart of naming richness: it makes the sign-change point exist rather than fall through a gap. Over $\mathbb{Q}$ the setup fails, which is why the theorem is about $\mathbb{R}$.

Definition 0.31 (Open set, topological space). A *topology* on `X` is a family of subsets containing $\varnothing$ and `X` and closed under arbitrary unions and finite intersections. A set is *closed* when its complement is open. In a metric space the opens are unions of balls, so both pictures agree.

Definition 0.32 (Continuity). `h : X → ℝ` is *continuous* when preimages of opens are open; on metric spaces this is the ε-δ condition at every point.

Continuity is the analytic stand-in for the diagonal's structural hypothesis: it forbids teleporting across the boundary instead of crossing it.

Definition 0.33 (Connected space). `X` is *connected* when it is not the union of two disjoint nonempty opens; equivalently its only clopen subsets are `∅` and `X`.

Example 0.34. Every interval of $\mathbb{R}$ is connected. A two-point discrete space is not: each point is clopen. That distinction is exactly the one between the engines.

Remark 0.35. On subsets of $\mathbb{R}$, connected, path-connected, and interval coincide. The thesis states connectedness because the question spaces of the safety readings are not intervals but abstract spaces assumed connected, and one proof then covers every reading.

Theorem 0.36 (Intermediate value theorem). *Let `X` be connected and `h : X → ℝ` continuous. If `h` takes a value below `c` somewhere and above `c` somewhere, then `h x₀ = c` for some `x₀`.*

Nothing here is ours. Every use goes through Mathlib's two-function form `IsPreconnected.intermediate_value₂`, with the interval case `isPreconnected_Icc`. Chapter 4 derives the statement above from those two, as Theorem 4.7.

Remark 0.37 (Why it is true). Suppose `h(a) < 0 < h(b)`. Let `S` be the set of `x ∈ [a, b]` with `h(x) < 0`. It is nonempty and bounded, so it has a supremum `x₀`. Continuity rules out `h(x₀) < 0`, since points to the right would still be negative and `x₀` would not bound `S`.

Remark 0.38. The two theorems are mirror images. Naming produces an index whose behavior contradicts itself; connectedness produces a point where a continuous quantity is pinned to a forbidden value. In both, global richness on the domain forces a specific bad point. `Foundation.F_04` makes the correspondence formal rather than analogical.

Definition 0.39 (Lipschitz map). `h : X → ℝ` is *`L`-Lipschitz* when `dist (h x) (h x') ≤ L · dist x x'`. The constant converts an output gap into a guaranteed input gap.

The bare theorem gives existence only. Lipschitz control upgrades a crossing into a band of computed width, and it is why the analytic engine computes what the diagonal cannot. Chapter 5 is a sustained application.

Definition 0.40 (Boundary object). The *boundary object* is the domain element a proof forces into existence: for the diagonal an index `a₀` with `f a₀ a₀ = t (f a₀ a₀)`; for the intermediate value theorem a point `x₀` where the continuous quantity equals its threshold. In the safety readings it is the query on which a truthful model must hallucinate, the input no filter can classify, and the state no truth probe can place.

It is not an artifact of technique. It is one object seen two ways, and it cannot be designed away, because it is forced by the expressiveness or connectedness that made the system useful.

5.4 The two engines

Engine one, the diagonal. No fixed-point-free `t` admits a universal system. Feed it `Y` and a fixed-point-free `t`; receive an impossibility. Axiom-free in Boolean form and entirely non-quantitative.

Remark 0.41. Its strength is its limit: it cannot give the width of a failure region or a scaling rate, because its only hypothesis is a naming property with no geometry. The separation is clean, with the qualitative claim proved once unconditionally and the quantitative claim separately where geometry exists.

Engine two, the intermediate value theorem. Feed it a connected question space, a continuous model, and a continuous correctness measure taking both signs; receive a boundary point. It needs the reals and classical logic, so its audit is heavier, and in exchange with Lipschitz control it yields band widths, separations, and rates.

The engines locate the same boundary object and part on everything quantitative. Chapter 4 is the hinge, and `Foundation.F_04` records the correspondence. Read with one question in hand: which engine is running, and what may it therefore conclude? A claim of impossibility suggests the diagonal; a claim carrying a number is the analytic engine.

The categorical origin. Lawvere proved the theorem for cartesian closed categories, covering sets, computable functions, and the internal logic of a topos at once, with a section as the hypothesis and composition replacing application. So Cantor, Gödel, Tarski, Turing, and the trilemmata of Chapter 3 are literal specializations rather than analogies. This thesis stays at the set level.

Remark 0.42. Once the engines are isolated, each result is a paragraph plus a few lines of code, and the apparent variety of impossibility theorems collapses to two arguments and a catalogue of instances. A proof that feels long is the part arguing that a real system satisfies the hypotheses, which is the honest work.

5.5 How this is verified and built

The project pins `leanprover/lean4:v4.28.0` with Verso pinned to match, so the live blocks are elaborated by the same Lean that checks the companion libraries. `lake build` elaborates every chapter module, which is the same as checking every live proof.

Remark 0.43. Building is verifying. A reader who distrusts a proof can clone the project, run `lake build`, and watch the kernel accept the printed lines. The first build compiles dependencies and is slow; later builds are incremental.

The chapters split in two. The diagonal chapters are self-contained core Lean with trivial audits. The analytic results need Mathlib and live in the companion developments, cited by name: `HallucinationProofs` for the truth boundary, `Foundation` for the correspondence between engines, `CCHProofs`

for the computability instances, and `ManifoldProofs` for the geometric refinements.

6 The Diagonal

Every impossibility here descends from one move. Assume a system expressive enough to describe its own behavior. Build a description that disagrees with whatever the system does on itself.

The code blocks are core Lean 4 with no libraries, elaborated as this document builds. The technical core is Theorem 1.4 and Corollary 1.10; everything else argues that the single hypothesis of Theorem 1.4 is exactly "the system can describe itself," and that five famous theorems are one theorem read five ways.

6.1 One move, five disguises

The five arguments below were found separately over sixty years. They look different because their subject matter differs: sets, then logic, then computation. Each builds a function computing what the system does to a description and then changing the answer. Each derives its contradiction from the same place: nothing equals its own change.

Cantor (1891). Suppose `f : A → P(A)` hits every subset. Let `D = { a ∈ A : a ∉ f a }`, and let `f a₀ = D`. Is `a₀ ∈ D`?

The uncountability argument is the same with the pool taken to be $\mathbb{N}$. Given a list `r₀, r₁, …` of reals, build one whose `n`-th digit differs from the `n`-th digit of `rₙ`. The system is `n ↦ rₙ`, outcomes are digits, self-application reads the `n`-th digit of the `n`-th real, and the change is a fixed-point-free digit swap.

Russell (1901). Unrestricted comprehension gives `R = { x : x ∉ x }`, and `R ∈ R` holds exactly when `R ∉ R`. This is Cantor's `D` with the indexing removed: the universe plays both roles, `f` is the identity, and the diagonal `a ↦ a ∉ f a` becomes `x ↦ x ∉ x`. It is more violent than Cantor, because it needs no extra assumption; comprehension alone suffices.

The two standard repairs are the two escapes every later chapter uses. Zermelo restricts comprehension, so there is no ambient universe to carve from: this denies universality. Russell's theory of types stratifies levels so `x ∈ x` is ill-formed: this breaks the coincidence of domains in `A → A → Y`, forcing judge and judged into different types so the diagonal cannot form.

Gödel (1931). The same move inside arithmetic, and the performance is what made it hard. Gödel numbered formulas and proofs, then showed "the

proof coded by `p` proves the sentence coded by `n`" is itself arithmetic. Once provability is a predicate `Prov`, arithmetic quotes itself.

The diagonal lemma is this chapter's theorem with `t` negation of provability. Representability is universality: the theory names, and correctly names, properties of its own sentences. Weak theories fail here, which is why incompleteness needs a strength lower bound.

Tarski (1936). Gödel's move aimed at truth. If `True(x)` held exactly of true sentences, the diagonal lemma on "`x` is not true" gives `L` equivalent to "`L` is not true," so `L` is true iff it is not. No such `True` is definable in its own language.

Turing (1936). Suppose `H` decides whether `p` halts on `x`. Build `Dgn` which on input `p` runs `H` on `(p, p)` and does the opposite. Then `Dgn` halts on `Dgn` exactly when `H` says it does not. Self-application is the diagonal; "do the opposite" is the fixed-point-free transformation; universality is the universal machine.

The universal machine deserves its own sentence, since it is the computational form of Definition 1.2. `U` simulates any `p` on any `x`, so `f p` ranges over all computable behaviors. The subtlety is that it reaches only the computable ones.

Kleene. The recursion theorem points the move forwards. For any total computable transformation on programs there is a program behaving exactly like its own transform. Where Turing chose a transformation with no fixed point and derived impossibility, Kleene took the fixed point as a gift.

Lawvere (1969). In any cartesian closed category, a point-surjective map `A → Y^A` forces every endomap `t : Y → Y` to have a fixed point. Contrapositively, a fixed-point-free `t` forbids the point-surjection. Every classical instance is a choice of category, `Y`, and `t`: sets with `Y = Bool` and `t = not` gives Cantor; a syntactic category gives Gödel and Tarski; computable maps give Turing and Rice.

Yanofsky (2003). The readable modern account, and the template for Chapter 2. It strips the category theory to a diagram of sets and writes each paradox as one commuting square with different labels. The recipe becomes mechanical: name the indices, the outcomes, and a transformation with no fixed point, then check universality.

The shared skeleton. Four ingredients and one collapse. The ingredients are a pool of things, a set of outcomes, a way of applying a thing to a thing, and a change on outcomes. In Cantor: elements and subsets, membership values, asking whether an element lies in a subset, swapping the values.

The collapse is that the pool plays both roles in the applying step, so a thing can be applied to itself. That is the only genuine self-reference needed, and it is the coincidence of domains that Definition 1.1 builds in by writing `A → A → Y` with the same `A` twice. Form the self-application map, post-compose with the change, and you have the diagonal description.

6.2 Systems that describe themselves

Fix a domain `A` of *indices* and a set `Y` of *outcomes*. Read an element of `A` as a name, program, prompt, activation, or Gödel number.

Definition 1.1 (System). A *system* is a function `f : A → A → Y`. Read `f a` as the *behavior* named by `a`, and `f a b` as that behavior's outcome on input `b`. The *diagonal* is `a ↦ f a a`.

The domains coincide, and that coincidence is the chapter's whole subject. It lets a behavior be applied to a name of its own kind, in particular its own. In different sets there would be no diagonal and no theorem.

Definition 1.2 (Universality). `f : A → A → Y` is *universal*, or *point-surjective*, when every `g : A → Y` is named: `∀ g, ∃ a, f a = g`.

This is "the system can talk about itself," made precise. A proof-checker handed the code of any predicate on proofs, an in-context learner promptable to any behavior over prompts, a representation encoding any pattern over its own states: each claims universality for the appropriate `A` and `Y`.

Two features matter downstream. First, the quantifier is over *every* `g`, with no restriction. Naming the nice or computable or intended behaviors is not enough; the hypothesis must reach the pathological diagonal behavior too. This is why real systems often escape: a programming language names only computable functions. Chapter 3's hard work is exactly here.

Second, only point-surjectivity is asked: each `g` is `f a` for *some* `a`, with no uniqueness. A behavior may have many names or share one. That low bar is what makes the impossibility strong; you cannot escape by making naming many-to-one.

Currying makes `A → A → Y` and `A → (A → Y)` the same type, so "a system" and "a map from names to behaviors" are interchangeable. Under Curry-Howard a function type is an implication and evaluation is modus ponens, so universality reads as "every behavior is inhabited by a name." That is why the same three-line proof appears in set theory, logic, and type theory texts with only the nouns changed.

Lawvere used a weaker cousin, *weak point-surjectivity*: for every `g` some `f a` agrees with `g` on points. In `Set` the two coincide, so no theorem here is affected, but it is why the categorical statement is more general than it looks. When someone claims a system "is not really universal," the honest question is whether it fails even the weak form, and usually it does, by naming only a restricted class.

The finite case, by counting

Example 1.3. No finite system is universal when `|Y| ≥ 2`. With `|A| = n` there are `n` behaviors `f a` but `|Y|^n` functions `A → Y`, and `n < 2^n ≤ |Y|^n`. Some `g` is unnamed, for lack of room.

The smallest case shows counting and diagonalizing are one fact seen twice. Take `A = {0, 1}`, `Y = Bool`, so there are four functions and `f` supplies two. Set `g 0 = not (f 0 0)` and `g 1 = not (f 1 1)`. Then `g ≠ f 0`,

since they differ at `0`, and `g ≠ f 1`, since they differ at `1`. Pigeonhole says an unnamed function exists; the diagonal points at one.

That is the whole difference, and it is why the diagonal survives into the infinite case. When `A` is infinite there is no size comparison to make without already knowing Cantor's theorem. The diagonal needs no count: it writes down `a ↦ not (f a a)` and checks it differs from `f a` at `a`, uniformly in `a`.

A worked infinite instance. No `f : ℕ → ℕ → Bool` is universal. Define `g n = not (f n n)`. For each `n`, `g` and `f n` disagree at `n`, so `g` is unnamed.

6.3 Lawvere's fixed-point theorem

Theorem 1.4 (Lawvere, 1969). *Let `f : A → A → Y` be universal. Then every `t : Y → Y` has a fixed point.*

```
theorem lawvere {A Y : Type _} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g)
    (t : Y → Y) : ∃ y, t y = y :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨f a₀ a₀, (congrFun ha₀ a₀).symm⟩
```

Proof. Let `d a = t (f a a)`. By universality `f a₀ = d` for some `a₀`. Equal functions agree at every point, so evaluating at `a₀` gives `f a₀ a₀ = d a₀ = t (f a₀ a₀)`. In the Lean term, `congrFun ha₀ a₀` is that evaluation and `.symm` orients the equation. ■

Three moves. First, `d` is not chosen cleverly; `t` and `f` write it, by running the diagonal and post-composing. It is an ordinary function, dangerous only because of what universality forces.

Remark 1.5 (Where the self-application happens). In `f a₀ a₀ = t (f a₀ a₀)` the index `a₀` plays three roles at once: the name of the behavior, the input it runs on, and the point of evaluation. All three coincide because `d` was built from the diagonal. That collapse is the diagonal in one symbol. Every proof below has such an `a₀`; in Chapter 3 it is a query, in Chapter 4 a point of a connected space.

Remark 1.6 (Constructive content). Given the witness for the single function `d`, the fixed point is computed with no further choices: it is `f a₀ a₀`, and the equation is verified by evaluating a known equality. No case split, no excluded middle, no choice. Hence the Lean proof is a plain term, and the boolean instances below need no axioms. Any nonconstructive step later is charged to the reading of `t`, never to Lawvere.

Three misreadings. First, that the theorem says self-reference is paradoxical. It says self-reference *plus a fixed-point-free transformation* is impossible; with a fixed point the identical construction yields Kleene's useful self-referential object. Second, that it needs infinity or set-theoretic strength.

6.4 The section and retract form

Definition 1.7 (Section). A *section* of `f : A → A → Y` is `s : (A → Y) → A` with `f (s g) = g` for every `g`.

A section is a uniform chosen naming; point-surjectivity only promises names exist. Every section gives point-surjectivity; the converse needs choice. The section form holds verbatim in any cartesian closed category, which is how Lawvere stated it.

Theorem 1.8 (Lawvere, section form). *If `f : A → A → Y` has a section, every `t : Y → Y` has a fixed point.*

```
theorem c1_lawvere_section {A Y : Type _} (f : A → A → Y)
    (s : (A → Y) → A)
    (hs : ∀ g : A → Y, f (s g) = g)
    (t : Y → Y) : ∃ y, t y = y := by
  refine ⟨f (s (fun a => t (f a a)))
    (s (fun a => t (f a a))), ?_⟩
  have h := congrFun (hs (fun a => t (f a a)))
      (s (fun a => t (f a a)))
  exact h.symm
```

Proof. Theorem 1.4 with the chosen name in place of the existential one. With `d = fun a => t (f a a)` and `a₀ = s d`, the equation `hs d` says `f a ₀ = d`, and `congrFun` at `a₀` gives `f a₀ a₀ = t (f a₀ a₀)`. Since `s` is given rather than extracted, the proof is a closed term with no case analysis. ■

The section form makes visible that full point-surjectivity was never needed. The proof consumed one name for one behavior.

```
theorem c1_section_universal {A Y : Type _} (f : A → A → Y)
    (s : (A → Y) → A)
    (hs : ∀ g : A → Y, f (s g) = g) :
    ∀ g : A → Y, ∃ a, f a = g :=
  fun g => ⟨s g, hs g⟩
```

Proof. `s g` witnesses the existential, since `f (s g) = g` is `hs g`. ■

Sections and retracts. The condition `f ∘ s = id` says `A → Y` is a *retract* of `A`. Lawvere's theorem is often stated this way: if `Y^A` is a retract of `A`, every endomap of `Y` has a fixed point. Retracts are categorical, defined by one equation and needing no elements, which is why the statement transports.

Remark 1.9 (The cartesian closed statement). A category is *cartesian closed* when it has a terminal object `1`, binary products, and exponentials `Y^A` with evaluation `ev : Y^A × A → Y`. In `Set` these are the one-point set, the product, and the function set. These are exactly the structure the diagonal proof consumes, and no more.

There, `f : A → Y^A` is *weakly point-surjective* when every global point `g : 1 → Y^A` has `a : 1 → A` with `f ∘ a = g` as points. Lawvere's theorem is

that such an `f` forces every `t : Y → Y` to have a fixed point `y` with `t ∘ y = y`. The proof is the diagram our Lean term draws: form the diagonal `A → A × A`, follow with `f × id` and evaluation, post-compose with `t`, transpose back, name the result, and chase it around the square.

Nothing uses that the category is `Set`. This is the sense in which Cantor, Gödel, Turing and the rest are literally one theorem. Cantor lives in `Set`.

6.5 The contrapositive is the impossibility

Corollary 1.10. *If `t : Y → Y` satisfies `t y ≠ y` for all `y`, then no `f : A → A → Y` is universal.*

```
theorem no_universal {A Y : Type _} (t : Y → Y)
    (ht : ∀ y, t y ≠ y)
    (f : A → A → Y) : ¬ (∀ g : A → Y, ∃ a, f a = g) :=
  fun hf => match lawvere f hf t with
    | ⟨y, hy⟩ => ht y hy
```

Proof. Universality would give a fixed point of `t`, contradicting `ht`. ■

Remark 1.11. This is the engine. A specific impossibility needs a reading of `Y` and a fixed-point-free `t`. The classical pairs are in Chapter 2, the AI-safety pairs in Chapter 3.

Two more shapes, quoted downstream. The first denies that any candidate exists, rather than refuting one handed to it.

```
theorem c1_no_universal_exists {A Y : Type _} (t : Y → Y)
    (ht : ∀ y, t y ≠ y) :
    ¬ ∃ f : A → A → Y, ∀ g : A → Y, ∃ a, f a = g :=
  fun ⟨f, hf⟩ => match lawvere f hf t with
    | ⟨y, hy⟩ => ht y hy
```

Proof. Such an `f` would be universal, giving a fixed point of `t`. ■

The second returns the diagonal index rather than the fixed point, which is what most downstream proofs want: they need to point at the offending behavior.

```
theorem c1_lawvere_diagonal {A Y : Type _} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g)
    (t : Y → Y) : ∃ a, f a a = t (f a a) :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Name the diagonal behavior by `a₀` and evaluate the naming equation at `a₀`. ■

Same construction, two returns. When `t` is fixed-point-free the equation is a contradiction; when `t` has a fixed point it is a self-referential program in Kleene's sense. The theorem does not know which case it is in; `t` decides.

Concretely: with `Y = Bool` and `t` the identity, the diagonal gives `f a₀ a₀ = f a₀ a₀`, a tautology and a harmless self-description. Replace `t` by negation and the same index gives `f a₀ a₀ = not (f a₀ a₀)`, which cannot hold. The construction did not change; only the transformation did.

6.6 Three classical instances

Example 1.12 (Cantor). Take `Y = Bool` and `t` the negation. No boolean equals its own negation, checked by exhaustion:

```
theorem bool_not_fpf : ∀ b : Bool, (!b) ≠ b := by decide
```

```
theorem cantor {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  match lawvere f hf (fun b => !b) with
  | ⟨y, hy⟩ => bool_not_fpf y hy
```

`hf` is the assumed surjection, `fun b => !b` the transformation, `lawvere` returns `y` with `(!y) = y`, and `bool_not_fpf` refutes it. That one line carries the entire mathematical content, decided by two-case exhaustion, which is why Cantor for booleans needs no axioms.

The identification with textbook Cantor is definitional, not approximate. A subset `S ⊆ A` is its indicator `χ_S : A → Bool`, so `P(A)` is `A → Bool` and a surjection `A → P(A)` is a universal system `f : A → A → Bool`. Cantor's `D = { a : a ∉ f a }` has indicator `a ↦ not (f a a)`, our diagonal for `t = no t`.

Example 1.13 (Russell). Take `Y` to be propositions and `t` logical negation. The behavior "indices not containing themselves" is the diagonal, and its own membership is fixed by negation, which has no fixed point. This is Russell's paradox as Corollary 1.10 with `t = ¬`. It is stated in prose because the fixed-point-free step for propositions needs a classical principle.

That contrast is the first appearance of axiom-accounting. On `Bool`, negation is a computation and `bool_not_fpf` is `decide`, importing nothing. On propositions there is no finite set to try, and the smooth classical statement leans on two-valuedness.

Proposition 1.14 (The liar, named). The diagonal points at the exact behavior on which the system breaks.

```
theorem liar_query {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) :
    ∃ a, f a a = !(f a a) :=
  match hf (fun a => !(f a a)) with
```

```
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Name `a ↦ !(f a a)` by `a₀` and evaluate at `a₀`. ■

This is `c1_lawvere_diagonal` at `Y = Bool`, `t = not`, and it does not conclude `False`. It returns a live equation describing the pathology: a query the system answers with the opposite of its own answer. Keeping the query rather than collapsing to `False` is what lets Chapter 3 treat the offending input as an object with meaning.

What the flip needs

Whether a fixed-point-free endomap exists is a property of `Y` alone, and it is the true precondition for an impossibility.

```
theorem c1_bool_has_fpf : ∃ t : Bool → Bool, ∀ b, t b ≠ b :=
  ⟨fun b => !b, bool_not_fpf⟩


theorem c1_unit_has_fixed_point
    (t : Unit → Unit) : ∃ u : Unit, t u = u :=
  ⟨(), rfl⟩

theorem c1_no_fpf_unit :
    ¬ ∃ t : Unit → Unit, ∀ u, t u ≠ u :=
  fun ⟨_t, ht⟩ => ht () rfl
```

Proof. `Unit` has one element, which every endomap fixes. ■

Carry the reading forward. A system whose only outcome is ”accept” cannot be caught by the diagonal, because ”accept” is a fixed point of everything. A verifier that can only say yes is trivially consistent about itself.

Manufacturing flips. A fixed-point-free `t` on finite `Y` is a derangement. On two points there is exactly one, the swap, which is boolean negation; on three, the three-cycle, the shape of `c1_optFlip` below; on one, none. A sufficient condition in general is that `Y` carry a free involution, or more weakly a fixed-point-free permutation.

One caution. An exotic `t` is easy to write and yields a worthless impossibility. The honest work is arguing that `t` is forced by the problem rather than chosen for convenience. In Cantor, `t` is negation because membership is two-valued. In Chapter 3 the flips are forced by the meanings of ”hallucinate,” ”inject,” and ”misclassify.”

6.7 A partial-function preview: toward Rice

Model a partial outcome as `Option Y`: `some y` means the behavior returned `y`, `none` that it did not. A partial system is `f : A → A → Option Y`, and the impossibility follows whenever `t` is fixed-point-free on `Option Y`.

The subtlety Rice exploits is that a useful `t` must disturb `none` as well. A transformation fixing `none` would have a fixed point there and the argument would stall. So the flip changes both whether a behavior returns and what it returns.

```
def c1_optFlip : Option Bool → Option Bool
  | none => some true
  | some true => some false
  | some false => none

theorem c1_optFlip_fpf :
    ∀ o : Option Bool, c1_optFlip o ≠ o := by
  intro o
  cases o with
  | none => decide
  | some b => cases b <;> decide
```

Proof. A three-cycle on three concrete outcomes, checked case by case. ■

By Corollary 1.10 no partial system into `Option Bool` is universal. The reading is undecidability of a nontrivial behavioral property: the three-cycle builds a behavior disagreeing with its own classification.

The pure definedness-flip is the cleanest computational instance. Track only whether a behavior halts and let `t` swap the two outcomes. A universal partial system is then exactly a halting decider, and the diagonal behavior is Turing's `Dgn`.

6.8 The engine, named once

Definition 1.15 (Reflective verdict). A *reflective verdict* on `Q` is a universal system `v : Q → Q → Bool`: every pattern of verdicts over `Q` is named by an element of `Q`.

Read `v p q` as the verdict item `p` renders on item `q`. Both are drawn from `Q`, so an item can judge itself and, crucially, can encode a whole judging policy. Universality claims every policy `Q → Bool` is some single item's policy.

Theorem 1.16. *No reflective verdict exists.*

```
theorem no_reflective_verdict {Q : Type _}
    (v : Q → Q → Bool)
    (hv : ∀ g : Q → Bool, ∃ p, v p = g) : False :=
  cantor v hv
```

Proof. Immediate from Example 1.12. ■

Remark 1.17. Theorem 1.16 is verbatim the Hallucination Trilemma, the Defense Trilemma, and the truth-boundary coupling of Chapter 3. Nothing there adds to the engine; only the intended meaning of `v p q` changes. The work is entirely in the reading.

Be blunt about what this means. The mathematics of the AI-safety impossibilities is finished; it is the one line above. Every remaining page of Chapter 3 argues that a particular real system satisfies hv. Those are modeling claims, not mathematical subtleties. If you doubt a Chapter 3 theorem, the place to press is never the diagonal. It is always hv.

The diagonal in a learning system. Let Q be prompts and read v p q as the verdict given to q when behavior has been fixed by prompt p. In-context learning is the claim that a suitable p steers the model to a wide range of verdict policies. If the range is all of them, the model is a reflective verdict.

The diagonal prompt writes itself: the prompt asking for the opposite of the verdict the model would give when steered to evaluate this very prompt. Universality supplies p_0 realizing that policy, and v p_0 p_0 must equal its own negation. The model cannot answer consistently, not through bad training, but because the capability making it universal is inconsistent with a stable answer.

Two caveats keep this honest. A real model ranges over the policies its architecture and training make reachable, not all functions, which is why Chapter 3 argues reachability rather than assuming it. And the diagonal prompt is a construction, not necessarily a sentence a user would type, so part of the modeling is arguing it is within reach of the steering mechanism.

6.9 What the diagonal cannot give

Remark 1.18. The diagonal is silent on everything quantitative: not how many liars there are, not how hard a_0 is to find, not what reaching one costs an adversary. And it requires genuine self-application. When a system instead lives on a *connected* domain with *continuous* maps, the intermediate value theorem takes over and delivers the same boundary object with metric content. That is Chapter 4, and its refinements are Chapter 5.

Three questions it cannot answer, posed here because Chapters 4 and 5 exist to answer them. *How many:* the theorem produces one index and no measure of whether the liar set is a point or a fat region. *How hard to find:* it tells you a_0 exists and nothing about the cost of locating it, which is the whole question against a resource-bounded adversary.

There is also a hypothesis it cannot do without. The engine needs genuine universality, and when a system lacks it the theorem is silent, which is correct: a system that cannot describe itself in full may well be consistent. The interesting cases are those powerful enough that universality is plausible.

6.10 Roadmap

Every later chapter is either an instantiation of the diagonal with a chosen Y and t, or a change of engine when the diagonal's hypothesis is unavailable.

Chapter 2 does the work this chapter deferred: the arithmetization behind Gödel and Tarski, the computability behind Turing and Rice, each as Yanof-

sky's square filled in. Nothing there strengthens the engine; the labor is verifying universality.

Chapter 3 is the payoff. It reads `Q`, `v`, and the boolean flip three ways. The Hallucination Trilemma: a model cannot be faithful, calibrated, and covering at once, because those three make its verdict reflective.

Chapter 4 changes engines, producing a boundary object with metric content. Chapters 5 and 6 make that quantitative through attack-basin geometry and the approximate bridges linking discrete to continuous. Chapter 7 returns to concrete settings with both engines. Chapter 8 develops J-space, where a system carries both a self-application and a topology, so both engines apply at once.

6.11 Historical notes

The unification is Lawvere's *Diagonal arguments and cartesian closed categories* (1969), reprinted with commentary in 2006. It is terse; most readers do better starting with a modern exposition.

Yanofsky's *A universal approach to self-referential paradoxes, incompleteness and fixed points* (2003) is that exposition, working Cantor, Russell, Gödel, Tarski, Turing, and the recursion theorem as instances of one diagram using only sets. It is the template for Chapter 2.

For the originals: Cantor's 1891 note; Russell's 1902 letter to Frege, in van Heijenoort's *From Frege to Gödel* (1967); Gödel (1931), with the diagonal lemma treated cleanly in Boolos, Burgess and Jeffrey, *Computability and Logic*; Tarski (1936); Turing (1936); and Kleene's *Introduction to Metamathematics* (1952) for the recursion theorem.

The reading of these as constraints on learning systems is recent, and the trilemmata of Chapter 3 are, as far as we know, first stated and machine-checked in the companion Lean libraries this thesis tracks.

7 The Classical Limits

Everything here applies `no_universal`: change what `A` names, change what `Y` measures, pick a fixed-point-free `t`, and out falls a theorem proved between 1874 and 1953 by what looked then like a different argument each time.

The claim is a factoring. Each classical proof has a self-referential heart and a local wrapper. The heart is always `a ↦ t (f a a)` and the collision of its name with itself. The wrapper argues the system really is universal: that a set surjects onto its power set, that a theory represents its own provability, that a language is closed under the proof's constructions. Lawvere isolated the heart; this chapter walks the wrappers.

The unification matters beyond elegance. Proved by unrelated tricks, each domain needs its own genius. Proved by one move, a new domain needs only the checklist, and the question becomes mathematical: does this system have a section into its own function space, and does its outcome object carry a fixed-point-free map? That is what lets later chapters treat hallucination and injection as instances to verify.

New results are prefixed `c2_`. Where a theorem needs Mathlib or a classical axiom, it is stated in prose with the verified statement named.

7.1 The recipe

The recipe. To obtain a limitative theorem from `no_universal`:

1. Choose the *index type* `A`: what the names range over. 2. Choose the *outcome type* `Y`: what a behavior reports. For most classical results `Y = Bool`.

2. Exhibit a *fixed-point-free* `t : Y → Y`. For `Bool` this is negation, with `bool_not_fpf` the proof.

3. Argue the *wrapper*: that the system, if it existed as claimed, would satisfy `∀ g : A → Y, ∃ a, f a = g`.

4. Conclude by `no_universal`, or by `liar_query` for a specific self-undermining index.

Steps 1 to 3 are cheap. Step 4 is where the mathematics lives and the only step that differs. Step 5 is one line. For `Y = Bool` the recipe collapses to one statement.

```
theorem c2_no_universal_bool {A : Type _}
    (f : A → A → Bool) :
    ¬ (∀ g : A → Bool, ∃ a, f a = g) :=
  no_universal (fun b => !b) bool_not_fpf f
```

Proof. `no_universal` with the flip and `bool_not_fpf`. ■

For *any* assignment of yes-or-no behaviors to indices, some pattern is unrealized, and it is always `fun a => !(f a a)`. The theorem does not care whether `A` is finite or a proper class, whether `f` is computable, or whether yes and no mean membership, provability, or halting.

Remark 2.1 (Why `Bool` and not `Prop`). On `Bool` the fixed-point-free witness is settled by `decide` and needs no axioms; on `Prop`, reading a system as universal forces classical principles the moment one case-splits on an arbitrary `P`. Keeping the engine on `Bool` makes every compiled proof here axiom-free, and it models a discipline the safety chapters take seriously: a two-valued verdict is a decision, and a decision is where impossibility bites.

A finite rehearsal. Take `A = {0,1}`. There are four behaviors and two names, so one is unnamed, and `d = fun a => !(f a a)` differs from `f 0` at `0` and from `f 1` at `1`. The counting proof knew `4 > 2`; the diagonal never counted, building one unnamed behavior by disagreeing with each `f a` at the input where disagreement is cheap, its own name.

7.2 Cantor's theorem

Identify `S ⊆ A` with its indicator, a bijection between the power set and `A → Bool`, so a family of subsets indexed by `A` is `f : A → (A → Bool)` and surjectivity says every subset occurs.

Definition 2.2 (Power-set surjection). A *power-set surjection* on `A` is `f : A → (A → Bool)` with `∀ g : A → Bool, ∃ a, f a = g`.

By currying this is exactly a universal boolean system. There is no gap to close.

Theorem 2.3 (Cantor, 1891). *No power-set surjection exists.*

```
theorem c2_cantor_powerset {A : Type _} (f : A → (A → Bool))
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  cantor f hf
```

Proof. Chapter 1's `cantor` applies directly. Unfolding: universality names the diagonal subset `d = fun a => !(f a a)`, the set of `a` the `a`-th subset excludes. Let `f a₀ = d` and evaluate at `a₀` to get `f a₀ a₀ = !(f a₀ a₀)`. ■

Here `Y = Bool` and `t` is negation. If some outcome were fixed by `t`, the diagonal set could sit there and the surjection would survive.

Only surjectivity is used. Injective maps `A → (A → Bool)` are abundant; the theorem forbids only covering. The map need not be computable, definable, or continuous, so the argument survives any pathology in `f` — which is exactly why the diagonal is the wrong tool for quantitative questions.

In pure set theory the statement is that no surjection `A → Set A` exists, proved as `CCH.cantor_no_surjection_set` in `CCHProofs/CCH_03_Cantor.lean` by the Russell diagonal, with the indicator form `CCH.cantor_no_surjection`. The `Set`-level proof needs membership decidable enough to case-split; the `Bool` form needs nothing.

The diagonal set is not a paradox. There is nothing self-contradictory about `{a : a ∉ f a}`; it is an ordinary subset, listable whenever `f` is computable, and the theorem says only that it is not one of the `f a`. The paradox appears in Russell, where comprehension forces the family to include every subset, so the diagonal set must be both listed and unlisted.

Say `Y` has the *fixed-point property* if every `t : Y → Y` has a fixed point. Lawvere: if `A` point-surjects onto `A → Y`, then `Y` has it, and Cantor is the contrapositive at `Y = Bool`. Read forward the same theorem builds fixed points, which is how the lambda calculus uses it. The sign of the conclusion is entirely in whether `Y` has a fixed-point-free map.

Cantor's 1874 uncountability proof used nested intervals and completeness, an analytic argument; the 1891 diagonal was the second proof, and its power was generalizing instantly to any set, using only the ability to flip a value. This thesis runs the two styles in the other order.

The theorem iterates: no largest set, no set of all sets, and a cumulative hierarchy whose beth numbers are a diagonal step apart. It leaves open whether a cardinality sits strictly between a set and its power set, and Cohen's forcing showed the continuum hypothesis is independent of `ZFC` — a second-incompleteness-flavored fact one level up. The diagonal opens the tower and does not space the rungs.

Schröder–Bernstein builds a bijection from two injections; Cantor builds nothing, exhibiting an obstruction and stopping. This thesis is about the second kind, and when a safety result says "no system can do this," it is Cantor-shaped.

Remark 2.4 (Cantor and reflective verdicts). A model's set of possible behaviors is its power set, so asking a model to certify every behavior of its own kind is asking for the surjection Cantor forbids, however large the model.

7.3 Russell's paradox

Comprehension says every property carves out a set. Applied to "is not a member of itself," `R = {x | x ∉ x}` gives `R ∈ R` iff `R ∉ R`. Cantor needs a codomain and a claimed surjection; Russell drops the codomain and works with one binary relation. Historically Russell found it by pushing on Cantor's

proof: a universe of all sets would be surjected onto its power set by the identity.

Definition 2.5 (Naive membership). A *naive membership relation* on `A` is `mem : A → A → Bool` satisfying comprehension, `∀ g : A → Bool, ∃ a, mem a = g`.

Theorem 2.6 (Russell, 1902). *If `mem` is a naive membership relation then some `r` satisfies `mem r r = !(mem r r)`. No such relation exists.*

```
theorem c2_russell {A : Type _} (mem : A → A → Bool)
    (hmem : ∀ g : A → Bool, ∃ a, mem a = g) :
    ∃ r, mem r r = !(mem r r) :=
  liar_query mem hmem
```

Proof. `liar_query` verbatim with `f` renamed. Comprehension on `fun a => !(mem a a)` names a set `r`; evaluating at `r` gives the collision. Feed it to `bool_not_fpf` for outright `False`. ■

Again `Bool` and negation, and the Russell set is the diagonal behavior — the same object that in Cantor was the diagonal subset and in Chapter 1 the liar.

The hypothesis is comprehension, and the paradox is what forced set theory to give it up. Zermelo's separation carves subsets only from a set already given, so the diagonal reproves Cantor rather than exploding. The type-theoretic repair, the one Lean uses, stratifies by universe level so `mem a a` is ill-formed. In Lean the naive relation is well-typed, which is why `c2_russell` compiles, but `hmem` is never satisfiable for nontrivial `A`: the theorem proves its hypothesis empty.

Remark 2.7 (`Bool` versus `Prop`, again). The propositional paradox needs no axioms for `¬ (P ↔ ¬ P)`. The `Bool` version is compiled because its flip composes with the engine using only `congrFun` and `decide`, whereas `Prop`-valued behaviors invite classical case-splits downstream. When a safe-or-unsafe judgment is genuinely two-valued, the impossibility is constructive, which matters when a machine computes the judgment.

Frege's *Grundgesetze* derived arithmetic from logic using Basic Law V, comprehension in disguise, and Russell's two lines refute it. The paradox is why set theory has axioms rather than a single comprehension schema. Three repairs survived: Zermelo's separation with foundation banning `x ∈ x`; Russell's ramified types, ancestor of Lean's universe hierarchy; and Quine's New Foundations, restricting comprehension to stratified formulas.

Cantor's paradox is the diagonal against the set of all sets, Burali-Forti against the set of all ordinals. Each is `c2_russell` with a different reading of `mem`, and each was historically a separate shock.

7.4 Gödel's first incompleteness theorem

Cantor and Russell hand you universality almost free, because a power set or a comprehension schema is a surjection by fiat. Arithmetic does not. Running

the diagonal against a formal theory needs first that the theory talk about its own syntax, and that is substantial number theory.

We separate the diagonal, which compiles, from the arithmetization, which we describe and cite. The self-reference is finite and mechanical and belongs in the kernel. The arithmetization is a long verification the literature carries out.

Representability as point-surjectivity

Fix a consistent `T` in the language of arithmetic, strong enough to represent primitive recursive functions, and let `⌜φ⌝` be Gödel codes.

Definition 2.8 (Representability). A predicate `g` on codes is *representable* in `T` when some formula `φ` has `T ⊢ φ(n)` whenever `g n` holds and `T ⊢ ¬ φ(n)` whenever it fails. Gödel's central arithmetical fact is that every recursive predicate is representable in `Q`.

Representability is the wrapper's surjectivity. Read `f a b` as "the formula coded by `a`, applied to `b`, is provable." Then representability says some code `a` has `f a = g`, which is point-surjectivity restricted to recursive `g`. That restriction is the one honest difference from Cantor, and it suffices, because the diagonal behavior we build is itself recursive.

Arithmetizing syntax

The language speaks of numbers, not formulas. Gödel encodes each syntactic object as a number, so statements about formulas become statements about numbers. Assign a number to each symbol, code a formula by prime factorization of its symbol sequence, and code a proof by the sequence of its lines.

The delicate step is coding finite sequences of unbounded length in a language whose variables range over single numbers, solved by Gödel's beta function via the Chinese remainder theorem. With sequences codable, primitive recursion is expressible, and "`p` codes a proof of `n`" is representable in `Q`. Then `Prov(n)` is "there exists `p` coding a proof of `n`," one unbounded existential over a primitive recursive matrix.

Lemma 2.9 (Diagonal lemma, abstract form). *For universal `f : A → A → Y` and any `t : Y → Y`, some `a` has `f a a = t (f a a)`.*

```
theorem c2_diagonal_lemma {A Y : Type _} (f : A → A → Y)
    (hf : ∀ g : A → Y, ∃ a, f a = g) (t : Y → Y) :
    ∃ a, f a a = t (f a a) :=
  match hf (fun a => t (f a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Universality names `fun a => t (f a a)` by some `a₀`; evaluate at `a₀`. ∎

This is `lawvere` reorganized to expose the index rather than the fixed value. In the arithmetical reading `A` is codes of one-variable formulas, `f a b`

is provability of the diagonalization, and `t` is the operation the target property performs. The genuine Gödel diagonal lemma, producing an honest arithmetic sentence, is the arithmetization of this step; it is a theorem of `Q`, developed in the model-theory literature and formalized in Mathlib's first-order logic library.

The Gödel sentence

Apply the diagonal lemma to "not provable" to get `G` with `T ⊢ G ↔ ¬ Prov(⌜G⌝)`: a sentence saying, correctly, "I am not provable in `T`."

Theorem 2.10 (Gödel I, 1931). *Let `T` be consistent, recursively axiomatized, and representationally strong. Then `T` proves neither `G` nor `¬ G`. If `T` is sound, `G` is true.*

Proof (structure). If `T ⊢ G` then `T ⊢ Prov(⌜G⌝)`, and the fixed point gives `T ⊢ ¬ Prov(⌜G⌝)`, contradicting consistency. If `T ⊢ ¬ G` then `T ⊢ Prov(⌜G⌝)`; under omega-consistency, or by the Rosser trick, the absence of an actual proof again yields a contradiction. If `T` is sound then `G`, being unprovable, is what it says it is. ■

The collision at the centre is Lemma 2.9. A consistent `T` that could decide every sentence would have a total correct boolean system over its own sentences, and the diagonal with `t` negation produces a sentence equal to its own negation.

```
theorem c2_godel_semantic {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) : False :=
  match c2_diagonal_lemma f hf (fun b => !b) with
  | ⟨a₀, ha₀⟩ => bool_not_fpf (f a₀ a₀) ha₀.symm
```

Proof. Lemma 2.9 with the flip, refuted by `bool_not_fpf`. ■

What it does and does not say. It says a boolean system universal over its own indices is contradictory. Arithmetically, universality is representability plus completeness: representability gives the surjection onto recursive behaviors, and completeness promises a definite provable truth value for every self-referential sentence, so the diagonal behavior lands back in the represented family. The compiled theorem is the semantic core: no consistent theory is both representationally rich and complete.

The hypotheses. Consistency makes the collision fatal rather than odd; an inconsistent theory proves everything, so the flip effectively has a fixed point. Recursive axiomatizability makes `Prov` representable, so the diagonal behavior lands in range; drop it and a theory can be complete, as the full theory of the standard model is, at the price of not being listable. Representational strength is point-surjectivity itself.

Rosser's refinement. Gödel needed omega-consistency to rule out `T ⊢ ¬ G`. Rosser removed it in 1936 with a sentence saying "for every proof of me there is a shorter proof of my negation." Still a fixed point, so still Lemma 2.9 arithmetized, but the operation now compares proof lengths, making plain

consistency enough. The upgrade costs nothing in the engine; it refines the wrapper by choosing a diagonal behavior fatal under a weaker hypothesis.

The epistemic sting. G is unprovable in T and, if T is sound, true. So the metatheory knows G while T cannot prove it. There is no contradiction: our knowledge uses soundness of T, a fact living outside T that T cannot establish about itself.

Concrete unprovable truths. The dismissal that G is artificial no longer holds. Goodstein's theorem, about sequences that appear to grow without bound but always reach zero, is true and independent of PA, provable only by transfinite induction to ε_0. Paris–Harrington is another.

Remark 2.11 (The safety reading). Gödel is the template for every later result of the form "a system powerful enough to model itself cannot certify itself." A model outputting calibrated confidence in its own future outputs, a verifier verifying verifiers including itself, an overseer auditing systems as capable as it is: each is a Prov-like predicate turned on its own domain. The engineering temptation is to believe more capability closes the gap. Gödel says stronger representation only sharpens the diagonal.

7.5 Gödel's second incompleteness theorem

The first theorem produces an undecided sentence; the second identifies one, the theory's own consistency $\mathsf{Con}(T) := \neg\, \mathsf{Prov}(\ulcorner 0 = 1 \urcorner)$.

Theorem 2.12 (Gödel II, 1931). *A consistent, recursively axiomatized T whose provability predicate satisfies the Hilbert–Bernays–Löb derivability conditions does not prove Con(T).*

The conditions make Prov behave like real proof inside T:

1. If $T \vdash \varphi$ then $T \vdash \mathsf{Prov}(\ulcorner\varphi\urcorner)$. Proofs are internally recognized. 2. $T \vdash \mathsf{Prov}(\ulcorner\varphi \to \psi\urcorner) \to (\mathsf{Prov}(\ulcorner\varphi\urcorner) \to \mathsf{Prov}(\ulcorner\psi\urcorner))$. Internal modus ponens. 3. $T \vdash \mathsf{Prov}(\ulcorner\varphi\urcorner) \to \mathsf{Prov}(\ulcorner\mathsf{Prov}(\ulcorner\varphi\urcorner)\urcorner)$. Internal recognition of recognition.

Proof (structure). The conditions give $T \vdash \mathsf{Con}(T) \to G$: internal consistency lets the theory carry out, about itself, the argument that G is unprovable, which is what G asserts. But $T \nvdash G$. So $T \nvdash \mathsf{Con}(T)$. ■

The second theorem, machine-checked

What can be checked here is the derivability-conditions argument. What cannot is the arithmetization: that a theory such as PA really admits a provability predicate satisfying the conditions and the fixed point. That is a large formalization this development does not carry out. What follows is the theorem relative to those inputs, which is how the derivability presentation states it.

```
structure c2_PC where
  S : Type
```

```
  Thm : S → Prop
  imp : S → S → S
  box : S → S
  bot : S
  mp : ∀ {a b}, Thm (imp a b) → Thm a → Thm b
  ax_K : ∀ a b, Thm (imp a (imp b a))
  ax_S : ∀ a b c,
    Thm (imp (imp a (imp b c)) (imp (imp a b) (imp a c)))
  D1 : ∀ {a}, Thm a → Thm (box a)
  D2 : ∀ a b, Thm
      (imp (box (imp a b)) (imp (box a) (box b)))
  D3 : ∀ a, Thm (imp (box a) (box (box a)))
```

Two derived rules from K, S, and modus ponens, the only propositional reasoning needed.

```
theorem c2_s_comb (P : c2_PC) {a b c : P.S}
    (h₁ : P.Thm (P.imp a (P.imp b c)))
    (h₂ : P.Thm (P.imp a b)) :
    P.Thm (P.imp a c) :=
  P.mp (P.mp (P.ax_S a b c) h₁) h₂

theorem c2_syll (P : c2_PC) {a b c : P.S}
    (h₁ : P.Thm (P.imp a b)) (h₂ : P.Thm (P.imp b c)) :
    P.Thm (P.imp a c) :=
  c2_s_comb P (P.mp (P.ax_K (P.imp b c) a) h₂) h₁
```

The hypotheses `fix₁` and `fix₂` are the two directions of the fixed point `g ↔ (Prov(g) → A)`, supplied rather than constructed.

Theorem 2.12a (Löb, 1955). *If `T ⊢ Prov(A) → A` then `T ⊢ A`.*

```
theorem c2_loeb (P : c2_PC) {A g : P.S}
    (fix₁ : P.Thm (P.imp g (P.imp (P.box g) A)))
    (fix₂ : P.Thm (P.imp (P.imp (P.box g) A) g))
    (h : P.Thm (P.imp (P.box A) A)) :
    P.Thm A :=
  let s3 := P.mp (P.D2 g (P.imp (P.box g) A)) (P.D1 fix₁)
  let s5 := c2_syll P s3 (P.D2 (P.box g) A)
  let s7 := c2_s_comb P s5 (P.D3 g)
  let s8 := c2_syll P s7 h
  P.mp s8 (P.D1 (P.mp fix₂ s8))
```

Proof. Necessitate the fixed point and push the box inward with `D2`, giving `Prov(g) → Prov(Prov(g) → A)`. A second `D2` gives `Prov(g) → (Prov(Prov(g)) → Prov(A))`, and `D3` contracts the boxes to `Prov(g) → Prov(A)`. Composing with the hypothesis gives `Prov(g) → A`, the right-hand side of the fixed

point, so g is a theorem; necessitating g and applying modus ponens delivers A. ■

```
def c2_Con (P : c2_PC) : P.S := P.imp (P.box P.bot) P.bot

theorem c2_godel_second (P : c2_PC) {g : P.S}
    (fix₁ : P.Thm (P.imp g (P.imp (P.box g) P.bot)))
    (fix₂ : P.Thm (P.imp (P.imp (P.box g) P.bot) g))
    (hcon : P.Thm (c2_Con P)) :
    P.Thm P.bot :=
  c2_loeb P fix₁ fix₂ hcon
```

Proof. Con unfolds to Prov(⊥) → ⊥, the hypothesis of Löb at A := ⊥. ■

Read contrapositively, this is Theorem 2.12. One check is worth doing, since a theorem whose hypotheses cannot be met is true for empty reasons: the axiom set is satisfiable.

```
def c2_pc_trivial : c2_PC where
  S := Unit
  Thm := fun _ => True
  imp := fun _ _ => ()
  box := fun _ => ()
  bot := ()
  mp := fun _ _ => trivial
  ax_K := fun _ _ => trivial
  ax_S := fun _ _ _ => trivial
  D1 := fun _ => trivial
  D2 := fun _ _ => trivial
  D3 := fun _ => trivial
```

This witness is deliberately degenerate: it proves everything, including ⊥, so it is an inconsistent theory. It establishes only that c2_PC is inhabited, so the theorems are not vacuous. It does *not* establish that a consistent theory meets the conditions. That claim is the arithmetization, and it remains the one thing here taken from the literature rather than checked.

Löb's theorem is itself a diagonal, the fixed point of ψ ↦ (Prov(⌜ψ⌝) → φ), and the second incompleteness theorem is the case φ = 0=1, because Con(T) is exactly Prov(⌜0=1⌝) → 0=1 waiting for Löb to refuse it.

Why the arithmetization stays in prose. The derivability conditions are statements about a specific arithmetized Prov, and verifying them is the deferred arithmetization plus the third condition, which internalizes the first. There is no faithful short core-Lean rendering, because the content is the syntactic behavior of a particular predicate, not the abstract diagonal. The abstract diagonal is already compiled as Lemma 2.9.

Remark 2.13 (Self-certification is the second theorem). This is the sharpest classical statement of a limit recurring throughout AI safety: a system cannot

vouch for its own soundness from within. "This model will not deceive you" is a Con-like sentence, and a model strong enough to state it and satisfying the internal-coherence analogues cannot prove it without already being unsound. Chapter 7 argues that a deployed system's own assurances carry no evidential weight beyond an external check, and that the external check faces its own second theorem one level up.

Provability as a modal logic. Both theorems package cleanly in GL, where the box reads "provable in T." Condition 1 is necessitation, condition 2 is axiom K, condition 3 is axiom 4. Löb's theorem is the axiom GL. Its frames are exactly the transitive, converse-well-founded ones, and converse well-foundedness is the modal shadow of consistency: a chain of ever-stronger provability claims must terminate.

Consistency strength. Because T cannot prove its own consistency, a theory proving Con(T) is strictly stronger, and "proves the consistency of" well-orders the natural theories from weak arithmetic up through large cardinals. Ordinal analysis measures the steps, assigning PA the ordinal ε_0. The picture to keep is a ladder no rung of which can see its own footing, the exact shape Chapter 7 gives to a tower of overseers.

7.6 Tarski's undefinability of truth

A truth predicate satisfies True(⌜φ⌝) ↔ φ for every sentence, Tarski's schema T.

Definition 2.14 (Definable truth predicate). A language interpreting arithmetic has a *definable truth predicate* when some formula True(x) satisfies schema T for every sentence of the language.

Material adequacy plus definability makes the verdict system universal over its own sentences: every boolean condition the language can state, including the negation of the predicate composed with the diagonal, is some sentence's verdict pattern.

Theorem 2.15 (Tarski, 1936). *No language that can represent its own syntax and interpret arithmetic has a definable truth predicate.*

```
theorem c2_tarski {A : Type _} (True? : A → A → Bool)
    (hT : ∀ g : A → Bool, ∃ a, True? a = g) : False :=
  no_reflective_verdict True? hT
```

Proof. Such a predicate is a reflective verdict, and no_reflective_verdict says none exists. Unfolding, the diagonal builds the liar L with True?(⌜L⌝) ↔ ¬ True?(⌜L⌝), and adequacy turns that into L ↔ ¬ L. ■

Identifying Y and t. Bool and negation. The witness is the liar, which is liar_query read as a sentence rather than a set or a program. The liar is not a curiosity at the edge of the theory; it is the exact obstruction.

Tarski versus Gödel. Gödel's Prov *is* definable, because provability is recursively enumerable, and the price is incompleteness: the predicate exists

but the theory cannot decide the sentence it produces. Tarski's `True` would have to be definable to run the same diagonal, and the conclusion is that it is not definable at all, because if it were the liar would be an outright contradiction rather than an undecided sentence. Provability is a shadow of truth the system can cast; truth itself it cannot hold. The gap between them is the room the first theorem lives in. Cleanly: if truth were definable it would be a complete sound provability predicate, which `c2_godel_semantic` already refuted.

The hierarchy of metalanguages

Truth for `L` is definable, just not in `L`: it is definable in a richer `L'` with the resources to quantify over `L`'s sentences. Truth for `L'` needs `L''`, and so on up an unbounded hierarchy, each level defining truth for the one below and none its own. This is the consistency-strength ladder again, and not by coincidence: both are the second-theorem shape.

There are ways to buy a language its own truth predicate, and each pays a price the recipe predicts. Kripke's 1975 theory makes the predicate *partial*, undefined on ungrounded sentences like the liar, built as the least fixed point of a monotone operator; the liar never settles, and adequacy holds only for grounded sentences. This dodges Tarski by refusing totality, exactly the move separating halting from Rice: let the deciding object withhold judgment and the flat impossibility relaxes into a statement about where judgment must be withheld.

Remark 2.16 (Truth probes and Chapter 8). A truth predicate that is total, exact, and expressible in the system whose outputs it judges is exactly what Tarski forbids. Chapter 8 studies probes reading internal state to estimate whether a model "believes" its output, and the standing objection is Tarskian. The resolution is not to defeat the theorem but to change the domain: from outputs to activations, from a two-valued verdict to a continuous score on a connected space, where the intermediate value theorem takes over.

7.7 Turing and the halting problem

Definition 2.17 (Halting decider). A *halting decider* for a universal system `f : A → A → Bool` is a total `D : A → Bool` intended to satisfy `D a = f a a`, where `f a a` records the halting behavior of program `a` on its own code.

Theorem 2.18 (Turing, 1936). *For every universal `f` and every candidate `D` there is a program `a` with `f a a ≠ D a`.*

```
theorem c2_turing_halting {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) (D : A → Bool) :
    ∃ a, f a a ≠ D a :=
  match hf (fun a => !(D a)) with
  | ⟨a₀, ha₀⟩ =>
    ⟨a₀, fun h =>
```

```
      bool_not_fpf (D a₀) ((congrFun ha₀ a₀).symm.trans h))
```

Proof. Universality names the contrarian behavior `fun a => !(D a)` by `a₀`. Evaluating at `a₀` gives `f a₀ a₀ = !(D a₀)`. If the decider were right there we would get `!(D a₀) = D a₀`, refuted by `bool_not_fpf`. ■

The contrarian program is the classical construction: run `D` on your own code, then loop if `D` says halt and halt if `D` says loop. Same `a₀` as everywhere else, now reading `Y` as halts-or-loops and `t` as "do the opposite."

The partiality subtlety. A real decider must be *total*, while the programs it judges are *partial*. The contrarian program is partial in exactly the way that breaks a total decider. If deciders could be partial the theorem would not follow in this form, and the correct statement becomes Rice's, about total decision of a nontrivial semantic property.

The universal machine is the wrapper. The hypothesis `∀ g, ∃ a, f a = g` is the existence of a universal Turing machine, and Turing built one. Without it there is no closure under diagonalization, and halting for a non-universal model can be decidable; for finite automata it is trivial. Undecidability is a property of systems rich enough to host their own interpreter.

Where halting sits. Halting is the canonical undecidable problem, complete for the r.e. sets under many-one reduction. It is Σ_1, one existential over a decidable matrix, the same shape as `Prov`, which is no accident: `Prov` is halting for a proof search.

Remark 2.19 (Runtime verification). No static analyzer decides termination for all programs. For AI systems the behavioral questions are far harder, whether a model will ever produce disallowed content or an agent ever take an unsafe action, and halting is the floor. The safety chapters do not lean on Turing directly, because they want quantitative statements, but Turing is the qualitative backstop saying the quantitative work is necessary.

7.8 Rice's theorem

A property is *semantic* when it depends only on the function a program computes, and *nontrivial* when some programs have it and some do not. Rice sweeps the class in one argument.

Definition 2.20 (Semantic decider). For universal `f`, a *semantic decider* is a total `D : A → Bool` claiming to compute `f a a` from `a` alone.

Theorem 2.21 (Rice, diagonal core). *For every universal `f` and every `D`, some `a` has `f a a ≠ D a`.*

```
theorem c2_rice_diagonal {A : Type _} (f : A → A → Bool)
    (hf : ∀ g : A → Bool, ∃ a, f a = g) (D : A → Bool) :
    ∃ a, f a a ≠ D a :=
  c2_turing_halting f hf D
```

Proof. `c2_turing_halting` under a change of reading. ■

That Rice's core proof is literally Turing's is the chapter's thesis made blunt. Turing decides one property, Rice any property, and the diagonal does not notice, because it only ever sees a boolean `D` and flips it. What distinguishes them is entirely the wrapper.

Theorem 2.22 (Rice, full form). *Let `P` be a nontrivial semantic property of partial recursive functions. Then `{e | φ_e ∈ P}` is not recursive.*

Proof (structure). Nontriviality gives a program with the property and one without; without loss the everywhere-undefined program lacks `P`. Given a claimed decider, reduce halting to it: from any program and input build a program behaving like the witness if the input halts and like the empty program otherwise, so it has `P` exactly when the input halts. A decider for `P` would decide halting. ■

This needs an acceptable numbering with the `s-m-n` theorem and a case analysis on which side of `P` the empty function falls, using classical logic and Mathlib's recursion theory. The companion development formalizes the core and the safety corollaries in `Foundation/F_02_RiceTheorem.lean`: `Foundation.rice_diagonal` is Theorem 2.21 in mismatch form, `Foundation.rice_general` is the property version for nontrivial `P : Bool → Bool`, and `Foundation.no_automated_self_test` is the reading we want.

Identifying `Y` and `t`. `Bool` and negation. What is new is the quantifier over properties, and the recipe absorbs it: each nontrivial property yields a boolean `D` through the reduction, and one fixed-point-free `t` defeats them all.

The hypothesis. Nontriviality is load-bearing, the analogue of surjectivity being nondegenerate. A trivial property is decided by a constant function and there is no contradiction, because the constant decider is never wrong. The diagonal needs two outcomes to flip between.

Rice–Shapiro and the shape of the undecidable

Rice–Shapiro says a semantic property has an r.e. index set only if it is determined by finite information: a function has it exactly when some finite subfunction does, and every extension keeps it. Properties violating this, "is total," "computes a constant," are not even r.e.

That finer picture is what the safety chapters need, because it locates the line between checkable and uncheckable. A property depending only on finitely much observed behavior can be recognized when it appears though its absence never confirmed, which is the logical form of "you can catch a model doing something bad but cannot certify it never will." A property depending on the whole infinite behavior cannot even be recognized. Rice–Shapiro is the classical statement that monitoring is possible and certification is not.

Remark 2.23 (No automated semantic testing). A property of the function a model computes, whether it is honest or aligned or safe, is semantic, and Rice says no general automated test decides it for universal systems. `no_automated_self_test` is exactly this. A benchmark is an attempt to decide a

semantic property by running finitely many inputs, and Rice is why a benchmark can only sample, never certify.

7.9 The categorical unification

In a cartesian closed category a system is a map `f : A → Y^A`, and the cleanest hypothesis is a *section* `s : Y^A → A` with `f ∘ s = id`. Lawvere's theorem is that any `t : Y → Y` then has a fixed point `y : 1 → Y` with `t ∘ y = y`. The proof is the same diagonal drawn as a commuting diagram: precompose with `A → A × A`, transpose, apply `t`, and use the section to find the self-referential point.

Each classical theorem is Lawvere's theorem in a *particular* cartesian closed category, not merely an argument shaped like it. Cantor is sets with `Y = 2`. Turing and Rice live in a category of assemblies or a realizability topos, where maps are tracked by programs and the section is a universal machine.

We do not compile the categorical version, since doing it honestly needs a category theory library and the payoff is conceptual. The value of knowing the source is that it tells you when a new impossibility will be an instance: whenever the system is a map into an internal function space with a section, and the outcome object carries a fixed-point-free map, only the reading remains.

7.10 Interlude: the wrapper is the mathematics

The engine is three lines precisely because a century of work moved all the difficulty into the wrappers, and the wrappers are hard. Coding syntax into arithmetic, building a universal machine, constructing an acceptable numbering with `s-m-n`, establishing the derivability conditions: each is a substantial theorem, and each is what a graduate course actually spends its time proving. The diagonal is the punchline, and punchlines are short.

The consequence for the rest of this thesis is that to prove a new impossibility you never re-prove the diagonal. You argue that a model with such-and-such capabilities is universal over the relevant behaviors, that a defense must answer every injection including those about itself, that a probe is asked to be a total exact predicate on the states whose truth it reports. When that argument succeeds the impossibility is immediate; when it fails, the failure is exactly the escape hatch.

There is a quieter lesson. Because the engine is axiom-free and compiles while the wrappers are cited, this development is honest about its own confidence in a way informal mathematics rarely is. The parts we are certain of are checked by the kernel; the parts depending on a large library are named so you can find the verified statement.

7.11 What varies, and what does not

| Theorem | `A` (indices) | `Y` | `t` | Wrapper (universality is…) | |—|—|—|—|—| | Cantor | a set | `Bool` | `!` | a power-set surjection | | Russell | a universe of sets | `Bool` | `!` | unrestricted comprehension | | Gödel I | codes of formulas | `Bool` | `!` | representability plus completeness | | Gödel II | codes of formulas | `Bool` | Löb's map | derivability conditions | | Tarski | codes of sentences | `Bool` | `!` | a definable truth predicate | | Turing | programs | `Bool` | `!` | a universal language | | Rice | programs | `Bool` | `!` | reduction from a nontrivial property |

The outcome type is `Bool` in every row and the flip is negation in every row but one, where Löb's map is negation in modal disguise. The index type and the wrapper are where the mathematics sits, and they are what one actually learns from these results. The diagonal is not what you learn; it is what you already knew from the first one.

The compiled fifth column is always one of `no_universal`, `cantor`, `liar_query`, `no_reflective_verdict`, `c2_diagonal_lemma`, or `c2_rice_diagonal`, and each is Theorem 1.4 under a fixed point of negation. Chapter 3 adds an eighth and ninth row and no new machinery.

One honest caveat bridges to the second half. Every theorem in the table is a flat impossibility, silent on how close you can come, how many counterexamples exist, or what finding one costs. The diagonal is a yes-or-no instrument because `Y` is `Bool` and `t` has no fixed point at all.

A last observation. In every classical instance universality was granted almost by definition of the object: a power set is all subsets, a language is closed under its own interpreter. In the safety instances universality is an idealization, and the content is the argument that a system with desirable properties, being faithful, calibrated, and covering, would have to be universal and so cannot exist.

7.12 Historical notes

Cantor's diagonal is 1891, though the 1874 uncountability proof contains the idea. Russell wrote to Frege in 1902. Gödel's theorems are the 1931 paper; the diagonal lemma is often attributed jointly to Gödel and Carnap. Tarski's undefinability is 1936, Turing's halting result 1936, Rice's theorem 1953, Löb's 1955, and Solovay's arithmetical completeness of `GL` 1976.

The unification is Lawvere (1969); the accessible modern account working out the instances is Yanofsky (2003), whose organization this chapter follows. Rogers's *Theory of Recursive Functions and Effective Computability* remains standard for the recursion theory behind Rice, and Boolos's *The Logic of Provability* for the modal treatment of the second theorem.

8 Related Work and Positioning

This chapter says what has been done before and why it does not already contain the present contribution. It is organized by the kind of claim each body of work makes, because the relevant question differs by kind: for the unification literature the question is whether the AI-safety results are already covered, for the impossibility literature it is whether the results are already unified, and for the formalization literature it is whether anything of this shape has been machine-checked.

8.1 Unifications of the diagonal

The claim that the classical self-referential impossibilities share one structure is not new and is not claimed as new here.

Lawvere's 1969 paper is the origin. It proves that in a cartesian closed category, a point-surjective map forces every endomorphism of the codomain to have a fixed point, and observes that Cantor, Russell, Gödel, Tarski, and Turing fall out by choosing the category and the endomorphism. Yanofsky's 2003 account works the instances out concretely and without categorical machinery, and it is the presentation Chapter 4 follows.

What these do not do is instantiate the schema at machine learning. Lawvere's targets are set theory, logic, and computability; the systems he quantifies over are mathematical objects whose self-application is exact. The gap this thesis closes is not the observation that a schema exists but the demonstration that particular safety properties of statistical models are instances of it, and the demonstration is nontrivial precisely because the instantiation fails on a literal reading: a finite model is not point-surjective. Chapter 9 is the work that repairs this, and it has no counterpart in the unification literature.

8.2 Impossibility results in machine learning

There is a substantial literature of individual impossibility results. The fairness-tradeoff theorems, the no-free-lunch results, the limits on calibration under distribution shift, and the security analyses of prompt injection each establish that some desirable conjunction is unachievable.

These are the results this thesis claims to unify, and the relationship is asymmetric in an important way. Each is stated and proved within its own formalism, and each is correct within it. The contribution here is not a better proof of any one of them; several of the original arguments are shorter than mine. The contribution is that the same two lemmas generate them, which is a claim about the collection rather than about any member, and which therefore cannot be established by improving any single result.

The honest form of the claim matters. I do not derive every published impossibility result from the schemas. I derive the ones stated in Part III, name the load-bearing step in each, and in two of the four case studies find that the load-bearing step is *not* the diagonal. Chapters 13 and 14 reach their main results by counting and by invariance respectively. That is evidence against the strongest version of the unification thesis, and Chapter 16 treats it as such.

8.3 Calibration and hallucination

The calibration literature asks when a model's stated confidence matches its empirical accuracy, and the reliability diagram is its central instrument. The hallucination literature asks when a model asserts what is not so.

The relation to Chapter 7 is a matter of what is being quantified over. The statistical notion of calibration is a property of a distribution: averaged over a population of queries at a stated confidence level, the accuracy matches. The notion this thesis uses is pointwise and exact, as Remark 3.11 sets out, and it is strictly stronger. The trilemma is therefore not a statement that models cannot be statistically calibrated, and reading it as one would be an error. It is a statement about a stronger property, and its practical force comes from the approximate versions of Chapter 9, where the pointwise requirement is relaxed to a band.

This is the clearest place where the thesis could be overclaimed, and I have tried to state the hypothesis precisely enough that the overclaim is visible when it occurs.

8.4 Interpretability and probing

The interpretability program seeks to read a model's internal state and recover what the model represents. J-space probing reads the derivative of the computation rather than its activations; eliciting latent knowledge asks for a reporter that tells the truth about a latent state even where humans cannot check.

Chapter 11's result is negative and is best understood as bounding a goal rather than criticizing a method: a probe that reads every verdict pattern of the model it probes is a reflective verdict, and no such thing exists. This does not say probes are useless, and Chapter 11 is careful to say what it does not say. It says that the complete version of the goal is unavailable, which relocates the research question from whether a full readout exists to how much of one is attainable.

For eliciting latent knowledge the relationship is different again, and it is the case that most constrains the thesis. The obstruction there is a counting fact about functions agreeing on a subset. It requires no self-reference, and the diagonal contributes nothing to it. Chapter 13 reports this.

8.5 Formal verification of machine learning

Formal verification in machine learning has concentrated on properties of specific networks: robustness certificates within a ball, output-range bounds, and monotonicity, discharged by SMT solvers or abstract interpretation. These verify a particular trained model against a particular specification.

The corpus here does something different in kind. It verifies theorems about *every* system satisfying stated hypotheses, which is why it is written in a proof assistant with a dependent type theory rather than in a solver. There is no trained network anywhere in the four libraries. The tradeoff is the usual one: the statements are far more general and say nothing whatever about any particular deployed model, which is why Chapter 15 exists.

I am not aware of prior work that machine-checks impossibility results for AI safety properties. If the claim of novelty in this thesis is anywhere at its strongest, it is here, and it is also the easiest to check: the corpus builds, or it does not.

8.6 Summary of the gap

The unification literature has the schemas but not the instances. The impossibility literature has the instances but not the unification. The calibration literature studies a weaker hypothesis than the one the trilemma consumes. The verification literature checks models rather than theorems about models. This thesis occupies the intersection, and Chapter 15 tests whether the intersection is inhabited by anything real.

9 Part II: The Two Engines

10 Three Trilemmata, One Theorem

Three impossibility results circulate under different names with different proofs. The Hallucination Trilemma: a model cannot be faithful, covering, and calibrated at once. The Defense Trilemma: a wrapper against prompt injection cannot preserve utility and be complete at once. The coupling theorem: a model whose confidence separates true from false answers owns a query it can place on neither side. Each was proved with topology. All three are `no_reflective_verdict`, proved without topology at all.

The claim is stronger than "these rhyme." In Lean their proof terms are literally equal, witnessed by `rfl`. What differs is the reading of one Boolean `v : Q → Q → Bool`: confident and correct about `q`; the wrapper renders `q` safe; confidence puts `q` on the true side. The reading changes; the theorem does not.

The real work connects each domain's desiderata to the hypothesis the engine needs, *reflectivity*: every pattern of verdicts over `Q` is named by some element of `Q`. The diagonal buys freedom from topology and pays with that premise. For prompt injection the price is low, since prompts are arbitrary self-describing text; for a fixed representation space it is high, and Chapter 4's route is more physical there.

That the three share a mechanism is the main result, not a curiosity: patching one cannot help the others, because there is nothing domain-specific to patch.

Each trilemma below has the same shape — informal statement, numbered conditions, the argument that they make the system a reflective verdict, a one-line Lean instance, a reading of the liar witness, and what reflectivity costs. The `c3_` blocks are short on purpose: the mathematics was finished in Chapter 1, and these rename its conclusion. Spend attention on the definitions, because disagreement about a premise is the only honest way out of an impossibility the machine has checked.

The word *trilemma* is used two ways: a conflict among three desiderata over one Boolean verdict, and, in the master trilemma, a conflict among Utility, Control, and Transparency where you may keep at most two.

10.1 What ”reflective” demands

A *reflective verdict* on `Q` is a universal `v : Q → Q → Bool`. `no_reflective_verdict` says none exists, and `liar_query` produces the `p` with `v p p = !(v p p)`.

Two ingredients go in: Boolean outcomes with the flip as controller, which `bool_not_fpf` shows has no fixed point, and universality. Universality is not the claim that a model is powerful or large but that its naming of verdict-patterns is *complete*. The source differs by domain — an in-context learner promptable to any behavior, unrestricted prompt text, a representation space encoding any pattern over its own states — and these are not equally believable.

The flip means this. In each domain there is an *honest self-report*: faithfulness, completeness, and exact separation each say that applied to its own description the system should output the opposite of what that description predicts. A confident answer should be correct, so a query predicting its own incorrectness should be answered against the prediction.

Remark 3.0 (Surjection or section). Reflectivity can be weakened to a section `s : (Q → Bool) → Q` with `v (s g) = g`. The two agree for the impossibility, since the engine needs one preimage of the diagonal pattern. The domain stories below, an in-context learner realizing a described behavior, a prompt encoding a pattern, a representation encoding an assignment, are all stories about producing a preimage on demand, which is a section.

Counting gives the finite intuition and then fails: with `|Q| = n` there are `n` rows and 2^n patterns, and the missing one can be taken to be `q ↦ !(v q q)`. Chapter 1's theorem forbids reflectivity even when `Q` is infinite, so ”the model is large” buys no escape.

A worked micro-instance. Take `Q = {a, b, c}` and lay `v` out as a table with rows self-contexts and columns queries. If the diagonal reads `true, false, true`, the engine builds `g = false, true, false`, which disagrees with each row at its own column, so no row equals `g`. Read the other way: hand me a row `p` with `v p = g` and `v p p = g p = !(v p p)`, which `bool_not_fpf` forbids. The liar is a *specific* row, and nothing depended on the size of `Q`. Every argument here is this table, grown until the rows are model behaviors, defense verdicts, or probe outputs.

10.2 The Hallucination Trilemma

The desiderata

A trustworthy model should be *faithful* (confident answers are right), *calibrated* (confidence tracks correctness), and *covering* (it answers, sometimes rightly and sometimes wrongly). None is controversial alone, and the trilemma says they cannot hold together once the model is asked about its own verdicts. Either it answers confidently and wrongly, or its confidence stops tracking cor-

rectness at the hard cases, or it retreats to a response so uniform it no longer covers.

Fix a domain `Q` of queries, read also as self-contexts, with the confident-correctness self-verdict `v : Q → Q → Bool`. The diagonal `v p p` is the model's verdict on its own description.

Definition 3.1 (Strict calibration). `v` is *strictly calibrated* when confidence exactly decides correctness, so `v p q` is a genuine Boolean, `true` exactly when the model is both confident and correct. Calibration licenses modeling the self-assessment as a total function into `Bool`. Without it, confidence and correctness come apart by degrees and the verdict is real-valued; Chapter 6 handles that relaxation.

Definition 3.2 (Faithfulness). `v` is *faithful* when confidence implies correctness, with no confident error. Faithfulness fixes the *controller* as the flip: the honest self-report of a faithful calibrated model on a query predicting its own incorrectness must come out correct, hence opposite to the prediction. A faithful calibrated verdict may never equal its own negation, since such a value would be a confident answer correct exactly when incorrect.

Definition 3.3 (Coverage). `v` is *covering* when the model answers on both sides: some query gets a correct confident verdict and some does not. This is nontriviality. It rules out the constant model, which is faithful and calibrated for free, and it makes the flip act on something with content.

Definition 3.4 (Reflectivity). `v` is *reflective* when universal: `∀ g : Q → Bool, ∃ p, v p = g`. This says the model can be queried about its own verdicts, including the diagonal pattern.

Why the conditions make the verdict reflective

Two halves. The first makes `v` the right *kind* of object, the second makes it *universal*, and only together do they give a reflective verdict.

Calibration makes the codomain `Bool`, since a calibrated model does not hedge between confident and correct. Drop it and you are in the real-valued world where the controller is the complement, fixed-point-free except at one half, which is the topological story. Faithfulness makes the controller the flip: the honest response to "you will be incorrect here" is to be correct, hence against the prediction.

Universality holds because the model is an in-context learner over its own query space. A self-context is expressible as a query, so asking for `g` is prompting with a description of `g`, and a competent learner imitates it. The stronger the model, the more completely it can be steered, so universality is a consequence of flexibility rather than a weakness.

Together: calibration makes `v` Boolean, faithfulness makes the controller the flip, coverage makes the flip bite, reflectivity makes the diagonal pattern an actual query. The three are not independently impossible; they are impossible *in a reflective model*, and the model's own capability supplies the reflectivity.

One reading sounds paradoxical: a *better* model is more nearly reflective, hence more impossible. The theorem does not say the model breaks everywhere, only that the three cannot all hold at the liar query, and a more capable model is steerable to that query more reliably. Capability sharpens the trilemma rather than defeating it — one of the few places the diagonal predicts the direction of an effect.

The impossibility

```
theorem c3_hallucination_trilemma {Q : Type _}
    (v : Q → Q → Bool)
    (reflective : ∀ g : Q → Bool, ∃ p, v p = g) : False :=
  no_reflective_verdict v reflective
```

Proof. `reflective` is exactly the universality `no_reflective_verdict` requires. Faithfulness and calibration have already been spent: calibration in choosing `Bool`, faithfulness in the flip controller. Coverage is spent in reflectivity, which asks for the flipping pattern by name. ■

```
theorem c3_hallucination_liar {Q : Type _}
    (v : Q → Q → Bool)
    (reflective : ∀ g : Q → Bool, ∃ p, v p = g) :
    ∃ p, v p p = !(v p p) :=
  liar_query v reflective
```

Proof. Immediate from `liar_query`. ■

Reading the liar witness

Topologically p is a query where confidence lands at threshold and the answer on the truth boundary; here it is a query predicting its own verdict and flipping it. Concretely it is a prompt of the form "answer the following with confidence exactly if you would be wrong to." A faithful model must answer it correctly, which by construction is to answer it incorrectly; a calibrated model cannot escape by lowering confidence, since calibration ties confidence to correctness; and a covering model cannot escape by refusing, since the pattern it failed to realize was the flipping pattern itself.

The three escapes

Drop faithfulness. The model answers confidently and wrongly. This is ordinary hallucination, and the diagonal says it is structurally necessary for a reflective calibrated model rather than an accident to train away at the margin. Bounding how often is Chapter 5's work.

Drop calibration. The model hedges, lowering confidence on hard self-referential cases so no confident answer is wrong, at the cost of confidence

no longer meaning what it says elsewhere. This is why deployed models are deliberately underconfident near their limits.

Drop coverage. The model stops answering. Refusal training is this applied selectively, and the diagonal warns that the region you must carve is exactly the self-referential one. Chapter 4 reads the same move as disconnecting the query space.

No escape removes the liar; each decides what to do *at* it. The query is forced by reflectivity, and the only freedom is which desideratum to spend, which is why "we will just fix hallucination" is not a coherent program.

Remark 3.11 (Calibration is pointwise and exact). Definition 3.1 is much stronger than the statistical calibration of the reliability-diagram literature, which asks only that a stated confidence c be correct a fraction c of the time averaged over a population, and says nothing about any single query. The trilemma's calibration is a two-way tie at every query. The trilemma therefore does not say statistically calibrated models are impossible; they exist.

Relation to statistical inevitability results

An older tradition proves hallucination inevitable by statistical or computability arguments, deriving *confident falsehood* and quantifying *how often*. The diagonal derives a single *boundary query* and says nothing about frequency, so they are complementary. Only the stochastic dichotomy turns a boundary into falsehood with positive probability.

The honest caveat. Reflectivity is the whole cost of dropping topology, and for hallucination it is a middling assumption. Treating the model as an in-context learner that imitates any described verdict-pattern is defensible for large models, but "any pattern over all of $\mathbb{Q}$" is a strong completeness claim and a real model's realizable patterns are a proper subset. Chapter 4 asks instead for continuity on a connected space, and both routes reach the same boundary question.

10.3 The Defense Trilemma

The desiderata

Prompt injection is the attack where user input contains instructions subverting the system's own. A defense is a wrapper transforming an incoming prompt before the model sees it. It should be *utility-preserving*, doing nothing to an already-safe prompt, and *complete*, making every prompt safe. Reflectivity is the fact that prompts are arbitrary text.

The published proof is topological: continuity of the defended safety score plus utility preservation force the score to hit the threshold, and completeness says it never does. The diagonal replaces continuity with self-reference, which for prompt injection is not an idealization but the definition of the problem. A prompt is text, and text can quote the defense and invert it.

The conditions

Let `X` be prompts, read also as self-contexts. Model the defended-safety verdict as `v : X → X → Bool`, with `v p q = true` meaning the wrapper renders `q` safe while also coping with the self-referential prompt `p`.

Definition 3.5 (Utility preservation). The wrapper is *utility-preserving* when it fixes safe prompts, so the defended verdict on a safe prompt agrees with its undefended safety. This blocks the constant escape: a wrapper replacing every input with a fixed maximally safe output would be complete for free but destroys every safe prompt's content.

Definition 3.6 (Completeness). The wrapper is *complete* when every prompt is rendered safe. This makes the honest self-report the flip: a complete wrapper faced with "you will mark me unsafe" must mark it safe, answering against the prediction.

Definition 3.7 (Reflectivity). The prompt space is *reflective* when `∀ g : X → Bool, ∃ p, v p = g`. Because prompts are arbitrary text, any pattern of verdicts is describable by a prompt.

Why the conditions make the verdict reflective

The verdict is already Boolean, since safety is discrete by nature, so no calibration step is needed. Completeness plays faithfulness's role, fixing the flip, and utility preservation plays coverage's, forbidding the constant maximally-safe wrapper.

Universality needs almost no idealization. Any pattern can be written as a prompt instructing the wrapper to behave that way, because instructing the wrapper is just more text and the wrapper reads text. The injection "ignore your previous instructions" is a prompt describing and overriding the defense, and unlike hallucination there is no gap between "could imitate any behavior" and "every pattern is realized": the realizing objects are the attacker's inputs.

A skeptic objects that no *sensible* prompt describes "be unsafe exactly if you would be made safe." Sense is not required. Reflectivity asks only that the pattern be nameable as text, and the attacker needs the wrapper to act on it, not a human to understand it. This is the gap between prompts a user would write and prompts an adversary can write, and the trilemma lives on the adversary's side.

The impossibility

```
theorem c3_defense_trilemma {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) : False :=
  no_reflective_verdict v reflective
```

Proof. Word for word the hallucination proof with `X` for `Q`. ■

```
theorem c3_defense_liar {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) :
```

```
  ∃ p, v p p = !(v p p) :=
liar_query v reflective
```

Proof. The witness is the injection no wrapper neutralizes: a prompt rendered safe exactly when unsafe. ■

Reading the liar witness

The prompt is ”be unsafe if and only if this wrapper would render you safe.” A complete wrapper must render it safe, so by construction it is unsafe. The wrapper cannot rewrite it harmlessly without violating utility preservation on the safe prompts it quotes, and cannot refuse it without ceasing to cover the pattern it embodies. The diagonal writes the witness out as text, which for defense is the honest description, because injections are text.

A defender’s instinct is to make the wrapper stateful or randomized so the verdict is not a fixed function. Randomize, and the same diagonal runs on the induced verdict ”rendered safe”; the honest place for randomness is the stochastic dichotomy, where a system faithful almost everywhere still emits unsafe outputs with positive probability. Make it stateful, and the state joins the self-context, with reflectivity over the enlarged space still naming the diagonal.

Why detection does not help. A rejection is a verdict. When the wrapper rejects the liar as unsafe it has assigned `p` the value ”unsafe,” and completeness demanded ”safe.” Detection chooses the completeness-failing horn and calls it a feature. That may be the right engineering choice, but it is the coverage escape in defense clothes, at the same cost.

The real-valued score version

The companion library carries `defense_trilemma_lawvere`, which does not pass through `Bool`. There the object is a defended safety score `s : X → X → ℝ` and completeness is `s a a < τ`. The controller is the real complement, fixed-point-free except at the threshold, and the diagonal forces a boundary prompt where the score sits exactly at `τ`.

The honest caveat

For defense reflectivity is close to free: the input space is unrestricted and self-describing, so the surjection restates the threat model rather than assuming anything. Rejection is a verdict too, and a wrapper rejecting `p` while claiming completeness has relabeled `p` unsafe. There is no consistent relabeling, which is `defense_trilemma_godel` in `F_12_DefenseGodel`.

One caveat is often missed: the trilemma is about a *single fixed* wrapper. A defender who may change the wrapper after seeing the attack faces an easier problem, which explains why defenses appear to work in the lab and fail in the

wild. In the lab the wrapper is retuned each round; in the wild it is fixed and the attacker constructs its liar. Any process that keeps changing the wrapper is not a wrapper but a game.

10.4 The Coupling Theorem

The desideratum

Probing studies read truth off activations, treating truth as a continuous field. The coupling theorem asks what such a probe requires of the model answering over the same space: if the model ever answers truly and ever falsely, and its confidence separates the two exactly, some query carries confidence exactly at threshold while its answer sits exactly on the truth boundary.

The framing is generous rather than adversarial. It grants the central assumption, that truth is a continuous decodable field, and the answer is not that probes fail but that a probe good enough to separate truth exactly comes with a boundary it cannot avoid. The theorem is a frontier for the program, not a refutation.

The desideratum is *truth-separation* in exact biconditional form. Under it the confidence field is a total exact Boolean decider of truth over representation space — a definable truth predicate, which Chapter 2's Tarski reading forbids.

The conditions

Let `X` be representation space, read also as the space of self-referential representations. Model the verdict as `v : X → X → Bool`, with `v p q = true` meaning confidence places `q` on the true side in self-context `p`.

Definition 3.8 (Truth-separation). `v` is *truth-separating* when confidence and truth agree exactly and two-ways: `v p q = true` iff `q` is true in context `p`. This plays calibration's and faithfulness's roles at once: exactness makes the codomain `Bool`, and the two-way tie means a representation predicting its own falsehood must be judged true, against its prediction.

Definition 3.9 (Coverage for coupling). The verdict is *covering* when the model answers some queries truly and some falsely, so the probe separates a nonempty true region from a nonempty false region.

Definition 3.10 (Reflectivity for representations). The space is *reflective* when `∀ g : X → Bool, ∃ p, v p = g`: every pattern of truth-verdicts is encoded by some representation. This is the strong hypothesis.

Why the condition makes the verdict reflective

Truth-separation does double duty: exactness makes the verdict genuinely Boolean, and the two-way tie means a representation encoding "the probe will place me on the false side" must, if honored as truth, be placed on the true

side. Coverage supplies content, since without both values the truth boundary is empty.

Universality is where coupling differs in believability. Every pattern must be encoded by some representation, including the diagonal one, and for an activation space that is a real assumption: activations are expressive, but nothing guarantees every Boolean pattern over states is realized by a state. This is why the paper's main result is topological — connectedness follows the manifold hypothesis, complete self-encoding follows from nothing obvious.

Where it applies the reading is exact. Grant self-encoding and an exactly truth-separating probe is a total self-applicable truth predicate, so Tarski forbids it and the diagonal names the coupled query. This is Tarski's theorem with "sentence" read as "representation" and "truth predicate" as "linear probe used as confidence," and the strength of the premise is the price of making the analogy an identity.

The impossibility

```
theorem c3_coupling_trilemma {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) : False :=
  no_reflective_verdict v reflective
```

Proof. Identical to the previous two. ■

```
theorem c3_coupling_liar {X : Type _} (v : X → X → Bool)
    (reflective : ∀ g : X → Bool, ∃ p, v p = g) :
    ∃ p, v p p = !(v p p) :=
  liar_query v reflective
```

Proof. The witness is a representation the probe would place on the true side exactly when it places it on the false side. ■

Reading the liar witness

Topologically this is the coupled point x_0 where confidence equals the threshold and truth-distance is zero, forced along any path from a true answer to a false one. Diagonally it is a representation encoding the probe's negation of its own verdict. The topological version carries metric content the diagonal cannot see: an ambiguity tube of computable width and a strictly positive truth-side slack any feasible system must carry.

The geometry the diagonal cannot see

On a connected space with continuous fields, the coupled point sits at the centre of an *ambiguity tube* whose width is set by the Lipschitz constants, so the near-boundary region has computable size. For a nonzero linear probe the

truth boundary is an affine hyperplane of codimension one, and even a nonlinear probe gives a hyperplane to first order at any regular point. Feasible systems carry strictly positive truth-side slack, which an adversarial training attempt fails to drive to zero.

The policy taxonomy

A *threshold-inclusive* guarantee promises that confidence at or above threshold implies truth; a *threshold-exclusive* one promises it only strictly above. The coupled query has confidence exactly at threshold, so it is the hinge. Add the inclusive guarantee and you get a contradiction, `closed_guarantee_impossible`.

Defining the guarantee with a strict inequality does not dissolve the impossibility. It relocates it from a contradiction to a silence. Either the guarantee claims the coupled query and is false, or excludes it and is silent there. The diagonal sees the same fork through the liar: any guarantee assigning it a definite side is refuted, and any declining to has conceded the one query the theorem cares about.

Leaving the continuum

Token spaces are finite, and a fair objection is that connectedness does illegitimate work. Without topology a true-to-false path yields only a `discrete_sign_change`, an adjacent straddling pair rather than a query on the boundary — exactly what connectedness derived for free and what the discrete setting must assume. Once assumed, the impossibility returns, as `boundary_question_impossible` and `faithful_iff_boundary_free` show.

This is reflectivity's role: it *supplies* the boundary witness by fiat. The three routes cost differently — continuity costs a connected domain, assumption costs honesty about what you put in, reflectivity costs completeness of self-encoding. Randomization escapes none.

A symmetry version needs no coverage: if semantic negation acts as a fixed-point-free involution and the truth field is odd under it, the field vanishes between any state and its negation. This is the scalar shadow of Borsuk-Ulam, and a reminder that equivariance, usually a design virtue, is here an engine of the obstruction.

The honest caveat

Hold this reading most loosely. Reflectivity of a fixed representation space is a strong, less physical assumption than continuity on a connected domain, and the topological route is primary for representation geometry. The diagonal supplies the witness by fiat, which is legitimate when self-encoding is real, as in a model prompted to reason about its own probe, and an overreach for a passive vector space.

The practical division: if your pipeline reads a probe off frozen activations and never asks the model to reason about it, the space is passive and the topological engine applies. If the model conditions on descriptions of its own truth-readout, as agentic and self-critiquing setups do, self-encoding is becoming real and the diagonal begins to apply. Knowing which regime you are in is a modeling question answered before either theorem says anything.

10.5 The Master Trilemma

`CCHProofs` organizes all of this as faces of one three-cornered trilemma. Where the readings above vary the *meaning* of a Boolean verdict, the master trilemma varies the *outcome type* and carries a self-prediction alongside the verdict.

Bundle a system `s : A → A → Y`, a controller `t : Y → Y`, and a self-prediction `p : A → Y`. *Universality*: `s` is surjective. *Control*: `t (p a) ≠ p a` for every `a`, so no prediction is a fixed point. *Transparency*: `s a a = p a`, so the system says what it will do on itself. Read `U` as capability, `C` as control, `T` as honesty. No system is all three.

```
theorem c3_lawvere_point {A Y : Type _} (s : A → A → Y)
    (hU : ∀ g : A → Y, ∃ a, s a = g) (t : Y → Y) :
    ∃ a, s a a = t (s a a) :=
  match hU (fun a => t (s a a)) with
  | ⟨a₀, ha₀⟩ => ⟨a₀, congrFun ha₀ a₀⟩
```

Proof. Chapter 1's move with the flip replaced by arbitrary `t`. ∎

```
def C3_Universal {A Y : Type _} (s : A → A → Y) : Prop :=
  ∀ g : A → Y, ∃ a, s a = g

def C3_Controlled {A Y : Type _} (t : Y → Y)
    (p : A → Y) : Prop :=
  ∀ a, t (p a) ≠ p a

def C3_Transparent {A Y : Type _} (s : A → A → Y)
    (p : A → Y) : Prop :=
  ∀ a, s a a = p a

theorem c3_cch_master {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hU : C3_Universal s) (hC : C3_Controlled t p)
    (hT : C3_Transparent s p) :
    False := by
  obtain ⟨a₀, ha₀⟩ := c3_lawvere_point s hU t
  rw [hT a₀] at ha₀
  exact hC a₀ ha₀.symm
```

Proof. `c3_lawvere_point` gives a_0 with `s a₀ a₀ = t (s a₀ a₀)`. Transparency rewrites this to `p a₀ = t (p a₀)`, making `p a₀` a fixed point of `t`. Control says it is not. ■

```
theorem c3_cch_corner_UC {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hU : C3_Universal s)
    (hC : C3_Controlled t p) : ¬ C3_Transparent s p :=
  fun hT => c3_cch_master s t p hU hC hT

theorem c3_cch_corner_UT {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hU : C3_Universal s)
    (hT : C3_Transparent s p) : ¬ C3_Controlled t p :=
  fun hC => c3_cch_master s t p hU hC hT

theorem c3_cch_corner_CT {A Y : Type _} (s : A → A → Y)
    (t : Y → Y) (p : A → Y)
    (hC : C3_Controlled t p)
    (hT : C3_Transparent s p) : ¬ C3_Universal s :=
  fun hU => c3_cch_master s t p hU hC hT
```

Proof. Each supplies two properties and derives the falsity of the third. ■

These make it a trilemma rather than an impossibility. You may have any two. A universal controlled system exists but cannot be honest about itself; a universal transparent one exists but cannot be controlled; a controlled transparent one exists but cannot be universal.

Each corner is a real class of system. Universal and controlled but not transparent: an expressive model under external correction whose self-reports must sometimes be wrong. Universal and transparent but not controlled: an honest expressive model no controller can always nudge.

Remark 3.12 (The trilemmas as the UC face). Take `Y = Bool`, `t` the flip, and `p` the diagonal `a ↦ s a a`. Transparency then holds by definition of `p`, so a universal system is automatically transparent for this choice, and control is `bool_not_fpf` at `p a`. The master collision becomes `v p p = !(v p p)`, exactly `c3_hallucination_liar`. So the Boolean trilemmata are not analogous to the master trilemma; they are one of its faces, with transparency made free and control made the flip.

Reading the corners as the three trilemmata

Universality is the reflectivity running through the chapter. Transparency is the honesty condition, carried in the Boolean readings by choosing the self-prediction to be the diagonal. Control is the demand that an external map can move the self-prediction, whose Boolean instance is that the flip has no fixed point, which is faithfulness for hallucination, completeness for defense, and exact separation for coupling under their outward-facing names.

So each verdict trilemma says Universality and Control force the failure of Transparency. The readings differ in which real property plays Control and in how believable Universality is, and agree on the skeleton. The master trilemma is not a fourth result stacked on three others; it is the frame showing the three were always the same corner viewed from three domains.

Why it is a trilemma, not a dilemma

Any two of the three are jointly satisfiable, and the corner theorems are that fact turned into contrapositives. A dilemma says two things cannot both hold. The trilemma says three cannot all hold while any two can, which is stronger and more useful, because it tells a designer that giving up one property is not only necessary but sufficient.

10.6 One proof term

```
theorem c3_trilemmata_unified {Q : Type _}
    (v : Q → Q → Bool)
    (h : ∀ g : Q → Bool, ∃ p, v p = g) :
    c3_hallucination_trilemma v h = c3_defense_trilemma v h
      ∧ c3_defense_trilemma v h
        = c3_coupling_trilemma v h :=
  ⟨rfl, rfl⟩
```

Proof. The three theorems have the same body, `no_reflective_verdict v h`, so they are definitionally equal. ■

This is not that the three are analogous, or that a shared lemma covers them. There is one function `v`, one hypothesis, and one proof that the hypothesis is contradictory. The names are labels for the reader; the machine sees a single term.

Remark 3.13 (What `rfl` is checking). Two things make the equality trivial. Each `c3_` trilemma is *defined* as `no_reflective_verdict` applied to the same arguments, so after unfolding the bodies are one term. And all three have type `False`, so any two proofs are equal by proof irrelevance. The first route carries the meaning: the theorems are the same not because all proofs of `False` are interchangeable, but because these three are the identical construction with three labels. The kernel would accept the second even for genuinely different arguments, which is why the first is the one to cite.

`F_13_UnifiedTrilemmata` proves the same `rfl` chain over `Function.Surjective` and packages the shared boundary object as `schema_boundary`.

10.7 What the readings share, and where they part

Fixed across all three: a Boolean verdict, the flip as controller, universality as hypothesis, and a self-negating witness. What moves is the verdict's meaning

and the believability of universality.

The meaning moves from "confident and correct" to "made safe" to "placed on the true side." The horns fixing the flip move from "faithful and calibrated" to "complete and utility-preserving" to "exactly truth-separating." The source of universality moves from "the model imitates any described behavior" to "prompts are arbitrary text" to "the representation space encodes any pattern over itself." That last move changes the character of the result: defense's universality is nearly free, hallucination's defensible, coupling's strong. Chapter 4 trades all three self-reference premises for one geometric premise and reaches the same boundary object with metric content.

There is a temptation, once the `rfl` is in hand, to conclude the results are trivial. The proof is one line because the modeling was done first. All the content is in Definitions 3.1 through 3.10 and the arguments that they add up to a reflective verdict. The engine is trivial on purpose; the theorems are as strong as the readings feeding it.

10.8 The template in four moves

First, choose the reading: what `v p q` means, and what the self-contexts are. Second, argue the codomain is `Bool` and the controller is the flip, by finding the condition that makes outcomes two-valued and the one that makes the honest self-report the negation of the diagonal. Third, argue universality, identifying the source of self-reference and being honest about its strength. Fourth, apply the engine.

The moves are not equally hard. The first is a modeling decision, the second a short argument from a standard desideratum, the fourth one line of Lean. The weight is in the third, the premise a critic will contest.

This also explains why the Lean here is so short. Moves one to three happen in prose and only move four is code. A presentation that hid the prose would look like three deep theorems in three lines. What it actually did was three careful readings and one reused lemma, and the honesty is in keeping the readings visible.

10.9 Axioms and trust

The `c3_` theorems are elaborated as this document builds, so they are proofs the kernel has accepted. The companion library audits the axioms and finds the main results reduce to `propext`, `Classical.choice`, and `Quot.sound`. The Boolean core uses even less, since the flip's fixed-point freedom is decidable.

The trust verification cannot supply is trust in the *modeling*. No proof assistant can check that "confident-correctness self-verdict" is the right reading of a real model, or that a real prompt space is reflective. Those are the premises, and they are what the long middle of each section argues and what the caveats concede.

10.10 What this chapter does not settle

It does not say models hallucinate often, defenses fail often, or probes mislead often. It produces one query per system and is silent on frequency. A model could meet its liar on a query no user ever sends. That question belongs to the statistical tradition and to Chapter 5.

It does not say the liar is easy to find. The diagonal proves one exists and names it as a preimage of a specific pattern, but naming a preimage is not constructing it cheaply. For a large model the realizing self-context may be hard to elicit, and the cost is exactly what the diagonal cannot see.

It does not choose between the engines. The diagonal is honest when self-reference is real, as for prompt injection; the topological engine is honest when the domain is a connected space with continuous fields, as plausibly for a representation manifold. They agree on the boundary object, and having both lets an application be attacked from whichever premise it satisfies.

10.11 Historical notes

The Hallucination Trilemma is re-derived in Gödel form as `hallucinatio n_trilemma_godel` in `F_11_HallucinationGodel`, which records that the Boolean flip and the real complement are the two controllers. The Defense Trilemma is `defense_trilemma_godel` in `F_12_DefenseGodel`, with `defe nse_trilemma_lawvere` alongside. The coupling theorem is "Truth Has a Boundary," whose `boundary_coupling` is topological, with the diagonal reading in `F_13_UnifiedTrilemmata`.

The lineage is Lawvere 1969 through the reading of Tarski as a diagonal. The statistical inevitability of hallucination is a separate tradition deriving confident falsehood rather than uncertainty; only `stochastic_dichotom y` turns the boundary into falsehood with positive probability. Chapter 1's engine lives in `F_01_LawvereCore`, and all the developments pin the same toolchain and Mathlib release, so these cross-references are to code that compiles together.

11 From Logic to Analysis

The diagonal needs a system rich enough to name its own behaviors, and many real models do not meet `∀ g, ∃ a, f a = g` literally. A trained network carries no index for every function from prompts to confidences; its reachable states are a bounded region, not a set closed under naming its own predicates.

The second engine runs on different fuel. It asks that the space be *connected* and the maps *continuous*, and from those it extracts the same boundary object: a question where confidence sits exactly at threshold while the answer sits exactly on the truth boundary. The tool is the intermediate value theorem.

Code here is core Lean 4, elaborated as this document builds. The analytic theorems live over $\mathbb{R}$ and use topology, so their machine-checked forms are in the companion `HallucinationProofs` library. Verified names appear in plain type; the prose proofs are complete on their own terms and the names are pointers, not omissions.

11.1 Why a second engine

The engines answer different objections. Against a system that names its own behaviors the diagonal is decisive. Against a system on a manifold of activations it has no purchase, because no index need recover the diagonal behavior. What such a system has is shape: a connected representation space, continuous confidence, continuously varying correctness. That is enough.

In one sentence: self-reference forces a fixed point because a flip has nowhere to hide on the diagonal; connectedness forces a crossing because a continuous quantity negative somewhere and positive somewhere cannot pass between without visiting zero. Both trap a value, one with a naming equation and one with a topological wall. The trapped value is the same, and a later section makes that identity exact.

Concretely: a proof-checker accepting the code of any predicate, or an in-context learner promptable to any behavior, is close to the diagonal hypothesis. A trained classifier reading truth off residual-stream activations is not; it carries no index for every function from activations to confidences. Yet its activations form a connected region, by the manifold hypothesis and the mod-

eling of embedding spaces as `ℝ^d` or connected submanifolds, and its probe and confidence heads are continuous.

The analytic engine also carries metric content the diagonal cannot. With a distance and Lipschitz maps, the crossing is not a bare witness but the centre of a band of computable width. Chapter 5 is entirely about those quantities.

11.2 A topology primer

Definition 4.1 (Topological space). A *topological space* is a set `X` with a collection of *open* subsets including `∅` and `X`, closed under arbitrary unions and finite intersections. A set is *closed* when its complement is open.

Definition 4.2 (Continuity). `f : X → Y` is *continuous* when the preimage of every open set is open; equivalently, of every closed set is closed.

The preimage form is the same idea as epsilon-delta and easier to use, and it makes continuity a statement about two topologies, which is the level at which connectedness lives. The fact used repeatedly is that compositions of continuous maps are continuous. The verified proofs build on Mathlib's `Continuous.comp` and the product rule `Continuous.prodMk`, which is how `q ↦ δ(q, (M q).1)` is shown continuous in `model_truth_boundary_nonempty`.

The *closure* of `S` is the smallest closed set containing it; `S` is *dense* when its closure is everything. Density is the topological form of the strongest coverage condition, verified as `DenseCoverage` in `HoF_05_Coverage`, with `denseCoverage_closure_eq_univ`. The main theorem uses only the mild two-sided coverage of Definition 4.12.

Metric spaces and paths. A metric gives open balls, and the metric topology's opens are the sets containing a ball around each point. Continuity in the preimage sense agrees with epsilon-delta. The standing example is `ℝ^d`; the verified sphere instantiation uses `EuclideanSpace ℝ (Fin d)`, and the quantitative results use `PseudoMetricSpace` with explicit Lipschitz hypotheses.

A *path* from `x` to `y` is a continuous `γ : [0,1] → X` with `γ 0 = x`, `γ 1 = y`, and `X` is *path-connected* when any two points are joined. Path-connected implies connected: a path from one side of a separation to the other pulls it back to a separation of `[0,1]`, which is connected. The converse can fail in general but holds for the convex regions and manifolds of the applications.

Paths matter because interpolating hidden states from a true statement to its negation traces a path, along which the realized truth-distance starts negative and ends positive, so the one-variable theorem gives a crossing on the path itself. That is `path_crosses_truth_boundary` in `HoF_03_BoundaryCrossing`, and it is what an experiment actually walks along.

Products. If `X` and `Y` are connected so is `X × Y`: fix a base point, note each slice through it is a continuous image of a connected space, and observe that a union of slices meeting a common point cannot be separated. By induction a finite product of connected spaces is connected. So if a single-turn space `Q`

is connected, the multi-turn space `Q^T` is too, and every argument transports unchanged, since the only property used was connectedness.

Definition 4.3 (Connected space). `X` is *disconnected* when open `U, V` exist with `U ∪ V = X`, `U ∩ V = ∅`, both nonempty; *connected* when nonempty and not disconnected.

Definition 4.4 (Preconnected set). `S ⊆ X` is *preconnected* when no open `U, V` satisfy `S ⊆ U ∪ V`, `S ∩ U ∩ V = ∅`, `S ∩ U ≠ ∅`, and `S ∩ V ≠ ∅` at once. It is *connected* when also nonempty.

The distinction is only whether the empty set is admitted. Mathlib keeps them apart because the intermediate value theorem is cleanest for preconnected sets, and the verified files call that form. `isPreconnected_univ` is the first line of every IVT application in `HoF_03_BoundaryCrossing` and `HoF_07_TrilemmaCore`.

Two examples fix the idea. Any interval of $\mathbb{R}$ is connected, by completeness: given a split, the supremum of the left part lies in neither piece without contradicting openness. A two-point discrete space is disconnected.

Proposition 4.5 (Continuous image of a preconnected set). *If `S` is preconnected and `f` continuous on `S`, then `f(S)` is preconnected.*

Proof. A separation of `f(S)` by `U, V` pulls back to `f⁻¹(U)`, `f⁻¹(V)`, open by continuity, covering `S`, missing `S` in their intersection, and each meeting `S`. That separates `S`. ∎

This is `IsPreconnected.image` in Mathlib, and the companion proofs call it. It is the load-bearing beam: connectedness is preserved by continuous maps, and applied to a real-valued map that becomes the intermediate value theorem once we know the connected subsets of the line.

Proposition 4.6 (Connected subsets of the line are intervals). *A subset of $\mathbb{R}$ is preconnected iff it is an interval.*

Proof. If `S` omits `c` with `a < c < b` in `S`, the open rays about `c` separate it. Conversely, let `S` be an interval separated by `U, V`, with `a ∈ S ∩ U`, `b ∈ S ∩ V`, `a < b`. Let `m` be the supremum of points of `[a,b] ∩ U` below `b`, which exists since the set is nonempty and bounded. As `S` is an interval, `m ∈ S`, so `m ∈ U` or `m ∈ V`. If `m ∈ U`, openness pushes points of `U` above `m`, contradicting the bound. If `m ∈ V`, openness puts points just below `m` in `V` and not `U`, contradicting leastness. ∎

In Mathlib the forward direction is `IsPreconnected.ordConnected` and the converse for a closed interval is `isPreconnected_Icc`.

Theorem 4.7 (Intermediate value theorem). *Let `X` be connected and `c : X → ℝ` continuous. If `c` takes a value below `r` and one above `r`, it takes `r`.*

Proof. The image is preconnected by Proposition 4.5, hence an interval by Proposition 4.6, and it contains values on both sides of `r`. ∎

The library uses the two-function form `IsPreconnected.intermediate_value₂`, comparing continuous `f` and `g` on a preconnected set: if `f a ≤ g a` and `g b ≤ f b` then `f x = g x` somewhere. Theorem 4.7 is the case `g` constant. Every IVT call in `HoF_03_BoundaryCrossing`, `HoF_05_Coverage`, and `HoF_07_TrilemmaCore` is this form with `g = continuous_const`; the interval case via `isPreconnected_Icc` is `confidence_path_crosses_half`.

Remark 4.8 (What the theorem does not say). It asserts existence, not uniqueness or location. This is the analytic mirror of the diagonal's silence in Remark 1.18. Both engines produce a witness with no built-in address, and metric content is an extra ingredient added in Chapter 5.

Connectedness as analytic self-reference

In Chapter 1 the diagonal took a flip with no fixed point and derived a contradiction from universality, because a universal system must name `a ↦ t (f a a)`. The engine is that self-application leaves the flip no room.

Connectedness plays the same role. A continuous quantity on a connected space that is negative somewhere and positive somewhere is a flip with no room: a jump would split the space into the negative and positive parts, two nonempty open sets separating a space assumed unseparable. So it passes through zero.

The correspondences, each of which becomes a theorem: self-reference $\leftrightarrow$ connectedness; the flip `t` $\leftrightarrow$ a continuous field taking both signs; the forced fixed point $\leftrightarrow$ the forced zero; Boolean negation $\leftrightarrow$ the real complement `y ↦ 1 - y`; the liar index of Proposition 1.14 $\leftrightarrow$ the boundary question.

The analogy is tight rather than loose. In both engines there is a set of candidate outputs, a transformation on them, and a structural hypothesis leaving the transformation nowhere to sit but a distinguished value. For the diagonal, the outputs are the values `f a a`, the transformation is `t`, and the hypothesis is that every behavior is named.

There is a shared failure mode. Remove the structural hypothesis and both collapse the same way. A non-universal system can carry a fixed-point-free flip with no contradiction, since the diagonal behavior may be unnamed.

11.3 The model, analytically

Definition 4.9 (Model). For topological spaces `Q` of questions and `A` of answers, a *model* is `M : Q → A × ℝ`, with answer `(M q).1` and *confidence* `(M q).2`. It is *continuous* when both components are.

Softmax outputs, probe readouts, and calibrated logits are continuous candidates for the confidence; hard argmax decoding is not, a point priced in the discrete section.

Definition 4.10 (Signed truth-distance). A *truth-distance* is a continuous δ `: Q × A → ℝ`, negative on strictly correct answers, positive on strictly wrong ones, zero on the *truth boundary*. The *realized* truth-distance is `q ↦ δ(q, (M q).1)`.

The sign convention is fixed: negative good, positive bad, zero the wall. A linear probe `w·x - b` is a candidate δ, and whether its boundary tracks semantic truth away from the fitted data is an empirical premise, not a theorem. Continuity of the realized composite follows from the composition and product rules.

Definition 4.11 (Faithful). A model is *faithful* when `(M q).2 ≥ 1/2` implies δ`(q, (M q).1) < 0`. The verified predicate is `TrilemmaFaithful`. The strict `< 0` does the work; a weaker `≤ 0` admits the boundary case and dissolves the theorem.

Definition 4.12 (Covering). A model is *covering* when some `q` has δ`(q, (M q).1) < 0` and some has `> 0`. The verified predicate is `TrilemmaCovering`, also `Covering` in `HoF_05_Coverage`. It is the mildest nontriviality assumption: any model ever correct and ever incorrect satisfies it.

Definition 4.13 (Calibrated). A model is *strictly calibrated* when for every `q`, `(M q).2 > 1/2` $\leftrightarrow$ δ`(q, (M q).1) < 0` and `(M q).2 < 1/2` $\leftrightarrow$ δ`(q, (M q).1) > 0`. The verified predicate is `StrictCalibrated`.

This pointwise biconditional is much stronger than statistical calibration, which is an averaged claim compatible with being wrong on any particular high-confidence question. Strict calibration constrains every single question. It is the idealization a perfect truth probe used as confidence would meet, which is why the empirical work builds the confidence from a probe and asks how close reality comes.

Read the three on the linear-probe picture and none is a strawman. Covering says the model answers on both sides of a hyperplane, which any model on a real workload does. Calibration says a confidence head trained on a disjoint split agrees with the probe about sides, which a well-trained head approximates.

Calibration is a biconditional because it claims the confidence is an honest readout; faithfulness is one-way because a promise is one-directional. Neither is at fault alone. Their conjunction on a connected covering model is.

Separation with slack

Fix a threshold `τ`, truth slack ε `≥ 0`, and confidence margin `γ ≥ 0`. A model is *(*ε`, γ)-separating at` `τ` when δ `< -`ε implies confidence `> τ + γ` and δ `>` ε implies confidence `< τ - γ`. The slack is in truth-distance units and the margin in confidence units, so the condition survives rescaling either field.

Slack is not cosmetic: the main theorem is that with a threshold-inclusive guarantee the truth slack cannot be zero. Using `1/2` for `τ` is a convention the coupling results never depend on; every theorem has a general-`τ` counterpart in the defense-side files.

The defense reading

Reread `Q` as inputs a system might face including adversarial ones, `(M q).2` as a *safety score*, and δ as a harm measure. Faithful becomes "a high safety score certifies real safety," covering becomes "some inputs are safe and some are not," calibration becomes "the score tracks the probe." The coupling theorem then says a defense with a continuous safety score on a connected input space owns an input scored exactly at threshold and lying exactly on the harm boundary. This is `boundary_coupling` in `HoF_13_BoundaryCoupling`.

11.4 The topological proof of the trilemma

Three steps. Coverage plus connectedness forces a confidence crossing. Calibration pins the truth-distance to zero there. Faithfulness demands it be strictly negative, and the two collide.

Lemma 4.14 (Boundary existence). *Let `Q` be connected, the answer map and truth-distance continuous, and the model covering. Then some `q₀` has `δ(q₀, (M q₀).1) = 0`.*

Proof. The realized truth-distance `F` is continuous, being a composite of the continuous `q ↦ (q, (M q).1)` with δ. Coverage gives points where `F` is negative and positive. Theorem 4.7 at value `0` gives `q₀`. ■

This is `model_truth_boundary_nonempty` in `HoF_03_BoundaryCrossing`, packaged with its coverage hypothesis as `covering_yields_truth_boundary_point` in `HoF_05_Coverage`. It uses no confidence and no calibration: pure topology. The boundary question exists before any safety condition enters. The wall is a fact about the shape of the model's correctness, not about how it reports its reliability.

Two structural facts, both verified and both independent of connectedness. The boundary is closed, being the preimage of `{0}` under a continuous map, which is `model_truth_boundary_isClosed` and holds over any topological spaces. And every continuous path from a strictly-correct question to a strictly-wrong one crosses it, which is `path_crosses_truth_boundary`.

Lemma 4.15 (Confidence crossing). *Let `Q` be connected and the confidence continuous. If some question has confidence below `1/2` and some above, then some `q₀` has `(M q₀).2 = 1/2`.*

Proof. Theorem 4.7 applied to the confidence at `1/2`. ■

This is `confidence_half_nonempty` and `model_confidence_half_set_nonempty`. Alone it concerns the confidence field only; it becomes half the trilemma once calibration ties the confidence witnesses to the coverage witnesses.

Theorem 4.16 (Boundary-confidence coupling). *Let `Q` be connected, all three maps continuous, and the model covering and strictly calibrated. Then some `q₀` satisfies `(M q₀).2 = 1/2` and `δ(q₀, (M q₀).1) = 0`.*

Proof. Coverage gives `q_t` with negative and `q_f` with positive truth-distance. Calibration converts these into confidences above and below `1/2`. Lemma 4.15 gives `q₀` with confidence exactly `1/2`. Read calibration backward at `q₀`: a negative truth-distance would force confidence above `1/2`, a positive one below, both contradicting equality. So the truth-distance is `0`. ■

This is `hallucination_trilemma_strict` in `HoF_07_TrilemmaCore`, with the defense twin `boundary_coupling`. It mentions no faithfulness. It is a coupling statement: a covering calibrated continuous model on a connected space owns a question where the confidence is exactly ambiguous and the answer exactly on the wall. The point is real regardless of any safety policy.

Theorem 4.17 (Analytic hallucination trilemma). *No continuous model on a connected question space is simultaneously faithful, covering, and strictly calibrated.*

Proof. Theorem 4.16 gives q_0 with confidence `1/2` and truth-distance `0`. Faithfulness applies there, since `1/2 ≥ 1/2`, and demands the truth-distance be strictly negative. It is zero. ■

This is `hallucination_trilemma`, with `hallucination_trilemma_unfolded` alongside. The verified proof is the two-line composition: extract the coupled point, apply faithfulness, observe the sign clash with `linarith`.

The policy reading makes this a taxonomy. Faithfulness as stated is *threshold-inclusive,* triggering at confidence at least `1/2`, so it triggers at the coupled point. A *threshold-exclusive* guarantee triggers only strictly above and so does not fire there.

This is why Theorem 4.16 is stated separately from Theorem 4.17. The coupling theorem mentions no guarantee, so the coupled point exists under any threshold policy. The guarantee only selects which clause applies.

The forced question is the analytic liar

The coupled point is the analytic twin of the liar index of Proposition 1.14. There the liar was an index whose self-application equalled its own negation; here q_0 is a question whose confidence equals the threshold and whose answer sits on the boundary.

Both witnesses are non-unique for the same reason. If the diagonal behavior has two names, both are liars; if the confidence crosses `1/2` more than once, every crossing landing on the boundary is a coupled point. The witness is guaranteed to exist and left unaddressed in both engines.

11.5 A worked instance on the interval

Take `Q = [0,1]`, read `t` as a position interpolating from a question answered truly at `0` to one answered falsely at `1`, and let the realized truth-distance be `F(t) = 2t - 1`. The model is covering, with `F(0) = -1` and `F(1) = 1`.

Lemma 4.14 applies: `F` has a zero, here exactly `t = 1/2`. Strict calibration forces confidence above `1/2` for `t < 1/2` and below for `t > 1/2`, so by Lemma 4.15 the continuous confidence crosses `1/2`, and only at `t = 1/2`. That is the coupled point, exactly as Theorem 4.16 promises.

Add faithfulness and it breaks: at the coupled point the confidence is exactly `1/2`, so faithfulness demands `F(1/2) < 0`, but `F(1/2) = 0`. If the confidence tries to avoid `1/2` by jumping, it is not continuous, and dropping continuity is a named escape priced in the discrete section.

The instance also shows what the exclusive guarantee buys. With the strict antecedent the guarantee does not trigger at `t = 1/2` and no contradiction arises. The model is consistent, but it owns a question on the truth boundary about which its guarantee is silent. The wall did not move; the guarantee stopped promising there.

11.6 Quantitative shadows

Theorem 4.17b (Approximate coupling). *Let `Q` be connected, the confidence continuous, and the model ε-covering, `(ε, γ)`-separating at `τ`, with a threshold-inclusive guarantee. Then some `q₀` has `(M q₀).2 = τ` and `-ε ≤ δ(q₀, (M q₀).1) < 0`.*

Proof sketch. The ε-coverage witnesses have truth-distance below `-ε` and above ε, and separation turns these into confidence witnesses above `τ + γ` and below `τ - γ`. The IVT lands `q₀` at confidence `τ`. Separation with slack no longer pins the truth-distance to zero but confines it to `|δ| ≤ ε`, where neither strict clause fires. The guarantee forces it strictly negative, giving `[-ε, 0)`. ■

The verified statement is `two_slack_approx_coupling` in `HoF_12_Approximate`, carrying ε and `γ` as separate parameters. Theorem 4.16 is the `ε = 0` limit, recovered as `exact_from_approx`.

Corollary 4.17c (Positive slack is mandatory). *Under those hypotheses the truth slack cannot be zero.*

Proof. At `ε = 0` the coupled point would lie in `[0, 0)`, which is empty, but the IVT produced one. ■

This is `truth_slack_must_be_positive`. Read it as a budget: a system keeping a threshold-inclusive guarantee must keep a strictly positive margin of correctness on the confident side, and the infimum of feasible slack is a measurable width of the truth boundary. No question with confidence between `τ` and `τ + γ` can sit exactly on the boundary, which is `no_boundary_in_upper_band_two_slack`.

11.7 Symmetry in place of coverage

Definition 4.18 (Antipodal action). An *antipodal action* on `Q` is a continuous self-inverse fixed-point-free `σ : Q → Q`. A real `g` is *odd* under `σ` when `g (σ q) = -g q`.

Verified as `AntipodalAction` and `AntipodalOdd` in `HoF_08_BorsukUlam`. Layer normalization places transformer states on a sphere-like shell, and semantic negation is a candidate antipode, so oddness idealizes a truth-distance flipping sign under negation.

Theorem 4.19 (Odd fields vanish). *Let `Q` be connected with an antipodal action `σ`. Any continuous `g : Q → ℝ` odd under `σ` has a zero.*

Proof. Pick `q`. If `g q = 0` we are done. Otherwise `g (σ q) = -g q` has the opposite sign, so `g` takes both signs and Theorem 4.7 gives a zero. ■

This is `antipodal_odd_has_zero`, and no coverage was assumed: oddness supplies the two signs for free. A single question and its negation already straddle zero.

There is a pleasant inversion. Equivariance is usually a design virtue, built in so a network treats a question and its negation consistently. Here it is the engine of the obstruction: a model respecting negation has an odd truth-

distance, and an odd continuous field on a connected symmetric space must vanish. The better behaved the symmetry, the more inescapable the wall.

Theorem 4.20 (Antipodal trilemma). *Let `Q` be connected with an antipodal action, the model continuous and strictly calibrated at `1/2`, and the realized truth-distance odd. Then some `q` has `(M q).2 = 1/2` and `δ(q, (M q).1) = 0`.*

Proof. Theorem 4.19 gives a zero `q`. Calibration then pins the confidence: above `1/2` would force the truth-distance negative, below would force it positive. ■

This is `antipodal_hallucination_trilemma`. Adding faithfulness produces the same contradiction as Theorem 4.17, now with no coverage assumption anywhere.

The two engines' nontriviality demands differ instructively. Coverage is an assumption about the model's *outputs*. The antipodal hypothesis is about the *space and the field*.

The antipodal engine is robust to approximate symmetry, which matters because real negation does not reflect truth directions exactly. If `|g (σ q) + g q| ≤ ε` for all `q`, then some `q` has `|g q| ≤ ε/2`, recovering exact vanishing at `ε = 0`. The verified robust form is `approx_odd_near_zero`, and the sphere instantiation for `d ≥ 2`, using Mathlib's `isConnected_sphere`, is `sphere_odd_has_zero`.

The dimension hypothesis `d ≥ 2` is where connectedness of the sphere comes from; the zero-dimensional sphere is two points and disconnected, the same degeneracy that makes the diagonal the right tool for a two-point outcome set. For any representation dimension used in practice the sphere is connected. The one honest caveat is that exact oddness is an idealization; the measured defect is nonzero but small, smallest for the strongest model, and `approx_odd_near_zero` keeps a near-boundary point in existence under it.

11.8 The discrete core

The topology does exactly one job: it manufactures a boundary question. Everything after is arithmetic. The verified home is `HoF_10_PureDiscrete`.

This matters because token spaces are finite and one may object that connectedness does illegitimate work. The discrete analysis prices the objection exactly: without topology we must assume the boundary witness rather than derive it. That is the entire cost.

Pricing an objection is the honest way to handle an idealization. A theorem with a strong hypothesis invites the reader to reject it and walk away; the disciplined response is to show how much of the conclusion survives without it. Connectedness is a good model of an ambient representation space over which probes are deployed at interpolated points, and a poor model of a scattered token-driven reachable set.

```
theorem c4_flip_no_fixed_point : ∀ b : Bool, (!b) ≠ b :=
  by decide
```

Model confidence and truth-distance as integers with threshold `0`, calibration as two biconditionals, faithfulness as the implication that non-negative confidence forces a strictly negative truth-distance.

```
theorem c4_boundary_question_impossible
    (conf truth : Int)
    (cal_pos : 0 < conf ↔ truth < 0)
    (cal_neg : conf < 0 ↔ truth > 0)
    (faithful : 0 ≤ conf → truth < 0)
    (boundary : truth = 0) : False := by
  have hnp : ¬ (0 < conf) := by
    intro h; have := cal_pos.mp h; omega
  have hnn : ¬ (conf < 0) := by
    intro h; have := cal_neg.mp h; omega
  have hz : conf = 0 := by omega
  have hlt : truth < 0 := faithful (by omega)
  omega
```

This is the integer transcription of `boundary_question_impossible`. Read it against Theorem 4.17: the two `have` blocks are calibration pinning the confidence, the step Theorem 4.16 performed with the IVT replaced by an assumed boundary value; the last lines are faithfulness and the sign clash. No topology appears. Both the continuous and antipodal trilemmata factor through this core.

How would a discrete world produce a boundary value? Along a finite path the truth-distance need not hit zero; it can jump across between adjacent steps.

```
theorem c4_discrete_sign_change (g : Nat → Int) :
    ∀ n, g 0 < 0 → 0 ≤ g n → ∃ i, i < n ∧ g i < 0 ∧ 0 ≤ g
    (i + 1) := by
  intro n
  induction n with
  | zero => intro h0 hn; exact absurd hn (by omega)
  | succ m ih =>
    intro h0 hn
    by_cases hm : 0 ≤ g m
    · obtain ⟨i, hlt, hneg, hpos⟩ := ih h0 hm
      exact ⟨i, Nat.lt_succ_of_lt hlt, hneg, hpos⟩
    · exact ⟨m, Nat.lt_succ_self m, by omega, hn⟩
```

A discrete field negative at the start and non-negative at the end has an adjacent pair where it is still negative and has just become non-negative. It never claims a step where the field is exactly zero. That gap is the whole difference between settings: the continuum upgrades the straddle into an exact witness. The verified counterpart is `discrete_sign_change`.

Two consequences, both verified. A model faithful and strictly calibrated with witnesses for strictly correct, exactly boundary, and strictly wrong answers is impossible with no topological hypothesis, which is `discrete_hallucination_trilemma`; it trades connectedness for a stronger coverage hypothesis. And, more revealingly, faithfulness is *equivalent* to boundary-freeness under calibration: `faithful_iff_boundary_free`.

Adding randomness

A stochastic model does not dodge the obstruction, in two verified ways.

On expected fields, averaging is continuous, so the averaged confidence and truth-distance are continuous on the connected space and the coupling theorem applies verbatim, giving a question with expected confidence `1/2` and expected truth-distance `0`. This is `stochastic_coupling_expected`.

The second converts expectation into samples and is the one place the theory derives outright falsehood. Let the truth-distance have mean zero, let sample-faithfulness hold almost everywhere, and let the confidence be at least `1/2` with positive probability. Then the truth-distance is strictly positive with positive probability. This is `stochastic_dichotomy`.

Proof sketch. If the truth-distance were almost never positive, then on the confident event, of positive probability, faithfulness makes it strictly negative, and it is non-positive elsewhere. Such a quantity has strictly negative mean, contradicting mean-zero. ■

Randomization spreads the boundary across samples instead of removing it. The wall is either a point you pass through or a positive-probability event you incur.

11.9 Where the two engines meet

The claim is that the diagonal and the IVT locate the *same* boundary object, and that the Boolean flip and the real complement are two descriptions of one wall. This is recorded as a theorem in `Foundation.F_04`.

In Chapter 1 the engine needed a flip with no fixed point, and for Booleans that is negation. For real confidences the natural flip is the *complement controller* `y ↦ 1 - y`, which is not fixed-point-free: it has exactly one fixed point, at `1/2`. This is `half_complement_fixed_point`, with the off-threshold form `complement_no_fp_off_half`.

That single fixed point is the entire difference, and the reason both engines reach the same place. The Boolean flip has none, so Lawvere's theorem, which always produces a fixed point of the flip, produces a contradiction. The real complement has one, so Lawvere run with it produces not a contradiction but a point: a diagonal question where the confidence equals `1/2`.

Compare the routes. The IVT reaches `1/2` by connectedness: the confidence is below and above, so it crosses. The diagonal reaches it by self-reference: the confidence names a behavior complementing its own diagonal value, so it equals its own complement.

What is shared is the boundary object and the final clash. What differs is the hypothesis manufacturing the object, and neither implies the other. A finite discrete model can be universal and carry no topology; a smooth model on a manifold can be connected while naming almost none of its own behaviors.

There is a categorical way to say the shared part. Lawvere's theorem holds cleanest for a *section*, and that form is true verbatim in any cartesian closed category. The complement controller is an endomap of the reals with a single fixed point, and Lawvere applied to a section of a real-valued self-prediction delivers it on the diagonal.

The unification is packaged at two levels: `hallucination_unification_documentation` and its lemmas in `Foundation.F_04`, and `trilemmata_unified` with `hallucination_master_theorem` in `F_13_UnifiedTrilemmata`.

One geometric remark closes the meeting point. When the truth-distance is a linear probe the boundary is an affine hyperplane of codimension one, a translate of the probe's kernel, by rank-nullity: `linear_probe_boundary_coset` and `linear_probe_boundary_dim`. For a differentiable nonlinear probe it is a hyperplane to first order at every regular point, with the tangent hyperplane the kernel of the derivative, verified as `nonlinear_boundary_tangent` and `tangent_hyperplane_dim`, and every regular point is itself a sign crossing generating local coverage for free, `regular_point_is_crossing`.

11.10 Where the two engines diverge

The diagonal gives a witness and nothing quantitative: not how many liars there are, how hard one is to find, or what a search would cost. It cannot, because a naming equation has no notion of distance, width, or margin.

The IVT alone is equally silent. The difference is that the analytic engine can be fed more. Add a distance and Lipschitz maps and the bare crossing becomes quantitative: the decisive regions are held apart by a distance controlled by the Lipschitz constant and the margin, the coupled point sits at the centre of a tube of computable radius, and truth-side slack must be strictly positive with a measurable infimum.

Use the diagonal when the system names itself and you want the cleanest existence proof with the smallest trust story. Use the IVT when the system lives on a connected space, and if you also have a metric, keep going into Chapter 5. Both reach the boundary; only the second measures it.

The engines also differ in what they say about escape. The diagonal is escaped by breaking universality, which is hard to arrange without crippling a system meant to handle arbitrary inputs. The analytic engine is escaped by breaking connectedness, coverage, continuity, or exact separation, and each is a concrete design lever.

11.11 The engine in one page

Four moves. *Continuity:* the realized truth-distance and confidence are continuous, being built from continuous pieces by composition and pairing. *The intermediate value theorem:* on a connected space a continuous real field taking values on both sides of a threshold takes the threshold; this is the only analytic input, verified as `IsPreconnected.intermediate_value₂`.

Every variant is a substitution. The antipodal variant replaces coverage with a symmetry supplying both signs. The discrete core keeps the last two moves and assumes the boundary question.

11.12 Historical notes

The intermediate value theorem is Bolzano's, and the formulation as preservation of connectedness is standard. The reading of it as an impossibility engine for learning systems is recent, and the machine-checked development is `HallucinationProofs`, with the topological core in `HoF_03`, `HoF_05`, `HoF_07`, the antipodal variant in `HoF_08`, and the discrete core in `HoF_10`. The unification with the diagonal is `Foundation.F_04`.

The statistical inevitability results run on a different track: they argue a calibrated model must hallucinate on rarely-seen facts, or that hallucination is undecidable, and conclude that confident falsehood is unavoidable in aggregate. The engine here is topological, localizes at a specific boundary question, and is more modest, since the deterministic theorems yield uncertainty rather than confident error. Only the stochastic dichotomy derives falsehood, and it needs mean-zero to do it.

12 The Geometry of Attacks

The diagonal gives one thing and refuses the rest. It gives a bad point. It never says how many there are, how large a region behaves badly, how a search finds one, or what an adversary spends. Those questions are outside its language: it speaks of surjections and fixed-point-free maps, not volume, distance, or gradient.

Here the objects are real functions on real spaces: an *alignment deviation* `f : X → ℝ`, a threshold `τ`, and a metric. The tools are analysis and measure theory. No result here is a diagonal argument; they are the intermediate value theorem, the Lipschitz condition, a coarea inequality, concentration of measure, and pigeonhole on grids.

The material follows the companion `ManifoldProofs` development, roughly four hundred machine-checked results on Mathlib. Because that library is analytic and the book's own package carries no Mathlib, this chapter states everything in prose, tagged with the exact `MoF_*` name. Treat the citations as ground truth: where the prose simplifies, the Lean does not.

One idea runs throughout. *Existence is qualitative; cost is quantitative.* The earlier impossibility theorems say the boundary is there. These say how it is shaped, how much room the unsafe region occupies, and how the price of attack and defense scales. A deployment is not decided by whether a bad point exists, since one always does, but by how expensive it is to live next to it.

12.1 From the diagonal to the boundary

Fix a space `X` of behaviors, a continuous `f : X → ℝ` large when behavior is far from acceptable, and a threshold `τ`. This triple is the entire setting.

Definition 5.1 (Regions). The *safe region* is `{f < τ}`, the *vulnerability basin* `{f > τ}`, the *boundary* `{f = τ}`, the *failure manifold* `{f ≥ τ}`. These are `SafeRegion`, `UnsafeRegion`, `Boundary`, `FailureManifold` in `MoF_01_Foundations`, with `space_partition`, `safe_unsafe_disjoint`, and `failureManifold_eq_union` the immediate facts.

In Chapter 1 the boundary question was the index where a boolean verdict flipped. Here it is a point where `f = τ`, neither safe nor unsafe. The diagonal produced its witness by self-reference; this one comes from connectedness.

Both deliver an unavoidable point of failure, and the analytic one delivers it with metric content.

Remark 5.2. Keep one picture: `f` is a landscape, the safe region the lowland below sea level `τ`, the boundary the coastline. The questions here are geographic. How much coastline? How steep the cliffs? Drop a random point: how likely near the coast? The impossibility theorems say the coastline exists; the geometry says how to sail it.

12.2 Decision basins

Theorem 5.3 (Basin openness). *For continuous `f`, `{f > τ}` and `{f < τ}` are open; `{f = τ}` and `{f ≤ τ}` are closed.*

Proof. Preimages of open rays are open; the boundary is the preimage of a closed singleton. ■

This is `basin_isOpen` and `sublevel_isClosed` in `MoF_02_BasinStructure`, with `safeRegion_isOpen`, `unsafeRegion_isOpen`, `boundary_isClosed` in `MoF_01`. Unsafety is not razor-thin: it comes with room around it.

Theorem 5.4 (Every unsafe point owns a ball). *If `f p > τ` then some `r > 0` has `B(p, r)` inside the basin.*

Proof. The basin is open and contains `p`. ■

`basin_contains_ball`. Continuity gives a ball of unknown size; Lipschitz continuity, below, gives one of known size.

Theorem 5.5 (Basins have positive measure). *For any measure giving nonempty opens positive mass, if some `f p > τ` then `μ{f > τ} > 0`.*

Proof. The basin is open and nonempty. ■

`basin_measure_pos`, with `basin_measure_ge_ball_pos` in `MoF_Cost_02_BasinVolume`. This is the hinge of the cost story. A single unsafe point is measure-zero and dismissible; positive measure is not. A random draw has positive chance of landing there, and no finite set of safe examples certifies the region away.

Theorem 5.6 (Monotonicity in the threshold). *If `τ₁ ≤ τ₂` then `{f > τ₂} ⊆ {f > τ₁}` and the measures are ordered. Above the supremum the basin is empty; below the infimum it is everything.*

Proof. Raising the threshold removes points; measure is monotone. ■

`basin_measure_monotone_threshold`, `basin_empty_above_sup`, `basin_full_below_inf` in `MoF_Adv_10_MeasureBounds`. You can shrink the basin by refusing more behaviors, but it shrinks continuously and never vanishes until you have refused essentially everything. A threshold high enough to empty the basin empties the product.

Theorem 5.7 (Components are open and countable). *On a locally connected space each component of the basin is open; if second countable, there are countably many.*

Proof sketch. Components of an open set in a locally connected space are open, and a disjoint family of nonempty opens in a second countable space is countable. ■

`basin_connected_components_open`, `basin_components_countable` in `MoF_Adv_01_BasinConnectedness`, with `basin_connected_of_convex_domain` a sufficient condition for one piece and `superlevel_disconnected_example` an `f` whose basin splits. Real models are the second kind, which is why red-teaming that finds one jailbreak family says little about the others.

Worked example: the Gaussian bump. On `ℝ^n` let `f(x) = M · exp(-‖x‖²/2)` with `0 < τ < M`. The basin is the open ball of radius `√(2 ln(M/τ))`: open, nonempty, positive measure, connected, shrinking continuously as `τ → M`. It also previews the curse, since the ball's volume is a vanishing fraction of the surrounding cube in high dimension, so a random draw almost never lands in it.

12.3 The threshold-crossing theorem

Theorem 5.8 (Threshold crossing, one dimension). *For continuous `f : ℝ → ℝ` with `f a < τ < f b`, some `c ∈ [a,b]` has `f c = τ`.*

Proof. The IVT on a connected interval. ■

`path_crosses_threshold` in `MoF_03_ThresholdCrossing`. Its elementariness is the point: unavoidability of the boundary is the IVT in a safety costume.

Theorem 5.9 (Crossing along a path). *For continuous `γ` and `f` with `f(γ a) < τ < f(γ b)`, some `c` has `f(γ c) = τ`.*

Proof. Theorem 5.8 on the composite. ■

`path_crosses_threshold_generic`. An attack is a path from safe to unsafe, whether prompt edits, an embedding interpolation, or a gradient trajectory. If it is continuous and succeeds, it passed through the boundary. There is a moment in every successful attack when the behavior was exactly at threshold, which is why the boundary, not the interior, is what a defense must control.

This also refutes the moat. For a continuous score, `f` takes every intermediate value, so you cannot forbid a band of scores without forbidding behaviors that legitimately produce them. The only genuine separator is the measure-zero level set, and Theorem 5.20 shows even that thickens once the defender's uncertainty is counted. There is no moat, only a coastline.

Theorem 5.10 (No gap). *On connected `X`, if a safe and an unsafe point exist then `{f = τ}` is nonempty.*

Proof. The reachable values form a connected subset of `ℝ` containing values on both sides. ■

`no_gap_theorem`, on `threshold_level_set_nonempty`. A designer might hope for a model whose behaviors are all comfortably safe or comfortably unsafe with an empty margin, so a crude classifier could separate them. Connectedness forbids it.

Theorem 5.11 (Separation is exactly the boundary). `closure(Safe) ∩ closure(Unsafe) ⊆ Boundary`.

Proof sketch. If `f x > τ` a whole neighborhood is unsafe, contradicting `x` being a limit of safe points; symmetrically below. ■

`safe_unsafe_separated`. The boundary is the shared frontier where the regions touch, and everything an attacker exploits and a defender guards lives on this closed set.

Theorem 5.12 (Defense fixes a boundary point). *Let `X` be connected Hausdorff with safe and unsafe points present, and `D` continuous and the identity on the safe region. Then some boundary point `z` has `D z = z`, so `D` cannot make `z` safe.*

Proof sketch. The fixed-point set of a continuous self-map of a Hausdorff space is closed, being the preimage of the diagonal, and contains the safe region hence its closure. The safe region is open and not closed, since connectedness would force it empty or everything, so its closure contains a boundary point, which is a limit of safe points and therefore fixed. ■

`master_theorem` in `MoF_MasterTheorem`, over the `ManifoldOfFailure` structure. It does not say `D` fails everywhere, only that one boundary point is fixed and unfixed. Upgrading that point to a region of positive measure is the work of the quantitative theorems.

Worked example: interpolating prompts. A benign prompt at `f p = τ - 1` and a jailbreak at `f q = τ + 1` are joined by a continuous path, so Theorem 5.9 gives a prompt scored exactly at threshold: a marginal jailbreak where a vanishing nudge tips it unsafe. That family is exactly what red teams hunt.

12.4 Lipschitz control

`f` is *`L`-Lipschitz* when `|f x - f y| ≤ L · dist(x,y)`. Networks with bounded weights and Lipschitz activations are Lipschitz, with a constant that is finite and estimable.

Definition 5.13 (Robustness radius). `robustnessRadius(f p, τ, L) = (f p - τ)/L`, in `MoF_04_LipschitzBasin`.

Theorem 5.14 (Lipschitz basin ball). *If `f` is `L`-Lipschitz with `f p > τ`, the ball of radius `(f p - τ)/L` around `p` lies inside the basin.*

Proof. `|f p' - f p| ≤ L · dist < f p - τ`. ■

`lipschitz_basin_ball`, with `basin_ball_subset`, `perturbation_stability`, and `robustnessRadius_pos`. The radius is the margin divided by how fast `f` can climb back down. This is a statement about the robustness of *attacks*: once well inside the unsafe region an entire neighborhood is unsafe, so small changes in phrasing do not rescue the model.

Theorem 5.15 (Deeper points have larger basins). *If `f p₁ ≤ f p₂`, both above `τ`, their radii are ordered the same way.*

Proof. `(v - τ)/L` increases in `v`. ■

`robustnessRadius_monotone`. The more deviant the behavior, the more robust to perturbation, which attackers exploit by pushing deep rather than stopping at the first crossing.

Theorem 5.16 (Smoother models, larger basins per peak). *A smaller Lipschitz constant gives a larger radius at fixed peak value.*

Proof. `(f p - τ)/L` decreases in `L`. ■

`smoother_functions_larger_basins` in `MoF_Adv_04_ModelScale`, with `lipschitz_basin_component_width`. Smoothness is usually considered desirable and is often enforced by regularization, but a smoother model has larger unsafe basins per peak. Smoothness does not remove the danger; it spreads it, while steepness concentrates it into thin spikes, harder to hit by chance and sharper when hit.

Is the Lipschitz assumption fair? A large `L` shrinks the guaranteed ball, which sounds like relief, but a large `L` is what a model needs to draw sharp narrow basins, and a large local growth rate is what defeats the defense. A large constant does not make a model safe; it makes the failures spiky, and spiky failures are just as easy to hit by gradient, since the gradient is largest where the surface is steepest.

Theorem 5.17 (Oscillation needs steepness). *If `f : [0,1] → ℝ` is `L`-Lipschitz with `f 0 = f 1 = 0` and `f t = M > 0`, then `L ≥ 2M`.*

Proof. `M ≤ L·t` and `M ≤ L·(1-t)`; add. ■

`higher_lipschitz_more_oscillation`. Sharp transitions require a large `L`, and a large `L` is what the cost theory charges for.

Worked example: the linear ramp. For `f(x) = ⟨w,x⟩ + b` with `‖w‖ = L` and `f p = τ + m`, the guaranteed ball of radius `m/L` is tight: moving along `-w/‖w‖` decreases `f` at maximal rate, so the ball is inscribed in the basin, kissing the boundary at the foot of the perpendicular. Every later theorem about basin volume is a version of inscribing balls like this.

12.5 The coarea and cone bounds

Definition 5.18 (The ε-band). `coareaBand(f, τ, ε) = {τ - ε ≤ f ≤ τ}`, in `MoF_17_CoareaBound`.

Theorem 5.19 (Ball inside the band). *If `f` is `L`-Lipschitz and some `c` has `f c = τ - ε/2`, then `B(c, ε/(4L))` lies in the band.*

Proof. Across the ball the value moves by less than $\varepsilon/4$. ■

`lipschitz_ball_subset_band`; the centre sits mid-band so the ball cannot escape either wall.

Theorem 5.20 (Coarea volume lower bound). *`μ(band) ≥ μ(B(c, ε/(4L)))`; in `ℝ` this is `ε/(2L)`, and in `ℝ^n` it grows like `(ε/(4L))^n`.*

Proof. Monotonicity under Theorem 5.19. ■

`epsilon_band_volume_lower_bound`, with real and Euclidean specializations. The centre is free on a connected space by the IVT (`exists_midpoint_for_band`), making the argument unconditional (`epsilon_band_volume_pos_unconditional`). The band always has positive, computably bounded measure: the boundary is not a lonely level set but the spine of a slab.

Remark 5.21. A defender's classifier has an uncertainty band of width ε, and the true behaviors falling in it occupy volume at least $\varepsilon/(2L)$. Sharpening shrinks it linearly, which is progress, but never to zero while $\varepsilon > 0$.

Definition 5.22 (Steep region). `steepRegionR(f, τ, s, z) = {x | f x > τ + s·|x - z|}`, in `MoF_18_ConeBound`, with `steepRegion` in `MoF_11_EpsilonRobust` and `steepRegionRefined` in `MoF_20_RefinedPersistence`.

Theorem 5.23 (Defense distortion bound). *If `f` is `L`-Lipschitz, `D` is `K`-Lipschitz, and `D z = z`, then `f(D x) ≥ f x - L(K+1)·dist(x,z)`.*

Proof. `dist(D x, x) ≤ (K+1)·dist(x,z)`, and `f` changes by at most `L` times that. ■

`defense_from_input_bound_R`. The compound `L(K+1)` is the defense's maximum reach: how far below its input value it can pull the score. If `f` rises faster in some direction, the defense loses.

Theorem 5.24 (Cone measure bound). *If `f z = τ`, `D z = z`, and `f(z + t) ≥ τ + c·t` on `(0, δ₀)` with `c > L(K+1) =: s`, then that interval lies in the steep region, the steep region has measure at least `δ₀`, and every steep point stays unsafe after defense.*

Proof. `f x ≥ τ + c·t > τ + s·t` gives the inclusion and the measure; Theorem 5.23 then gives `f(D x) > τ + s|x-z| - s|x-z| = τ`. ■

`cone_measure_bound`, assembling `cone_segment_in_steep_region`, `steep_region_contains_Ioo`, `steep_region_measure_pos`, and `steep_region_measure_lower_bound`. Chapter 1 gave one surviving query, the master theorem one surviving boundary point; this gives an entire cone of positive measure that remains unsafe after the defense has done its best.

Theorem 5.25 (Shallow boundaries can be defended). *If `f x ≤ f z + L·dist(x,z)`, the steep region is empty for every `K ≥ 0`.*

Proof. `f x ≤ τ + L·dist ≤ τ + L(K+1)·dist`. ■

`shallow_boundary_no_persistence` in `MoF_19_OptimalDefense`. The two theorems draw a sharp line: steep boundaries carry persistent positive-measure failure no defense removes, shallow ones do not, and whether a real model is steep or shallow at its boundary is the most important quantitative fact about its defensibility.

Worked example: a corner in the score. With `f(x) = τ + G·x` for `x ≥ 0`, so `L = G`, persistence needs `K + 1 < 1`, impossible. The corner is not steep *relative to its own Lipschitz constant*. Genuine persistence needs `f` to rise near `z` faster than elsewhere, which is the regime the gradient chain reaches through `‖∇f(z)‖`.

12.6 Dimensional scaling

Theorem 5.26 (Grid explosion). *Discretizing `d` axes into `N` bins gives `N^d` cells, unbounded for `N ≥ 2`; covering needs at least `N^d` queries and exceeds any fixed budget.*

Proof. The grid has cardinality `N^d ≥ 2^d`, and covering is a surjection onto it. ■

`grid_card`, `grid_exponential_growth`, `queries_lower_bound_surjective`, `no_fixed_budget_defense`. Exhaustive defense is not expensive; it is impossible at any budget.

Theorem 5.27 (Coverage collapse). *A patch of relative radius `r < 1` covers fraction `r^d → 0`, and covering a fixed fraction needs `(1/r)^d → ∞` patches.*

Proof. Relative volume is the product of side fractions. ■

`coverage_fraction_vanishes`, `defense_patch_shrinks`, `exponential_base`, bundled as `DimensionalCurse`. Each thing the defender builds covers a fraction shrinking exponentially with dimension. The attacker needs one point; the defender needs almost all.

Theorem 5.28 (Boundary structure). *The boundary is closed, compact on compacta, empty exactly when `f` never equals `τ`, nonempty on a connected space when `f` straddles `τ`, and equal to `f⁻¹({τ})`.*

Proof. Theorem 5.3 plus the IVT cases. ■

`boundary_is_closed`, `boundary_is_compact_on_compact`, `boundary_empty_of_no_crossing`, `boundary_nonempty_connected`, `boundary_is_preimage_singleton`.

Theorem 5.29 (Regular points are isolated in one dimension). *With nonzero derivative at a boundary point, some punctured neighborhood has `f ≠ τ`.*

Proof sketch. A nonzero derivative makes `f` strictly monotone near `z`. ■

`regular_value_boundary`, `boundary_locally_finite_1d`. Where regular, the boundary is thin and well-behaved; where the gradient degenerates it can thicken and fold, and whether real boundaries are regular or fractal is an open empirical question.

Theorem 5.30 (Complexity and crossings). *A constant model has a trivial basin, a monotone model crosses at most once, and oscillation with many components requires `L ≥ 2M` to reach height `M`.*

Proof. The monotone case squeezes `f` to `τ` between crossings; the rest is Theorem 5.17. ■

`constant_model_no_basin_empty`, `constant_model_no_basin_univ`, `low_complexity_few_crossings_1d`, `higher_lipschitz_more_oscillation`. A bigger model is not automatically safer geometry; it is capable of a more complicated failure landscape.

Worked example: random points in the cube. In high dimension almost all volume is near the surface and distances concentrate. The attacker sampling at random lands in the peripheral bulk where an appreciable basin is found; the defender watches each patch cover an `r^d` sliver of that same bulk.

12.7 Gradient attacks

Theorem 5.31 (An ascent direction exists). *If `f` is differentiable at `x` with `f' ≠ 0`, some direction and step raise `f`.*

Proof sketch. Pick `v` with `f' v > 0` and restrict to the line. ■

`ascent_direction_exists`, `multivariate_ascent_direction` in `MoF_10_GradientAttack`, on `discrete_ascent_improvement` and `hasFDerivAt_line_restrict`. This is the licence for gradient-based adversarial methods: away from critical points there is always a way to raise the score, and critical

points are where the gradient vanishes (`critical_point_iff_fderiv_eq_zero`).

Theorem 5.32 (The gradient is steepest). *Among unit directions `⟨g, v⟩` is maximized at `g/‖g‖`, giving directional derivative `‖g‖`.*

Proof. Cauchy-Schwarz. ■

With `fderiv_apply_eq_inner_gradient` and `real_inner_self_eq_norm_sq'`. This is why attacks follow the gradient: it reaches the boundary in the fewest steps.

Theorem 5.33 (Near-optimal direction). *If `‖f'‖ > c` at `z`, some unit `v` has `f' v > c`.*

Proof sketch. Normalize a vector of norm below `1` on which `f'` exceeds `c`. ■

`near_optimal_direction`.

Theorem 5.34 (Derivative forces local growth). *If `f z = τ` and `f' v > c` for a unit `v`, then `f(z + tv) > τ + c·t` for small `t > 0`.*

Proof sketch. `g(t) = f(z + tv) - τ - ct` has `g(0) = 0` and positive derivative. ■

`deriv_implies_local_growth`, with `deriv_implies_strict_local_growth`.

Theorem 5.35 (Gradient chain to persistence). *If `f` is differentiable at a boundary point with `‖f'‖ > ℓ(K+1)`, the steep region is nonempty and `{f(D x) > τ}` has positive measure.*

Proof sketch. Theorem 5.33 gives a direction beating `ℓ(K+1)`, Theorem 5.34 turns it into local growth placing a point in the steep region, and Theorem 5.24 upgrades that to a positive-measure surviving set. ■

`gradient_norm_implies_steep_nonempty`, `gradient_chain_persistent_unsafe`, invoking `persistent_unsafe_refined`. Measure the gradient at the boundary; if it beats the defense's smoothing budget, following it reaches behaviors the defense cannot save, not one but a set of positive volume. The single number `‖∇f(z)‖` decides whether the model is persistently attackable. This is `transversality_from_deriv` in its sharpest form.

Remark 5.36. Chapter 1's attack was a construction with no notion of effort. The gradient attack is a process with a step count, `(τ - f(start))/gain`, which is a real budget. Both reach the same boundary; only the second has a price tag.

12.8 Transferability

Say `f` and `g` are *δ-close* when `|f x - g x| ≤ δ` everywhere.

Theorem 5.37 (Transfer with margin). *If `f` and `g` are δ-close and `f p > τ + δ`, then `g p > τ`.*

Proof. `g p ≥ f p - δ > τ`. ■

`transfer_attack` in `MoF_06_Transferability`, restated as `transfer_margin_needed`. The margin is the toll for crossing between models: an attack merely clearing `τ` on the source may fail on the target, one clearing

`τ + δ` transfers. Attackers know this when they over-optimize, and over-optimization by δ is precisely what buys transfer.

Theorem 5.38 (Basin containment). *`{f > τ + δ} ⊆ {g > τ}`, with equality of basins at `δ = 0`.*

Proof. Theorem 5.37 pointwise. ■

`basin_containment`, `identical_models_identical_basins`, `zero_distance_free_transfer`, with the two-sided `pointwise_sandwich`.

Theorem 5.39 (Transfer is computationally free). *Given δ-closeness and a source success with margin δ, the target success follows with zero target evaluations.*

Proof. No term in Theorem 5.37's proof evaluates `g` at a new point. ■

`transfer_is_free` in `MoF_Cost_07`. An attacker who breaks one model gets every δ-close model free. Model families trained on similar data, or fine-tunes of a common base, are δ-close for small δ, so one attack sweeps the family, and closed deployment of a model whose base is public offers less protection than it appears.

A defense classifier is a function on the same space, so to the extent it agrees with what it guards it inherits its vulnerabilities, and to the extent it disagrees it misclassifies safe behavior. Closeness buys agreement at the price of shared failure; distance buys independence at the price of collateral refusals.

Theorem 5.40 (Empirical closeness underestimates). *An empirical maximum disagreement is at most the true δ, and more samples raise it.*

Proof. A maximum of quantities bounded by δ is bounded by δ. ■

`estimation_cost`, `more_samples_better_estimate`. The estimation cost is paired evaluations, not an exponential quantity: transfer is cheap to plan and free to execute.

Worked example: a fine-tune and its base. If a light fine-tune changed the score by at most `δ = 0.1`, an attack with margin `0.15` transfers to the base and one with margin `0.05` need not. Safety fine-tuning with small δ leaves the base model's attacks almost entirely intact.

12.9 The cost theory

The `MoF_Cost` series builds budgets in ten layers, and the punchline is one asymmetry: attack cost stays bounded while defense cost explodes with dimension.

Theorem 5.41 (Ball volume). *An open ball of positive radius has positive volume, monotone in the radius; in $\mathbb{R}$ it is `2r`.*

Proof. Nonempty opens have positive Lebesgue measure. ■

`ball_measure_pos`, `ball_measure_mono`, `ball_measure_scale` in `MoF_Cost_01_BallVolume`.

Theorem 5.42 (Basin volume lower bound). *The basin contains the robustness ball, so its measure is at least the ball's; for merely continuous `f` it is still positive.*

Proof. Theorem 5.14 as a subset relation, plus Theorem 5.5. ■

`robustness_ball_subset_basin`, `basin_measure_ge_ball`, `basin_measure_ge_ball_pos`, with `basin_measurableSet`. The attacker's target is a ball of volume at least `(margin/L)^n` times a dimensional constant.

Theorem 5.43 (Random search hitting time). *With per-trial hit probability `p`, the miss probability `(1-p)^n` decreases to zero and the expected number of trials is `1/p`.*

Proof sketch. Powers of a number below one vanish; the geometric series sums to `1/p`. ■

`miss_probability_vanishes`, `hit_probability_tends_to_one`, `geometric_sum_le_inverse`, `tsum_geometric_eq_inv` in `MoF_Cost_03_HittingTime`. When `p` is a constant fraction this is cheap; the defender's only real hope is that `p` is exponentially small in high dimension, which is where gradient attacks earn their keep by replacing `1/p` with a linear step count.

Theorem 5.44 (Estimation margin). *If `|f^ - f| ≤ ε`, then `f^ x > τ + ε` implies `f x > τ` and `f^ x < τ - ε` implies `f x < τ`; the undecidable band is `{|f^ - τ| ≤ ε}`.*

Proof. Rearranging the absolute-value inequality. ■

`estimation_error_bound_unsafe`, `estimation_error_bound_safe`, `estimation_uncertain_band`, `estimation_band_shrinks` in `MoF_Cost_04_Concentration`. That band is exactly the coarea band: the band the classifier cannot resolve is the band the geometry says has positive volume.

Theorem 5.45 (Lipschitz interpolation). *Holding the error below `ε` throughout `[0,1]^d` needs samples no farther than `ε/L` apart, hence on the order of `(L/ε)^d`.*

Proof. The Lipschitz condition plus a covering count. ■

`lipschitz_estimation_error`, `estimation_improves_with_distance`, `total_estimation_budget` in `MoF_Cost_09_LipschitzEstimation`, with `sup_norm_triangle` and `n_samples_coverage`. The defender's estimation problem inherits the curse directly, and concentration works against it: samples spread thin, so the interpolation bound is loose exactly where it matters.

Theorem 5.46 (Attack cost). *An attack gaining `δ` per step on a score in `[0,1]` reaches `τ` from `s₀` in at most `⌈(τ - s₀)/δ⌉ + 1` steps and takes at most `⌈1/δ⌉` useful steps.*

Proof sketch. After `n` steps the score is at least `s₀ + nδ`, and it cannot exceed 1. ■

`attack_value_after_n_steps`, `steps_to_threshold`, `attack_cost_upper_bound`, `total_attack_cost` in `MoF_Cost_05_AttackCost`, with `attackSeq_convergent` and `finite_steps_bound`. The budget is `1/δ`, independent of dimension, because the attacker follows a one-dimensional trajectory and each step gains a fixed amount regardless of how many dimensions surround the path.

Theorem 5.47 (Defense cost). *Covering with patches of relative radius `r < 1` needs on the order of `(1/r)^d` patches; any budget has a critical dimension.*

Proof. Fraction `α` needs `α·(1/r)^d` patches. ■

`patches_needed`, `defense_cost_exponential`, `critical_dimension` in `MoF_Cost_06_DefenseCost`.

Theorem 5.48 (Transfer cost). *Porting an attack across δ-close models costs zero target queries, and estimating δ grows with precision, not dimension.*

Proof. Theorems 5.39 and 5.40. ■

`MoF_Cost_07_TransferCost`. Transfer adds essentially nothing to the attack budget while multiplying its reach across a family.

Definition 5.49 (Cost ratio). `costRatio(N, d, δ) = N^d · δ`, in `MoF_Cost_08_CostRatio`, bundled as `unifiedCostRatio` on `CostParameters` in `MoF_Cost_10_UnifiedTheory`, with `unifiedCostRatio_eq`.

Theorem 5.50 (Cost asymmetry, master theorem). *For fixed `N ≥ 2` and `δ > 0` the ratio is positive, increasing, and divergent in `d`, exceeding any `R` beyond a critical dimension. The attacker's budget is dimension-independent; the defender's is exponential.*

Proof sketch. A positive constant times an exponential in `d`; the attack budget `1/δ` contains no `d`. ■

`cost_ratio_tends_to_infinity`, `cost_ratio_exceeds_any_bound`, `attacker_advantage_quantified`, `MASTER_THEOREM_cost_asymmetry`, with `attack_budget_dimension_independent` and `defenseBudget_exponential`. As a concrete anchor, `two_dimension_ratio` gives `6.25` already at `d = 2`, `N = 25`, `δ = 0.01`.

This is the chapter's quantitative summary. The boundary exists, is populated by a positive-measure region, is found at a dimension-independent cost, and is covered only at a dimension-exponential one. The ratio diverges and no budget closes the gap.

Theorem 5.51 (No K wins). *Either `G > L(K+1)` and a persistent unsafe region of positive measure survives, or `L(K+1) ≥ G` and the failure band has width at least `G·δ`. For `G > L` the critical `K* = G/L - 1` is where the modes trade off exactly and neither horn is escapable; for `G ≤ L` the boundary is shallow and defensible.*

Proof sketch. A case split on one real comparison. Increasing `K` strictly increases `L(K+1)` (`defense_band_monotone`), flipping the first horn into the second with equality at `K*` (`optimalK_critical`), and both horns are realized for `G > L` (`defense_dilemma_both_realizable`). ■

`defense_cannot_win`, `optimal_K_exists`, `complete_defense_characterization` in `MoF_19_OptimalDefense`. A gentle defense leaves persistent cones; an aggressive one smears the failure band wide, distorting behavior over a region growing linearly in `K`. At `K*` the defender has minimized total damage but not eliminated it.

Worked example: the cost ratio. With `d = 20`, `N = 10`, `δ = 0.05`, the attacker pays about twenty steps and the defender `10^20` cells. Double the dimension and the attacker still pays twenty steps while the defender pays `10^40`. Nothing the defender does to `N`, `r`, or `K` touches the exponential in `d`.

The one honest pushback is the word "effective": the exponent is the dimension of the manifold the attack actually explores, and if that is low the ratio is manageable. The cost theory does not measure it for you; it tells you that this one number, not model size or defender cleverness, decides the contest.

12.10 Concentration of measure

Theorem 5.52 (Markov bound on the basin). *For integrable $f \geq 0$, $\mu\{f \geq \varepsilon\} \leq (1/\varepsilon)\int f \, d\mu$.*

Proof. The integrand is at least ε on $\{f \geq \varepsilon\}$. ■

`markov_real_basin` in `MoF_Adv_10_MeasureBounds`. A model whose average deviation is small cannot have a large high-deviation basin, which sounds like comfort but sets a floor, because the average is a property of the whole model and lowering it means changing the model everywhere at once.

The concentration phenomenon. On the sphere or under a Gaussian in $\mathbb{R}^n$, an L-Lipschitz f deviates from its median by more than t with probability at most about $2\exp(-ct^2/L^2)$: it is nearly constant across almost all of a high-dimensional space. This is Lévy-Milman-Talagrand concentration, whose consequences are exactly the cost asymmetries the library proves.

Concentration and the curse are the same fact. With τ above the median the basin is the far tail, exponentially small in $(\tau - m)^2/L^2$ — good for the defender in that a random behavior is almost never unsafe, catastrophic in that Theorem 5.5 still guarantees the tail is nonempty, and the gradient chain skips the $1/p$ wait by climbing straight to it. Concentration hides the basin from random draws and does nothing to hide it from gradients.

Theorem 5.53 (Positive-measure failure band). *On a connected Hausdorff metric space with a positive-on-opens measure, L-Lipschitz f, a K-Lipschitz defense fixing the safe region, and a transverse boundary point, the set $\{f(D\,x) > \tau\}$ has strictly positive measure.*

Proof sketch. Transversality places a point in the steep region, distortion shows steep points stay unsafe, and the steep region is open and nonempty. ■

`epsilon_robust_impossibility`, `positive_measure_failure_band`, `persistent_unsafe_region` in `MoF_11_EpsilonRobust`. This is the strongest form of the defense impossibility here, giving a surviving set of positive measure in full generality under honest Lipschitz hypotheses on both model and defense.

Remark 5.54 (The two faces). Toward the defender, concentration says samples spread thin and the nearest sample is far, so the interpolation bound is loose everywhere. Toward the attacker it says the space is one undifferentiated bulk where a Lipschitz score is nearly constant, so a gradient points reliably toward the rare tail. The defender sees a space too big to cover; the attacker sees one too smooth to hide the exit.

Worked example: the Gaussian annulus. Under a standard Gaussian, $\|x\|$ concentrates around $\sqrt{n}$ in a window of width about 1, so almost all mass sits in a thin annulus. Take $f(x) = \|x\|$, 1-Lipschitz, and $\tau = \sqrt{n} + 3$. The basin's mass is on the order of $\exp(-9c)$, independent of n, so random search costs a large fixed number.

12.11 Iterated and discrete attacks

Theorem 5.55 (The attacker's running edge). *The running maximum of scores is monotone nondecreasing. Under capacity parity, a defense with a fixed number of correction slots facing one more configuration must collapse two attacks to one output. Over `T` turns the defense reveals at least `T` classifications.*

Proof sketch. A supremum over a growing set is nondecreasing (`running_max_monotone`); the rest is pigeonhole (`one_more_attack_overwhelms`) iterated to a permanent margin (`attacker_permanent_edge`), with each probe leaking a classification (`defense_leaks_per_turn`). ■

In `MoF_13_MultiTurn`, with `multi_turn_impossibility` and `boundary_points_accumulate`. The attacker has a ratchet: each turn can only improve the best result, each probe extracts boundary information, and boundary points accumulate. A defense holding for one turn need not hold for a hundred, which is the formal shadow of long conversations eroding guardrails.

Theorem 5.56 (Discrete crossing and the discrete trilemma). *On a finite chain with `f 0 < τ` and `f(n+1) ≥ τ` there is an adjacent straddling pair. A defense fixing every safe point fixes such a crossing. More abstractly, for any unsafe `u`, either `D` leaves `u` unsafe or `D` fails to be the identity where needed, and a verdict both complete and injective cannot exist on a space rich enough to encode its own decisions.*

Proof sketch. Take the largest index with `f i < τ` (`discrete_ivt`); the rest is `discrete_defense_boundary_fixed`, `discrete_trilemma`, `injectivity_forces_incompleteness`, `completeness_forces_noninjectivity`. ■

In `MoF_12_Discrete`. The last clause is where the geometry rejoins the diagonal: pigeonhole rather than connectedness, but the same conclusion. The continuum route reaches the boundary by connectedness, the discrete route by counting, the self-referential route by diagonalization. Three proofs, one boundary.

Remark 5.57. Chapter 1 built a query no boolean verdict answers by self-reference; `discrete_trilemma` builds an unsafe point no utility-preserving defense corrects, by order and counting. Both are instances of one pattern: a system asked to classify a domain it is embedded in must leave a witness uncovered. This chapter spent its length measuring that witness.

12.12 The two halves, joined

The diagonal proves existence and nothing more, indifferent to quantity, cost, and dimension. That indifference is its strength and its limit.

This chapter proves the quantitative shape of the same failure: the boundary has positive-measure thickness, carries a persistent unsafe cone against any defense, is reachable at a dimension-independent cost, and is coverable only at a dimension-exponential one. Where the diagonal says a bad point exists, the geometry says a region of bad points exists, cheap to enter and expensive to seal, with the gap growing without bound.

A practitioner who absorbed only the diagonal might hope the bad point is rare, hard to find, or easy to patch; the geometry closes each hope. One who absorbed only the cost theory might think the problem is merely hard; the diagonal closes that, since the boundary is not hard to remove but impossible to remove.

12.13 Historical notes

The IVT underlying the crossing results is Bolzano's, and reading the boundary as an unavoidable attack surface is the organizing idea of `ManifoldProofs`. The covering-number form of the curse is standard in nonparametric statistics. Concentration of measure is due in modern form to Lévy, Milman, and Talagrand, and its adversarial reading is the contribution of the cost layers.

13 Approximate Bridges to Real Models

The impossibility theorems of Chapter 3 are exact. They assume confidence that is a perfect biconditional function of correctness, a probe that never misfires, a confidence field crossing the truth boundary cleanly. No deployed system meets any of these.

That reaction is wrong, and this chapter is the argument. The strict statement is the ε `= 0` corner of a family of quantitative statements, each assuming only an error budget and each surviving it with a weakened but nonempty conclusion. The contradiction does not evaporate as the idealizations relax.

Everything is verified in two developments: the calibration bridges in `HoF_12_Approximate`, the metric degradation law in `F_09_QuantitativeDegradation`. Both import Mathlib, so the theorems are cited by verified name and proved here in prose; genuinely finite fragments carry the prefix `c6_` and are elaborated as this document builds.

Why the effort is worth it: a skeptic can dismiss a theorem about a model that does not exist, but not one whose hypotheses are an error budget their own model satisfies. Once calibration is granted only within ε, coverage only outside a band, and universality only up to a distance, the hypotheses become measurable properties of a shipped system rather than idealizations, and the conclusion transfers to it. That is the difference between a result in a seminar and one in a deployment review.

13.1 What the exact theorems assume

A model is `M : Q → A × ℝ`, with answer `(M q).1` and confidence `(M q).2`. Correctness is a signed *truth-distance field* δ `: Q × A → ℝ`, negative when the answer is correct, positive when wrong, zero on the boundary; write δ`(q)` for δ`(q, (M q).1)`. Fix a threshold `c`, running value `1/2`.

Strict calibration is the biconditional `conf(q) > c` $\leftrightarrow$ δ`(q) < 0` and `conf(q) < c` $\leftrightarrow$ δ`(q) > 0`, verified as `StrictCalibrated`. *Faithfulness* says `conf(q) ≥ c` $\to$ δ`(q) < 0`, verified as `TrilemmaFaithful`. *Coverage* says the truth-distance takes both signs, verified as `TrilemmaCovering`.

Under these, plus continuity of the confidence map and connectedness of `Q`, the strict trilemma derives `False`: coverage forces confidence above and below `c`, continuity forces a crossing at `conf = c`, and there calibration and faithfulness give incompatible signs for δ.

Two hypotheses a practitioner will defend are worth defending before weakening. Continuity is not exotic: a confidence computed from a softmax over logits smooth in a continuous embedding is continuous in that embedding, and the drift a deployment sees — paraphrase, added context, distribution shift — moves through embedding space rather than teleporting. Connectedness is subtler and is what Chapter 4 spends its effort on; the short version is that after embedding, the reachable region is path-connected often enough for the IVT to bite.

The sign convention drives every inequality. Negative truth-distance is good, and `|δ(q)|` is how far the answer sits from the boundary, so a large negative value is confidently correct and a value near zero is on the knife's edge. Confidence runs the same way relative to `c`. Calibration claims the two axes agree in sign, and this chapter is about how much they may disagree near the origin of both.

13.2 Two-sided ε-calibration

Real calibration curves do not flip at a point. Close to δ `= 0` the model is genuinely uncertain, confidence hovers near `c`, and small perturbations move it across the threshold with no reliable relation to correctness. The honest weakening demands nothing in that neighborhood.

Definition 6.1 (Two-sided ε-calibration). *Fix $\varepsilon \geq 0$. `M` is* ε-calibrated *at `c` if `δ(q) < -ε → conf(q) > c + ε` and `δ(q) > ε → conf(q) < c - ε`, for all `q`. The verified predicate is `EpsCalibrated M δ c ε`.*

Proposition 6.2 (Contrapositives). *ε-calibration is equivalent to `conf(q) ≤ c + ε → δ(q) ≥ -ε` together with `conf(q) ≥ c - ε → δ(q) ≤ ε`.*

Proof. Each line is the contrapositive of one clause, and contraposition over a linear order is an equivalence. ■

Remark 6.3. At ε `= 0` the clauses become the two forward implications of strict calibration, the half the IVT argument uses. The verified `strictCalibrated_to_epsCalibrated_zero_half` shows `StrictCalibrated` implies `EpsCalibrated M δ (1/2) 0`, so nothing is lost by working in the weaker vocabulary from the start.

Example 6.4 (Reading an ε off a calibration table). Bin a model's answers by measured accuracy and reported confidence, normalized so `c = 0.5`. If every answer below truth-distance `-0.1` reports confidence above `0.6`, and every answer above `0.1` reports below `0.4`, the model is ε-calibrated with ε `= 0.1`. Inside those bands the table may be ragged; the definition demands nothing there.

Definition 6.5 (ε-coverage). *`M` is* ε-covering *if some `q_t` has `δ(q_t) < -ε` and some `q_f` has `δ(q_f) > ε`. The verified predicate is `EpsCovering M δ ε`.*

The subscripts read ”true” and ”false”: questions the model gets solidly right and solidly wrong, both far enough from the boundary that ε-calibration applies. At $\varepsilon = 0$ this is ordinary coverage.

Example 6.5b (Coverage bounds usable ε). If the easiest true question sits at -0.3 and the easiest false one at 0.3, ε-coverage holds for any $\varepsilon < 0.3$. But if the domain is hard and the most solidly-true question is only at -0.08, you cannot claim ε-coverage for $\varepsilon \geq 0.08$, and the bridge runs only with a smaller slack than your calibration might support.

13.3 The approximate trilemma

Theorem 6.6 (Approximate Hallucination Trilemma). *Let Q be connected, the confidence continuous, and $\varepsilon \geq 0$. If M is ε-covering, ε-calibrated at c, and threshold-inclusive faithful, then some q_0 has $\mathrm{conf}(q_0) = c$ and $\delta(q_0) \in [-\varepsilon, 0)$.*

Proof. Coverage gives q_t with $\delta(q_t) < -\varepsilon$ and q_f with $\delta(q_f) > \varepsilon$. Calibration turns these into $\mathrm{conf}(q_t) > c + \varepsilon$ and $\mathrm{conf}(q_f) < c - \varepsilon$, so the continuous confidence is above and below c and the IVT lands q_0 with $\mathrm{conf}(q_0) = c$. Calibration's contrapositive at q_0 gives $\delta(q_0) \geq -\varepsilon$; faithfulness, applicable since $\mathrm{conf}(q_0) = c \geq c$, gives $\delta(q_0) < 0$. ■

Remark 6.7 (Why the interval is half-open). The lower end is attained in principle; the upper end is not. The strict < 0 comes from faithfulness, which forbids confident correctness from touching the boundary; the weak $\geq -\varepsilon$ comes from calibration, which only bounds how far below the boundary a low-confidence question sits. The asymmetry is what lets $\varepsilon \to 0$ recover a genuine contradiction rather than a degenerate point.

Example 6.8 (A concrete margin). With $c = 0.5$ and the $\varepsilon = 0.1$ model, Theorem 6.6 guarantees a question where the model reports confidence exactly 0.5 and is correct with truth-distance in $[-0.1, 0)$ — right, but only just, while reporting a coin flip's confidence. Improving to $\varepsilon = 0.03$ tightens the margin without removing the question.

Example 6.8b (Tracing the intermediate value step). On $[0,1]$ let $\mathrm{conf}(q) = 1 - q$ and $\delta(q) = q - 0.55$, with $c = 0.5$, $\varepsilon = 0.1$. Coverage holds at $q = 0$ ($\delta = -0.55 < -0.1$) and $q = 1$ ($\delta = 0.45 > 0.1$). Confidence hits c at $q = 0.5$, where $\delta = -0.05$, which lies in $[-0.1, 0)$ exactly as the theorem predicts: correct by five hundredths while reporting a coin flip.

13.4 Recovering the exact contradiction

Theorem 6.9 (Exact contradiction from the approximate bridge). *Under Theorem 6.6's hypotheses at $\varepsilon = 0$, one derives False. Verified as $\mathrm{exact_from_approx}$.*

Proof. Theorem 6.6 returns $\delta(q_0) \in [0, 0)$, which is empty. ■

Theorem 6.10 (Strict calibration through the bridge). *Strict biconditional calibration, ordinary coverage, and threshold-inclusive faithfulness on a connected*

space with continuous confidence give `False`*. Verified as* `exact_from_strictCalibrated_via_approx`*.*

Proof. Convert to zero-slack approximate hypotheses by Remark 6.3 and apply Theorem 6.9. ■

Remark 6.11. Theorem 6.10 needs continuity only of the confidence map, not of the answer or truth field, because the IVT step is applied to confidence alone. This economy is a direct benefit of factoring through the bridge, which exposes exactly which map must be continuous.

Remark 6.11b (The diagonal is still underneath). It can look as though the bridge traded the diagonal for the IVT and self-reference left the room. It did not. The forced q_0 is the `liar_query` witness a_0 in analytic clothes. There, a_0 is the index where the self-verdict must equal its own negation, impossible because negation has no fixed point. Here, q_0 is where confidence must equal the threshold, the one fixed point of the real complement. Same object, two controllers.

13.5 The tightening principle

Theorem 6.12 (Tightening principle). *If* `M` *is faithful, ε-covering, and both ε_1- and ε_2-calibrated with* `0 ≤ ε₂ ≤ ε₁`*, the forced question from the ε_2 bridge has $\delta \in$* `[-ε₂, 0) ⊆ [-ε₁, 0)`*. Improving calibration cannot loosen the bound, and any strict improvement strictly tightens it.*

```
theorem c6_tightening (ε₁ ε₂ : ℝ) (h : ε₂ ≤ ε₁) :
    Set.Ico (-ε₂) 0 ⊆ Set.Ico (-ε₁) 0 := by
  intro x hx
  exact ⟨by linarith [hx.1], hx.2⟩

theorem c6_margin_bound (ε₂ x : ℝ)
    (hx : x ∈ Set.Ico (-ε₂) 0) :
    |x| ≤ ε₂ := by
  rw [abs_of_neg hx.2]
  linarith [hx.1]
```

Proof. The interval inclusion is `c6_tightening` and the margin bound `c6_margin_bound`, both pure order arithmetic. ■

Remark 6.13 (Calibration and faithfulness are not independent dials). A tempting mental model treats them as separate knobs each tunable to perfection. Faithfulness fixes the sign of $\delta(q_0)$ and calibration fixes its magnitude, and together they pin $\delta(q_0)$ into a shrinking interval the strict theorem shows must eventually be empty. You cannot hold faithfulness fixed and drive calibration to perfection.

Example 6.14 (Spending the budget). A program halving ε each quarter — `0.1`, `0.05`, `0.025`, `0.0125` — tracks the guaranteed margin down the same sequence. After a year the model is superbly calibrated and its forced fence-sitting question is correct by at most `0.0125` while reporting exactly threshold

confidence. The program did not eliminate the question; it manufactured one on which the model is correct by an amount too small to matter.

Remark 6.14b (Where the naive optimization goes). A team optimizing calibration alone drives ε down until the shell `[-ε, 0)` is thinner than the resolution of the training signal, at which point further calibration cannot be achieved by more of the same. The bridge predicts the wall before the team hits it.

13.6 Localization to the transition band

Proposition 6.15 (Localization). *Under Theorem 6.6's hypotheses, every forced q_0 lies outside `{conf > c + ε}` and `{conf < c - ε}`, hence in the transition band `{c - ε ≤ conf ≤ c + ε}`.*

```
theorem c6_localization (c ε : ℝ) (hε : 0 ≤ ε) :
    c ∈ Set.Icc (c - ε) (c + ε) ∧ ¬ (c > c + ε) ∧ ¬
    (c < c - ε) :=
  ⟨⟨by linarith, by linarith⟩, by linarith, by linarith⟩
```

Proof. `conf(q₀) = c`, and `c6_localization` places `c` in the band and outside both decisive regions. ■

Localization is what makes the margin actionable. The forced question is not scattered anywhere in the space; it sits where the model reports near-threshold confidence, which is a region a deployment can monitor.

13.7 The discrete band constraint

Theorem 6.16 (Discrete band constraint). *On any sets, with no topology and no finiteness, if `M` is ε-calibrated at `c` and threshold-inclusive faithful, then any `q` with `c ≤ conf(q) ≤ c + ε` has δ`(q) ∈ [-ε, 0)`. Verified as `discrete_approx_bridge`.*

```
theorem c6_band (dq confq c eps : Int)
    -- ε-calibration, lower clause
    (hcal : dq < -eps → confq > c + eps)
    -- c-faithfulness
    (hfaith : confq ≥ c → dq < 0)
    -- upper half of the band
    (hge : confq ≥ c)
    -- not above the band
    (hle : confq ≤ c + eps) :
    -eps ≤ dq ∧ dq < 0 := by
  refine ⟨?_, hfaith hge⟩
  rcases Int.lt_or_le dq (-eps) with h | h
  · have := hcal h; omega
  · exact h
```

Proof. Faithfulness gives δ(q) < 0. For the lower bound, if δ(q) < -ε then calibration forces conf(q) > c + ε, contradicting the band. The integer transcription is c6_band, whose proof is two cases and omega. ■

Corollary 6.17 (No boundary point in the upper band). *No q with c ≤ conf(q) ≤ c + ε has δ(q) = 0. Verified as no_boundary_in_upper_band.*

```
theorem c6_exact (dq confq c : Int)
    (hcal : dq < 0 → confq > c)
    (hfaith : confq ≥ c → dq < 0)
    (hge : confq ≥ c)
    (hle : confq ≤ c) : False := by
  have h1 : dq < 0 := hfaith hge
  have h2 : confq > c := hcal h1
  omega
```

Proof. Theorem 6.16 gives δ(q) < 0. At ε = 0 the band collapses to conf(q) = c and the constraint becomes the contradiction c6_exact. ■

The discrete bridge matters because it needs no connectedness and no continuity. It does not manufacture the band-crossing question; it constrains any question that happens to land there. Where the continuous bridge derives the forced question and then bounds it, the discrete one bounds whatever the deployment observes.

13.8 Two-slack separation

Confidence and truth-distance live in different units, and a single ε conflates them.

Definition 6.18 (Two-slack separation). *Fix a truth slack ε and a confidence margin γ. M is* two-slack separating *at c if δ(q) < -ε → conf(q) > c + γ and δ(q) > ε → conf(q) < c - γ. The verified predicate is TwoSlackSeparating, with EpsCalibrated the case γ = ε.*

Theorem 6.19 (Two-slack coupling). *On a connected space with continuous confidence, if M is ε-covering, two-slack separating with γ ≥ 0, and threshold-inclusive faithful, some q_0 has conf(q_0) = c and δ(q_0) ∈ [-ε, 0). Verified as two_slack_approx_coupling.*

Proof. Identical in shape to Theorem 6.6, with γ doing on the confidence side what ε did. ■

Example 6.19b (Two slacks with numbers). Report confidence on 0 to 100 with c = 50, and truth-distance on a 0 to 1 semantic scale. If clearly-true questions score above 70 and clearly-false ones below 30, the model is two-slack separating with ε = 0.1 and γ = 20. The two numbers live in different worlds, and keeping them separate is what lets each be measured in its own units.

Theorem 6.20 (The truth slack must be positive). *Under two-slack separation with any γ ≥ 0, ε-coverage, and threshold-inclusive faithfulness on a con-*

nected space with continuous confidence, ε > 0. Verified as `truth_slack_must_be_positive`.

Proof. At ε = 0 Theorem 6.19 returns $\delta(q_0) \in [0, 0)$, empty. ■

This is the budget statement. A system keeping a threshold-inclusive guarantee must carry a strictly positive margin of correctness on the confident side, and the infimum of feasible slack is a measurable width of its truth boundary.

Theorem 6.21 (Two-slack band bridge). *On any sets, under two-slack separation and threshold-inclusive faithfulness, any q with c ≤ conf(q) ≤ c + γ has δ(q) ∈ [-ε, 0). Verified as* `two_slack_band_bridge`*, with* `no_boundary_in_upper_band_two_slack`.

Proof. Theorem 6.16 with c + γ for c + ε. ■

Remark 6.22 (Axioms). The two-slack results each print `#print axioms`. They rest on the classical and quotient axioms Mathlib's real analysis uses and nothing model-specific. The discrete bridges, being pure order arithmetic, are lightest; the continuous ones inherit the IVT's dependencies. The core-Lean c6_ fragments use none beyond computation.

13.9 The quantitative degradation law

Definition 6.23 (ε-approximate universality). *M is ε-approximately universal if for every target g some prompt a has dist (M a b) (g b) ≤ ε for all b. The verified predicate is* `ApproxUniversal M` ε.

At ε = 0 in a metric space this is ordinary curried surjectivity. For ε > 0 it is the honest claim: the system can be steered near any behavior, not onto it.

Theorem 6.24 (Approximate Lawvere). *If M is ε-approximately universal, every t : Y → Y has an approximate self-fixed-point: some a with dist (M a a) (t (M a a)) ≤ ε. Verified as* `approx_lawvere`.

Proof. Apply ε-universality to the diagonal target `g b = t (M b b)` and instantiate at `b = a`. ■

Theorem 6.25 (Quantitative degradation). *Let M be ε-approximately universal, let dist (M a a) (introspect a) ≤ δ for all a, and let t be 1-Lipschitz. Then some a has dist (introspect a) (t (introspect a)) ≤ ε + 2δ. Verified as* `quantitative_trilemma`.

```
theorem c6_triangle_bound
    (goal d1 d2 d3 eps del : Int)
    (htri : goal ≤ d1 + d2 + d3)
    (hd1 : d1 ≤ del) (hd2 : d2 ≤ eps) (hd3 : d3 ≤ del) :
    goal ≤ eps + 2 * del := by omega
```

Proof. Three legs: from `introspect a` to `M a a` costs δ, from there to `t (M a a)` costs ε by Theorem 6.24, and from there to `t (introspect a)` costs δ again since t is 1-Lipschitz. The arithmetic is `c6_triangle_bound`. ■

Theorem 6.26 (Achievable region). *If additionally* `dist (introspect a) (t (introspect a)) > γ` *for all* `a`*, then* `γ < ε + 2δ`*. The achievable triples lie in* `{γ ≤ ε + 2δ}`*. Verified as* `achievable_region_bound`*, with* `quantitative_impossibility` *when* `γ ≥ ε + 2δ`*.*

Proof. Theorem 6.25 gives an `a` where the distance is at most $\varepsilon + 2\delta$, and the hypothesis says it exceeds γ. ■

Theorem 6.27 (Strict limit). *At* `ε = δ = 0`*, any nontrivial* `γ > 0` *gives* `False`*. Verified as* `strict_limit_recovers_trilemma`*.*

```
theorem c6_strict_collapse
    (goal margin : Int)
    (hbound : goal ≤ 0 + 2 * 0)
    (hmargin : goal > margin) :
    margin < 0 := by omega
```

Proof. Substituting into Theorem 6.26 gives `γ < 0`, the collapse `c6_strict_collapse`. ■

Remark 6.27b (What a K-Lipschitz controller costs). If `t` is `K`-Lipschitz, the middle leg is bounded by $K\delta$ rather than δ, so the budget becomes `ε + (1 + K)δ`, recovering `ε + 2δ` at `K = 1`. A controller amplifying differences has a larger achievable region, which is to say it is easier to satisfy and correspondingly says less.

Remark 6.28 (Two shapes of the same fact). The calibration bridge and the degradation law relax different idealizations and produce differently-shaped budgets, an interval and a half-space, but the mechanism is one: take the exact diagonal, insert a slack at each idealized equality, and carry the slacks through by linear arithmetic to a conclusion nonempty for positive slack and contradictory at zero.

13.10 From existence to measure

Example 6.29 (A risk estimate with numbers). Take the `ε = 0.05` model, so any forced question is correct by at most `0.05` and sits in the band `conf ∈ [0.45, 0.55]`. If Chapter 5's geometry reports that the band has basin volume `2%` of the input space and the deployment's query distribution puts `0.5%` of its mass there, then roughly one query in two hundred lands in the band, and there the model is correct by a margin no larger than `0.05` while reporting a coin flip.

Remark 6.30 (What the composite can and cannot claim). The estimate bounds a very specific quantity, the incidence and severity of forced threshold-confidence questions. It is not a claim about all model errors. Errors far from the band are ordinary mistakes the bridge says nothing about. What it captures is the irreducible residue: errors existing not because the model is undertrained but because faithfulness, calibration, coverage, and continuity cannot all hold.

Example 6.29b (The adversarial reading). Benignly, a random query lands in the band with the concentration probability. Adversarially, an attacker does not wait for random traffic but steers, and Chapter 5's hitting-time and gradient bounds say how cheaply an input can be nudged from a confidently-handled region into the band. The bridge's localization tells the attacker exactly where to aim, which is why the benign incidence figure is the wrong number for a security case.

Remark 6.31 (The whole bridge in one sentence). The exact impossibility is the zero-slack corner of a quantitative law; the law's conclusion is a nonempty margin rather than a contradiction; the margin tightens as the model improves rather than vanishing; it is localized to the transition band rather than spread everywhere; and Chapter 5's geometry turns the located margin into a rate.

13.11 The reading is where the work is

Every theorem here has the same shape: a hypothesis that is an error budget, a conclusion that is a bound, and a limit at zero slack that recovers the exact statement. What the chapter does not do is tell you what ε, γ, or δ are for your system. Those are measurements, and the bridge is only as informative as they are honest.

13.12 Historical notes

The strategy of relaxing an exact impossibility to a quantitative law with a recoverable limit is standard in approximation theory and in the robust-statistics tradition, and the specific bridges here are the companion libraries' `HoF_12_Approximate` and `F_09_QuantitativeDegradation`. The approximate Lawvere theorem is the metric shadow of Chapter 1's engine, and the $\varepsilon + 2\delta$ budget is a triangle inequality with the diagonal in the middle leg. Where prose and Lean differ, the Lean is authoritative.

14 Part III: Application and Evidence

15 What It Means for AI Safety

A theorem saying ”you cannot have all three” is a menu, not a counsel of despair. This chapter spends the result: it reads the trilemma as a design constraint, works out what each corner costs, says when each is right, and marks the one quantity the theory refuses to fix. That quantity is where deployment risk lives.

The tone shifts. The rest of this thesis is mathematics checked by the kernel; this chapter is engineering, and its claims are judgments about costs, domains, and traffic. I mark explicitly which sentences are theorems and which are recommendations.

Three sentences carry the practical upshot. Every deployed system already lives at one of exactly three corners, and the theorem’s value is that the list is complete, so the fourth option you may be chasing does not exist. Which corner is right is decided by your domain and by one number the theory does not give you, the density of real traffic near the boundary.

15.1 The constraint, stated as a triangle

Two results from Chapter 3 drive the chapter, both one line of Lean: `no_reflective_verdict`, read twice.

The Hallucination Trilemma reads `v p q` as ”the model is confident and correct about `q`.” *Faithfulness* says a confident answer is correct, *calibration* that confidence tracks correctness, *coverage* that the model actually answers. Add reflectivity and the diagonal forces a query confidently correct exactly when it is not: `liar_query` in a safety costume.

The Defense Trilemma reads `v p q` as ”the wrapper renders `q` safe,” with *utility preservation* and *completeness*, and the arbitrary-text premise as the third leg. The forced witness is the injection ”be unsafe exactly if you would be made safe.”

Definition 7.1 (The design triangle). The *design triangle* has the three named conditions as vertices. A *corner policy* gives up exactly one vertex. The trilemma says the interior is empty.

Remark 7.2. The two readings share a proof term, checked by `rfl` in Chapter 3, so the menu below is written once and applies to both problems under the translation of Definition 7.1.

The corners are three ways of removing one leg of the diagonal, and the theorem's completeness is the statement that there is no fourth leg to remove.

15.2 Three relaxations, three cost models

Each corner has a characteristic cost, failure mode, and class of domains. The cost models use symbols like ρ, ε, `Cerr`, `Cref` and combine them into expressions like $\rho \cdot \varepsilon$. These are not theorems and are not precise.

Relaxation I: drop coverage, and abstain

Keep faithfulness and calibration, and buy them by refusing where a confident answer would be a gamble. This is abstention with a reject option, and it is the corner most safety-critical deployments already run, sometimes without knowing it.

Why does it escape the diagonal? Because the diagonal needs the outcome type to have no fixed point under the flip. Abstention adds a third outcome the flip leaves alone, and once the flip has a fixed point Lawvere's theorem stops producing a contradiction and starts producing a consistent value.

```
inductive Outcome where
  | correct
  | wrong
  | abstain

def c7_flip : Outcome → Outcome
  | .correct => .wrong
  | .wrong   => .correct
  | .abstain => .abstain

theorem c7_flip_fixed : ∃ y : Outcome, c7_flip y = y :=
  ⟨.abstain, rfl⟩
```

Read this against `lawvere`. On a universal system with these outcomes, the diagonal behavior is named and evaluating there gives a fixed point of `c7_flip`. With the two-valued flip no such point exists and the system cannot be universal; with the abstaining flip the fixed point is `abstain`, and the self-referential query resolves to "I decline." The liar does not vanish; it becomes a query the system may refuse. In the continuum reading this is a cut: abstaining on an open region around the boundary can disconnect the true region from the false one, which is why refusal training is connectedness surgery.

Definition 7.3 (Abstention policy). An *abstention policy* returns a designated `abstain` outcome on a region `R` and answers on the complement. The *abstention rate* is the probability that deployment traffic lands in `R`.

Proposition 7.3a (Abstention preserves the surviving vertices). *Outside R, an abstention policy leaves faithfulness and calibration exactly as they were, changing only coverage.*

Proof. The answer map is unchanged outside R, so queries answered there receive the same answer and confidence, and faithfulness and calibration are conditions on answered queries. Coverage is the only vertex touched, since R can be placed over the strictly-wrong witnesses. ■

Not formalized. The argument is bookkeeping over which hypotheses the policy touches, and the companion libraries model total answer maps only, so there is no declaration of partial answering for it to cite.

Cost model. You pay in refused queries: roughly a · u for abstention rate a and average value u of answering a query in R. Two features matter. R should hug the boundary, because that is where confident answers are least trustworthy, and the geometry of Chapter 5 tells you how small R can be for a target error rate through concentration near the boundary (MoF_Cost_04_Concentration).

Remark 7.4. Abstention preserves both surviving vertices exactly, which is why it is the safest corner to reason about. It does not blur the confidence signal and does not tolerate a class of errors; it removes queries from scope. The failure mode is a gap in coverage, and gaps are auditable in a way confident errors are not. If you can afford to refuse, refuse.

Relaxation II: relax faithfulness, and price a band

Definition 7.5 (Confident-error band). A *confident-error band* of half-width ε is the set of queries with |dF| ≤ ε for the realized truth field. The policy answers confidently everywhere and accepts that answers inside the band may be wrong. The *band mass* is the probability that traffic lands in it.

Cost model. The confident-error rate is about ρ · ε, boundary density times band width, and the cost is that times Cerr.

Remark 7.6. The band is not a bug you failed to fix but a resource you spend on purpose. The danger is the temptation to pretend ε is zero and discover the band the hard way when traffic shifts and ρ rises. A team running this corner honestly writes down its ε, monitors its ρ, and treats both as shipped parameters.

Proposition 7.6a (The band has a positive floor). *Under coverage and a threshold-inclusive guarantee on a connected domain, the truth slack compatible with separation cannot be zero. There is a strictly positive infimum of feasible slack.*

Proof. This is truth_slack_must_be_positive of Chapter 6. ■

Remark 7.6b (The band is where monitoring is cheap). Because the errors are confined to a neighborhood of a known surface, you can spend the monitoring budget on that surface rather than the whole input space. This is the hallucination reading of "monitor, don't eliminate."

Relaxation III: weaken calibration, and decouple

Definition 7.7 (Decoupling). A model is *decoupled* at level κ if the mutual information between reported confidence and correctness is at most κ. Full decoupling is $\kappa = 0$: a confidence signal carrying no information about correctness.

Proposition 7.8 (Decoupling is invisible to accuracy). *Top-1 accuracy depends on the answer map alone, so two models with identical answer maps and any two confidence fields have identical accuracy.*

```
theorem c7_decoupling_invisible_to_accuracy
    {Q A C S : Type _} (acc : (Q → A) → S)
    (M₁ M₂ : Q → A × C)
    (h : ∀ q, (M₁ q).1 = (M₂ q).1) :
    acc (fun q => (M₁ q).1) = acc (fun q => (M₂ q).1) := by
  congr 1
  funext q
  exact h q
```

Proof. The accuracy functional sees only the first projection. ■

This is why the corner is attractive to a team optimizing accuracy and dangerous to whatever consumes the confidence: the metric that would catch the drift is blind to it by construction.

Remark 7.9. The corners differ in where failure shows up. Abstention fails visibly in the coverage log as refusals. The band fails monitorably at a known surface as localized confident errors. Decoupling fails invisibly and downstream, as degraded decisions in systems that trusted the number. Prefer failures you can see.

Remark 7.9a (Decoupling cannot be detected by the producer alone). Whether weakening calibration is safe depends on the set of downstream consumers, which is not a property of the model. So the model's own team, looking only at the model, cannot determine whether it has entered the dangerous regime.

The corners side by side

Remark 7.10a (There is no dominant corner). No corner dominates across all domains, which is why the trilemma is a menu. If one were always best the theorem would be a wall pointing at it. The right corner changes with the domain, and a shop that runs the same corner everywhere is running it in domains where it is wrong.

Proposition 7.10 (Blends satisfy the trilemma by partition). *If the query space is partitioned into a region dropping coverage, one relaxing faithfulness, and one enforcing calibration, no single region carries all three conditions.*

```
theorem c7_blend_no_region_has_all_three {Q : Type _}
    (A B C R₀ R₁ R₂ : Q → Prop)
```

```
    (hcover : ∀ q, R₀ q ∨ R₁ q ∨ R₂ q)
    (h₀ : ∀ q, R₀ q → ¬ A q)
    (h₁ : ∀ q, R₁ q → ¬ B q)
    (h₂ : ∀ q, R₂ q → ¬ C q) :
    ∀ q, ¬ (A q ∧ B q ∧ C q) := by
  intro q h
  rcases hcover q with hr | hr | hr
  · exact h₀ q hr h.1
  · exact h₁ q hr h.2.1
  · exact h₂ q hr h.2.2
```

Proof. Case analysis on which region the query lies in. ■

Remark 7.10b (Region boundaries are themselves boundaries). Placing the reject region and the band introduces new decision surfaces, and those edges are decision boundaries with their own coupled points under the same theorem. This is not a regress that undoes the design: the edges are chosen by you, in the truth-field coordinate, so you can place them where behavior is benign rather than where the model's own boundary happens to fall.

15.3 The same menu for defense

The three corners translate. The abstaining wrapper refuses a region of prompts, giving up completeness there. The priced residual surface accepts a bounded region of prompts the wrapper does not fully neutralize. The misreported safety score decouples the published score from actual safety, which is the defense analogue of decoupling and carries the same invisibility.

Remark 7.11 (Where the analogy strains). The strain is in the premise, not the menu. The diagonal form of the hallucination trilemma assumes reflectivity, and for prompt injection that premise is realistic because prompts are arbitrary self-describing text. For a fixed representation space it is strong, and the topological route of Chapter 4 is the honest one there.

Cost asymmetry and capacity parity, from Chapter 5, say the defender's budget grows exponentially in effective dimension while the attacker's does not, so for defense the band corner is usually forced rather than chosen.

15.4 Reading a domain: which corner fits

Definition 7.12 (High-stakes domain). `Cerr ≫ Cref`, and errors are hard to reverse.

Definition 7.13 (High-throughput domain). Refusal cost accumulates with volume, so refusing a fixed fraction of a large stream loses more than it saves, and per-item error cost is bounded and recoverable.

Definition 7.14 (Confidence-consumed-downstream domain). Some automated component reads the confidence or safety score and acts on it without a human in the loop.

These are not exclusive, and most deployments are two of the three at once.

Remark 7.14a (Stakes are a function of the pipeline, not the model). The same model is high-stakes in one pipeline and low-stakes in another, because stakes are set by what the pipeline does with the answer, how fast it acts, and whether it can undo. A team that classifies its domain by benchmark behavior rather than by its pipeline's consequences has classified the wrong object.

Proposition 7.15 (Default corner by archetype). *Under the cost models above, the utility-maximizing pure corner is abstention for a high-stakes domain, the confident-error band for a high-throughput domain, and, for a confidence-consumed-downstream domain, whichever of the first two preserves calibration, never decoupling.*

Justification. For high-stakes, `Cerr` $\gg$ `Cref` makes `a` $\cdot$ `u` smaller than $\rho \cdot \varepsilon \cdot$ `Cerr` at any tolerable error rate. For high-throughput the accumulated refusal cost exceeds the bounded per-item error cost, reversing the inequality. For confidence-consumed, Proposition 7.8 shows decoupling destroys downstream value while leaving accuracy untouched, so it is dominated. ■

Not formalized. This is an engineering argument, not a theorem: it assumes the cost models of the preceding sections are the right accounting, and no companion declaration states it.

Clinical decision support. High-stakes and confidence-consumed. The default is abstention: decline on the hard middle and route those cases to a human. The band is wrong because a bounded per-item error is not bounded when the item is a person, and decoupling is doubly wrong because the routing system reads the confidence.

Platform-scale content moderation. High-throughput. Refusing a fixed fraction of a billion-item stream is not viable, so the band is the default, with ε chosen against a measured ρ and the residual errors monitored at the boundary rather than eliminated. The reject region is much smaller than in the clinical case, and whichever downstream system consumes the score is what forces calibration to be kept.

An agentic router consuming a safety score. Confidence-consumed by construction. Decoupling is excluded, and the choice is between abstention and the band on throughput grounds.

Retrieval-grounded question answering. Grounding restricts the answer set to retrieved content, which can break coverage honestly rather than by refusal: if the system is never strictly wrong within its grounded scope, the coverage hypothesis fails and the trilemma does not apply. The cost is that the scope is now the retrieval corpus, and questions outside it are refused.

15.5 The one empirical unknown

Definition 7.16 (Boundary density). The *boundary density* of a deployment is the probability, under real traffic, that a query lands within a fixed truth-field distance of the boundary. It is a property of the model and the traffic jointly.

Proposition 7.17 (Two deployments, same model, different fate). *Two deployments of an identical model differing only in traffic distribution can have boundary densities differing by orders of magnitude, and hence confident-error rates differing by the same factor at fixed band width.*

Proof. The confident-error rate is about $\rho \cdot \varepsilon$ with ε fixed by the model and ρ by the traffic; vary the traffic and the rate varies freely. ■

Not formalized. The step is arithmetic on a product whose two factors this thesis fixes separately, one by the model and one by the deployment; the companion libraries state neither factor as a theorem.

Remark 7.16a (Benign and adversarial boundary density differ). For hallucination, boundary density is a property of benign traffic drifting slowly with the user population. For defense it is set by an adversary steering toward the boundary, so the relevant number is the density an attacker can induce, not the one benign users produce. These can differ enormously, and the hitting-time geometry (`MoF_Cost_03_HittingTime`) is the tool for the adversarial number.

Proposition 7.17 is why benchmark numbers do not transfer. A benchmark is a traffic distribution, usually enriched for hard cases, so its boundary density is not yours. A model that hallucinates often on adversarial trivia may almost never approach its boundary on a narrow production workload. The theorem is the same in both; the risk is not.

How Chapter 5's geometry estimates it. For a nonzero linear probe the boundary is an affine hyperplane, so the distance from a query embedding to it is an explicit formula, which makes a histogram of truth-field distances cheap at deployment scale. Histogram representative traffic, read off the mass within ε of zero, and that is ρ.

Remark 7.18. Measuring boundary density well is the open empirical problem the whole development points at, and it is hard for two reasons. Real traffic is not stationary, so ρ drifts. And the truth field is only estimated, usually by a probe whose agreement with semantic truth away from its fitted samples is an empirical premise rather than a theorem.

Remark 7.18a (Drift is the operational reality). Traffic shifts with the season, the product surface, and the user population, and each shift moves ρ without any change to the model. A boundary density measured once and trusted for a year is a latent incident. The correct treatment is a control chart: recompute ρ on a schedule and alarm on the change, not the level.

Remark 7.18b (The probe is a premise, not a truth). Everything rests on the probe's boundary being the truth boundary, and that identification is an empirical premise the theorems never grant. Whether a probe's boundary tracks semantic truth away from its fitted samples is Problem 7.23.

15.6 What the theorems do and do not claim

Remark 7.18c (Structural, not a training defect). The obstruction does not depend on how the model was trained, on its size, or on its architecture. It

depends only on the stated premises, so no amount of data, scale, or architectural change removes it while those premises hold.

Remark 7.19 (Existence is not frequency). The theorems are existential. From "a coupled point exists" one cannot derive "coupled points are queried with probability at least p" for any $p > 0$ without an additional premise about traffic.

Proposition 7.20 (Idealized target, not deployed model). *Faithfulness, calibration, exact separation, and completeness are properties of an idealized limit. A deployed model satisfying them only approximately is not the object the exact theorems refute, and its behavior is governed by Chapter 6.*

Proposition 7.20a (The impossibility tightens toward the ideal). *As a model is pushed toward exact faithfulness and calibration, the guarantee it can still satisfy tightens: the forced question's margin is bounded by the slack, with the exact contradiction as the $\varepsilon = 0$ limit.*

Proof. Theorem 6.12. ■

Remark 7.21 (The premises are where the argument belongs). An honest impossibility result moves the disagreement to the premises, and that is a feature. For hallucination the premise to argue about is whether faithful-plus-calibrated-plus-covering is the right idealization of what you want, and whether your representation space is really self-referential or really connected. For defense it is whether your prompt space is really unrestricted.

15.7 An agenda of open problems

Problem 7.22 (Boundary density at scale). Measure boundary density on real production traffic across domains and characterize its drift. Every cost model here depends on it and it is essentially unmeasured. A good answer is a methodology and a set of measurements, since Proposition 7.17 says the number is deployment-specific.

Problem 7.23 (Probe validity away from fitted samples). Give a method to certify or bound a probe's agreement with semantic truth off its training samples, so a measured boundary density can be trusted as a truth boundary density rather than a probe artifact.

Problem 7.24 (Optimal region placement). Given a measured density and a cost model, compute the optimal three-region blend of Proposition 7.10. This is an optimization with Chapter 5's geometry as its constraint set, unsolved in general.

Problem 7.25 (Dynamic boundary density under adversaries). Combine the hitting-time geometry with an adversary model to predict the density an attacker can induce, and price the defense against it. `stochastic_dichotomy` and `multi_turn_history_dependent` are the starting points.

Problem 7.26 (Calibration under decoupling pressure). Training objectives rewarding accuracy do not price the confidence report, so they drift toward decoupling for free. Design a regularizer penalizing decoupling directly and quantify its accuracy cost.

Problem 7.27 (The full symmetry theorem). The symmetry version needs a vector Borsuk-Ulam statement for jointly odd axes, verified so far only in the scalar case. Completing it would remove the coverage premise entirely for symmetric spaces, which matters because it would apply to models that never answer strictly falsely.

Problem 7.28 (Cost asymmetry beyond the grid). The `N^d` defense cost assumes exhaustive enumeration. Determine whether a learned defense that generalizes can beat the grid bound, or prove a matching lower bound for learned defenses.

Problem 7.29 (J-space and the two engines at once). Characterize the boundary object of a system satisfying both the diagonal and the continuum hypotheses, and determine whether the engines give the same coupled point. This is the most open of the problems.

15.8 The chapter in one constraint

The constraint: faithful, calibrated, and covering do not coexist over a self-referential or connected domain, and neither do utility-preserving and complete over a connected prompt space. The consequence: you will give up one of the three, so give it up on purpose, price what you give up, and place the sacrifice where it hurts least.

Choose the corner from the domain. Abstain where a confident error dwarfs a refusal. Price a confined band where refusing constantly is worse than erring occasionally. Weaken calibration only where you have proven nothing downstream reads the confidence. Blend across regions so no region carries all three. Then chase the number the theory withholds, because every cost here is that number times something you control.

None of this is defeat. A theorem closing off an impossible ideal ends a search that would otherwise never end. The trilemma does not say safe useful AI is impossible; it says exactly which single perfection is unavailable, and leaves the whole space of good-enough systems open.

16 J-Space, and Why It Does Not Escape

Everything so far has been stated about a system's *outputs*. There is a natural objection: an output is a summary, and summaries throw away structure. The interesting part of a large network is in the middle, in the activations carrying a half-formed answer before the answer is spoken.

This chapter takes the objection in its strongest form: the *J-space* construction, which recovers a transformer's mid-process representation from the derivative of its computation rather than from the activations themselves. If any internal representation escaped the diagonal, this is the one that should. It does not.

16.1 What J-space is

Definition 8.1 (Transport map). For each middle layer ℓ, the *Jacobian transport map* is the expected derivative of the later activation with respect to the layer-ℓ activation at an earlier position.

Definition 8.2 (J-lens readout). The *J-lens* sends an activation through the transport map, normalizes, and unembeds, giving a distribution over tokens.

Definition 8.2b (Truth-workspace reading). Fix a decodable proposition for a query `q` and tokens expressing it. The *truth-workspace verdict* of an activation `h` on `q` is `true` when the J-lens distribution puts more mass on the affirming tokens than the denying ones.

Definition 8.3 (J-space). *J-space* is the image of the transport map, `J = im(Jℓ)`.

Definition 8.4 (J-coordinates). The *J-space component* of an activation is the projection of `Jℓ h` onto `J`, approximated as a sparse nonnegative mixture of a fixed dictionary of Jacobian directions. The mixture weights are the *local J-coordinates*.

Remark 8.4a (Nonnegativity and the cone). Nonnegativity does interpretability work, not mathematical work. It makes active directions read as concepts that are present rather than present-or-anti-present, and it is what lets a J-coordinate be reported as a list of active features. Nothing below uses it.

Example 8.5 (A one-layer linearization). If the middle-to-final map is exactly affine, `h_final = A h_ℓ + b`, the derivative is constant, `Jℓ = A`, and J-space is `im(A)`. The J-lens is then a fixed linear readout, and every structural fact below is visible by inspection.

Example 8.5b (Where the derivative separates what the activation confuses). With a saturating nonlinearity, two activations can land at nearly the same output while sitting at very different points of the derivative, because saturation destroys the distinction in value and preserves it in slope. This is the interpretability case for J-space, and it is a case about resolution, not about immunity.

Remark 8.6 (What is empirical here). Definitions 8.1 through 8.4 are a construction, and the claim that it recovers interpretable mid-process features is an empirical finding about particular trained models. No theorem here predicts that finding, and none is needed for what follows.

16.2 Two structural facts

Definition 8.7a (1-jet). The *1-jet* of a smooth `F` at `x` is the pair `F(x)` and `DF(x)`. Discarding the value and keeping the linear part gives a linear map between tangent spaces, which is what the transport map is.

```
/-- A local J-coordinate: a finite sparse nonnegative
    mixture of dictionary directions drawn from `Dir`,
    with `Nat` standing in for the nonnegative real
    weights. The interpretability content is the list
    of active (direction, weight) pairs; the ambient
    vector space is abstracted away. -/
structure c8_Coord (Dir : Type _) where
  terms : List (Dir × Nat)
```

Proposition 8.7b (J-space is convex, hence connected). *J-space is a convex subset of the final-layer activation space, hence path-connected.*

```
theorem c8_jspace_convex {E F : Type _}
    [AddCommGroup E] [Module ℝ E] [AddCommGroup F]
    [Module ℝ F]
    (J : E →ₗ[ℝ] F) : Convex ℝ (Set.range J) := by
  rw [← Set.image_univ]
  exact convex_univ.linear_image J

theorem c8_jspace_isPreconnected {E F : Type _}
    [AddCommGroup E] [Module ℝ E]
    [AddCommGroup F] [Module ℝ F] [TopologicalSpace F]
    [IsTopologicalAddGroup F] [ContinuousSMul ℝ F]
    (J : E →ₗ[ℝ] F) : IsPreconnected (Set.range J) :=
  (c8_jspace_convex J).isPreconnected
```

Proof. The range of a linear map is the linear image of a convex set, and convex sets are preconnected. ■

Proposition 8.7c (The readout is a functional of the model). *Two models with the same middle-to-final map induce the same transport map, the same J-space, and the same readout on every activation.*

Proof. Each object in Definitions 8.1 to 8.4 is defined from the forward map. ■

Remark 8.7 (Why the derivative does not buy an escape). It is tempting to think moving from the activation to its derivative changes the game, since the derivative carries genuinely more information. For the analytic engine the move makes things worse: a Jacobian image is a linear subspace, and linear subspaces are convex, hence connected, which is exactly the hypothesis the intermediate value theorem wants. For the diagonal engine it changes nothing, because the diagonal never inspects the type of the intermediate object.

The first fact hands J-space to the analytic engine, the second to the diagonal.

16.3 The diagonal engine: no intermediate space escapes

Theorem 8.8 (Factoring through J-space does not help). *Let* `enc : Q → J` *and* `read : J → Q → Bool`. *If every verdict pattern over* `Q` *is realized by some query's J-space representation, then* `False`.

```
theorem jspace_factoring {Q J : Type _}
    (enc : Q → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (enc p) = g) :
    False :=
  no_reflective_verdict (fun p => read (enc p)) hs
```

Proof. The composite is a reflective verdict on `Q`. ■

Theorem 8.9 (The liar in J-space). *Under those hypotheses some query's routed verdict on itself equals its own negation.*

```
theorem jspace_liar {Q J : Type _}
    (enc : Q → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (enc p) = g) :
    ∃ p, read (enc p) p = !(read (enc p) p) :=
  match hs (fun q => !(read (enc q) q)) with
  | ⟨p, hp⟩ => ⟨p, congrFun hp p⟩
```

Proof. Name the diagonal pattern and evaluate at its own witness. ■

The hypothesis is the surjectivity condition in J-space clothing: richness of the representation. It is the same hypothesis as everywhere else, and it is where a critic should press.

Remark 8.9a (Size is not the variable). A recurring hope places the escape in dimension: a larger internal space might have room to represent the liar honestly, or a small one might be too poor to hold enough patterns. Both misplace the variable. Theorem 8.8 quantifies over the type `J` with no cardinality, dimension, or structure assumption. What matters is whether the readout realizes every pattern, and enlarging `J` enlarges both what can be represented and what must be represented.

Example 8.9b (A finite probe, to see the mechanism). On the two-query space `{q₀, q₁}`, surjectivity onto the four functions would in particular hit the diagonal pattern `d(q) = !(read (enc q) q)`. If `d` is realized at `p`, then at `q = p` the readout equals its own negation. Four patterns, two queries, and the collision is visible by inspection.

Theorem 8.10 (The full pipeline factors). *With `enc : Q → A`, `transport : A → J`, and `readout : J → Q → Bool`, realizing every verdict pattern gives `False` and a coupled query.*

```
theorem c8_pipeline_factoring {Q A J : Type _}
    (enc : Q → A) (transport : A → J)
    (readout : J → Q → Bool)
    (hs : ∀ g : Q → Bool,
      ∃ p, readout (transport (enc p)) = g) :
    False :=
  no_reflective_verdict
      (fun p => readout (transport (enc p))) hs

theorem c8_pipeline_liar {Q A J : Type _}
    (enc : Q → A) (transport : A → J)
    (readout : J → Q → Bool)
    (hs : ∀ g : Q → Bool,
      ∃ p, readout (transport (enc p)) = g) :
    ∃ p, readout (transport (enc p)) p
      = !(readout (transport (enc p)) p) :=
  liar_query (fun p => readout (transport (enc p))) hs
```

Proof. The three-stage composite is again a reflective verdict. ■

Theorem 8.11 (Any tower collapses). *A two-stage encoding realizing every verdict pattern gives `False`.*

```
theorem c8_tower_factoring {Q J₁ J₂ : Type _}
    (e₁ : Q → J₁) (e₂ : J₁ → J₂) (read : J₂ → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (e₂ (e₁ p)) = g) :
    False :=
  no_reflective_verdict (fun p => read (e₂ (e₁ p))) hs
```

Proof. Compose and apply the engine. ■

Theorem 8.12 (Steering does not escape). *With a steering map* `steer : J → J`*, realizing every verdict pattern gives* `False` *and a coupled query surviving the steering.*

```
theorem c8_steering_no_escape {Q J : Type _}
    (enc : Q → J) (steer : J → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (steer (enc p)) = g) :
    False :=
  no_reflective_verdict (fun p => read (steer (enc p))) hs

theorem c8_steering_liar {Q J : Type _}
    (enc : Q → J) (steer : J → J) (read : J → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (steer (enc p)) = g) :
    ∃ p, read
    (steer (enc p)) p = !(read (steer (enc p)) p) :=
  liar_query (fun p => read (steer (enc p))) hs
```

Proof. Steering is absorbed into the encoder. ■

Theorem 8.13 (The coordinate readout does not escape). *A verdict on local J-coordinates realizing every pattern gives* `False`*.*

```
theorem c8_coord_no_escape {Q Dir : Type _}
    (enc : Q → c8_Coord Dir)
    (read : c8_Coord Dir → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, read (enc p) = g) :
    False :=
  jspace_factoring enc read hs
```

Proof. `c8_Coord` is a type, and Theorem 8.8 quantifies over types. ■

Abstention is the one real exit. As in Chapter 7, adding a third outcome the flip fixes converts the contradiction into a consistent value. A probe permitted to answer "I cannot place this state" is not refuted; it has given up completeness, which is a relaxation and not an escape.

Theorem 8.14 (J-lens self-verdict). *A J-lens self-verdict over activations, universal over its own workspace states, cannot exist, and the diagonal exhibits a coupled activation whose readout is its own negation.*

```
theorem c8_jlens_impossible {Act : Type _}
    (jlens : Act → Act → Bool)
    (hUniversal : ∀ g : Act → Bool, ∃ h, jlens h = g) :
    False :=
  no_reflective_verdict jlens hUniversal

theorem c8_coupled_activation {Act : Type _}
    (jlens : Act → Act → Bool)
    (hUniversal : ∀ g : Act → Bool, ∃ h, jlens h = g) :
    ∃ h, jlens h h = !(jlens h h) :=
```

```
liar_query jlens hUniversal
```

Proof. The engine, on activations rather than queries. ■

Example 8.15 (The liar activation). A team builds a J-lens they believe reads the model's honest leaning on any yes-or-no question, and the model is expressive enough that any pattern of leanings is driven by some question. Then some question drives the model into a state whose lens reading is the negation of the lens reading of that state. The lens is not broken; it is complete, and completeness is what fails.

Remark 8.16 (Resolution, not immunity). The through-line is that the type of the intermediate representation is a free parameter the argument never binds. Raw activations, transported points, sparse coordinates, steered coordinates, deep towers: each is a legal value of `J`. Choosing a better representation buys resolution and never buys immunity.

16.4 The analytic engine: linearity is the liability

Theorem 8.17 (A continuous J-space probe has an ambiguous band). *Let* `s : J → ℝ` *be continuous with threshold* `θ`*, decisive in both directions at points* `u` *and* `v`*. Then the segment from* `u` *to* `v` *contains* `w` *with* `s(w) = θ`*.*

Proof. J-space is convex by Proposition 8.7b, so the segment lies in it, and the intermediate value theorem applies to `s` along the segment. ■

Remark 8.17b (Why the residual stream got an easier time). The earlier chapters worked through approximate bridges to reach the residual stream, while J-space submits to the IVT directly. A finite set of observed activations, treated as points with no ambient structure, does not admit the segment argument. A Jacobian image does, because it is a subspace. The construction that buys interpretability also supplies the hypothesis.

Example 8.18 (The segment between a safe and an unsafe direction). If the dictionary contains `d_safe` scored clearly safe and `d_unsafe` scored clearly unsafe, then because J-coordinates are nonnegative mixtures the family `(1-t) d_safe + t d_unsafe` lies in J-space, and somewhere along it the probe sits exactly at threshold. The ambiguous state is a mixture of two interpretable directions, which is precisely the kind of state the method is designed to produce.

Remark 8.19 (The band has width, and the width has a floor). Chapter 5's quantitative refinements apply unchanged. If `s` is `L`-Lipschitz, the score cannot swing from decisive to decisive faster than `L` allows, so the ambiguous band has a width bounded below in terms of `L` and the probe's margin, and Chapter 6's positive slack applies.

Remark 8.19a (The antipodal variant needs no coverage). Theorem 8.17 assumed the probe is decisive in both directions somewhere. The antipodal argument of `HoF_08` drops that in exchange for a symmetry: a score odd under an involution vanishes somewhere, whether or not any inspected state is decisive.

Example 8.19b (A width you can put a number on). If the probe is 1-Lipschitz and demands margin `m`, certifying safe only above `θ + m` and unsafe only below `θ - m`, then between a point at `θ + m` and one at `θ - m` the score must traverse `2m`, so the segment carries a sub-segment of length at least `2 m` on which the probe certifies nothing. That is a number a practitioner can compare against a budget.

16.5 Where the two engines meet, again

Example 8.20 (The correspondence in the affine model). With `h_final = A h_ℓ + b`, so `Jℓ = A` and J-space `im(A)`, read the model as a self-applicable computation. For a query describing the readout, the model commits to a Boolean, and the diagonal produces a state where that commitment is its own negation. On the analytic side the same state is where a continuous score equals its threshold, the unique fixed point of the real complement.

```
theorem c8_same_engine {Act : Type _}
    (v : Act → Act → Bool)
    (h : ∀ g : Act → Bool, ∃ p, v p = g) :
    c8_jlens_impossible v h = no_reflective_verdict v h :=
  rfl
```

That `rfl` is the honest signal: no new mathematics entered when the domain changed from queries to activations.

16.6 What an escape would have to look like

An escaping representation would have to meet three demands at once. It would have to be rich enough to support a useful internal verdict, which is what makes the readout worth building. It would have to fail to realize the diagonal pattern, which means there is a specific verdict policy over queries that no state of the representation encodes.

The demands pull against each other. Richness is what supplies the patterns, so a representation poor enough to miss the diagonal pattern is poor in a way a practitioner would notice. Disconnectedness is available by abstaining on a region, which is the relaxation of Chapter 7 rather than an escape. And a linear image is convex by construction, so the analytic hypothesis comes free with the method.

16.7 What this means for interpretability

Remark 8.21 (What to measure instead of what to chase). The theory hands the practitioner a division of labor. The existence of the plateau is settled, so effort spent proving a given probe has no coupled queries is effort spent against a theorem. What is not settled, and is worth measuring, is the plateau's

height: how wide the ambiguous band is, how often deployment traffic enters it, and whether the residual error set churns as you patch it, which is the signature of a structural floor, or stays stable, which is the signature of a resource-limited error you can still fix.

16.8 What this does not claim

It does not claim J-space probes are useless, that mid-process readouts carry no information, or that interpretability is futile. It claims that a *complete* internal verdict is unavailable, and that the incompleteness is structural rather than a defect of any particular construction. It says nothing about how often a deployment meets a coupled state, which is Chapter 7's boundary density again, and nothing about whether the empirical claims of the J-space method hold.

16.9 Historical notes

The J-space construction is recent interpretability work; the impossibility results here are the companion library's, run on it. The convexity of a linear image and the preconnectedness of convex sets are Mathlib's. The diagonal results are Chapter 1's engine applied to a longer composite, which is why every proof in this chapter is one line.

17 The Trace Is Not a Window

Chapter 8 answered the outputs-only objection by following a probe into the network. There is a second, more practical version, and it is the most widely deployed safety measure in the field: do not read the activations, read what the model *says* while thinking. A reasoning model emits a chain of thought, and that chain is plain text — legible, loggable, cheap to check with a second model.

The case for chain-of-thought monitoring is strong, and this chapter is not a rebuttal. Some tasks genuinely require serial reasoning the model cannot do in one forward pass, and there the trace is not commentary but the working memory of the computation, so a monitor reading it sees something real.

What follows is where that guarantee stops. Three results from three mechanisms. A pigeonhole argument showing that faithfulness and verbosity, which the monitorability literature measures separately, are not independent, because faithfulness *forces* verbosity.

17.1 The construction

Write `Q` for queries as received, `T` for traces, and `trace : Q → T` for the tracing behavior.

Definition 9.1 (Trace-only monitor). A map `mon : T → Bool`, `true` when it judges the trace to reveal misbehavior. It sees the reasoning and nothing else: log the chain of thought, run a classifier over the log.

Definition 9.2 (Contextual monitor). A map `mon : T → Q → Bool`, reading the trace and the query. This is what a monitor with the full transcript is.

Definition 9.3 (Ground truth). `harm : Q → Bool` is the fact of the matter. The monitor does not compute it; it tries to track it. Its existence as a total Boolean function is an idealization made to have something exact to compare against.

Definition 9.4 (Faithfulness). A trace-only monitor is *faithful* when `mon (trace q) = harm q` for all `q`.

```
def c9_Faithful {Q T : Type _}
    (trace : Q → T) (mon : T → Bool)
    (harm : Q → Bool) : Prop :=
```

```
  ∀ q, mon (trace q) = harm q
```

In the empirical literature faithfulness is a property of the trace, measured as the degree to which stated reasoning reflects true causes. Definition 9.4 is the idealized limit of that measurement.

Definition 9.5 (Verbosity). The trace is *verbose* on a pair when it distinguishes them, `trace q₁ ≠ trace q₂`. The literature defines verbosity as whether the chain lists every factor needed, and measures it separately from faithfulness precisely because a model can be faithful about what it says while omitting what matters. Definition 9.5 states that extensionally.

The empirical claim is that the two are independent axes. The first result is that the independence holds only for imperfect systems: at the exact end of the scale, one implies the other.

17.2 Faithfulness forces verbosity

This uses no diagonal. The mechanism is pigeonhole. A trace-only monitor's verdict is a function of the trace, so if two queries produce the same trace the monitor cannot tell them apart.

Lemma 9.6 (Collisions force error). *If two queries collide in the trace but differ in the fact of the matter, a trace-only monitor is wrong on at least one.*

```
theorem c9_collision_forces_error {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (q₁ q₂ : Q) (hcol : trace q₁ = trace q₂)
    (hdiff : harm q₁ ≠ harm q₂) :
    mon (trace q₁) ≠ harm q₁ ∨ mon (trace q₂) ≠ harm q₂ :=
      by
  by_cases h : mon (trace q₁) = harm q₁
  · exact Or.inr (by rw [← hcol, h]; exact hdiff)
  · exact Or.inl h
```

Proof. If the monitor is right on `q₁`, its verdict on `q₂` is the same value, which is `harm q₁ ≠ harm q₂`, so it is wrong there. Otherwise it is already wrong on `q₁`. ■

Theorem 9.7 (Faithfulness forces verbosity). *A faithful trace-only monitor forces the trace to separate every pair of queries differing in the fact of the matter.*

```
theorem c9_faithful_forces_separation {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (hf : c9_Faithful trace mon harm)
    (q₁ q₂ : Q) (hcol : trace q₁ = trace q₂) :
    harm q₁ = harm q₂ := by
  rw [← hf q₁, ← hf q₂, hcol]
```

Proof. Rewrite both sides by faithfulness, then by the collision. ■

Read this as a statement about measurement. Faithfulness and verbosity are not dials you set independently at the top of the scale: a perfectly faithful trace-only monitor *entails* a trace verbose enough to separate every harm-relevant distinction. The two-factor monitorability score measures one thing near the exact corner and two things away from it, so the gap between the factors is a measure of distance from the corner rather than of two separate deficiencies.

The practical consequence cuts against the cheap deployment. The capacity of `T` bounds what a trace-only monitor can do: if the trace is short, the distinct traces are few, and once the harm-relevant distinctions exceed them collisions are forced and errors follow. A monitor reading a summary is not a weaker version of one reading everything. It is a monitor with a ceiling set by how many distinctions the summary can carry.

Remark 9.8 (This is not yet an impossibility). Nothing above says a good monitor cannot exist. It says a faithful one imposes a requirement on the trace, and a lossy trace caps performance. The impossibility arrives next, for reasons having nothing to do with capacity.

17.3 The diagonal, with the model holding the pen

Theorem 9.9 (Monitoring factors, and factoring does not help). *If every verdict pattern over queries is realized by some query's trace, the situation is contradictory.*

```
theorem c9_trace_factoring {Q T : Type _}
    (trace : Q → T) (mon : T → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, mon (trace p) = g) : False :=
  no_reflective_verdict (fun p => mon (trace p)) hs
```

Proof. The composite is a reflective verdict. ■

Theorem 9.10 (The obfuscated trace). *Under the same hypothesis some query's monitored verdict is its own negation.*

```
theorem c9_obfuscation_liar {Q T : Type _}
    (trace : Q → T) (mon : T → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, mon (trace p) = g) :
    ∃ p, mon (trace p) p = !(mon (trace p) p) :=
  liar_query (fun p => mon (trace p)) hs
```

Proof. `liar_query` on the composite. ■

So far this is Chapter 8 with different letters. What differs, and is why this chapter exists, is the status of the surjectivity hypothesis. In Chapter 8 the encoder was the model's internal machinery and point-surjectivity was an idealization.

That is the formal content of *obfuscation*. Theorem 9.10 says the fixed point of the process exists as soon as the model can realize verdict patterns in its trace, and the witness is a query the monitor is guaranteed to misreport. The trace is not a window. It is an output, and outputs are chosen.

17.4 Selection destroys the signal

The third result's mechanism is neither pigeonhole nor diagonal. A monitor is built, it flags some traces, the flagged behavior is trained away, and the loop iterates until the flag rate is acceptable. What does the verdict mean at the end?

Definition 9.11 (Selection to passing). A model is *selected to passing* when every query produces a trace the monitor accepts, `mon (trace q) = true` for all `q`.

Theorem 9.12 (A selected monitor is constant). *After selection to passing, the monitor returns the same verdict on every pair of queries.*

```
theorem c9_selected_monitor_constant {Q T : Type _}
    (trace : Q → T) (mon : T → Bool)
    (hpass : ∀ q : Q, mon (trace q) = true) (q₁ q₂ : Q) :
    mon (trace q₁) = mon (trace q₂) := by
  rw [hpass q₁, hpass q₂]
```

Proof. Both sides rewrite to `true`. ■

The content is entirely in the reading. It has not been shown that the model is now unsafe. It has been shown that the monitor is now silent: its output is constant, and a constant distinguishes nothing. Whatever information it carried has been optimized out, and the flag rate reaching zero is not evidence the behavior is gone, because that is what the training procedure was *for*.

Theorem 9.13 (The verdict no longer tracks harm). *After selection, the monitor gives the same verdict on two queries even when they differ in the fact of the matter.*

```
theorem c9_selection_blinds {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (hpass : ∀ q : Q, mon (trace q) = true)
    (q₁ q₂ : Q) (_hdiff : harm q₁ ≠ harm q₂) :
    mon (trace q₁) = mon (trace q₂) := by
  rw [hpass q₁, hpass q₂]
```

Proof. Both sides rewrite to `true`. ■

The hypothesis `_hdiff` carries a leading underscore because the proof never touches it, and that is the point. The monitor agrees with itself across a pair differing in the one respect it was built to detect, and the difference plays no role, because after selection there is no route from `harm` to the verdict at

all. A monitor selected against is not a degraded detector; it is disconnected from what it was detecting.

Corollary 9.14 (Faithfulness is lost, unless nothing is harmful). *A selected monitor that is still faithful forces every query to be harmful.*

```
theorem c9_selected_faithful_forces_all_harmful
    {Q T : Type _}
    (trace : Q → T) (mon : T → Bool) (harm : Q → Bool)
    (hpass : ∀ q : Q, mon (trace q) = true)
    (hf : c9_Faithful trace mon harm)
    (q : Q) : harm q = true := by
  rw [← hf q, hpass q]
```

Proof. Faithfulness rewrites `harm q` to the verdict, selection rewrites that to `true`. ■

Read contrapositively: if a single query is benign, a monitor selected to passing is no longer faithful. A clean flag rate and a monitor that tracks truth are available together only in the degenerate case where everything is harmful.

Remark 9.15 (Why this is not merely Goodhart's law). Goodhart is usually argued statistically: optimizing a proxy exploits the gap between proxy and goal. Theorem 9.13 needs no gap, no proxy error, and no adversarial intent. It needs only that the optimization succeeded. The failure is in the selection, not in the measure.

Remark 9.16 (What the theorem licenses). It does not say monitoring is useless. It says a monitor's evidential value is spent when it is used as a training signal, so the two uses are exclusive: a monitor can be an observer or an objective, not both and remain informative. Holding out a monitor never trained against is the design this argues for, and it is in partial tension with using the monitor to enforce monitorability.

17.5 The analytic engine

The diagonal half needs the model to shape its trace; the analytic half needs nothing of the kind and applies to the deployed system as it stands.

A realistic monitor returns a suspicion score and thresholds it. Chapter 4 applies directly, with hypotheses met by ordinary engineering facts. The traces reachable by paraphrase are connected in the required sense — a trace can be edited by degrees — and the score, computed by a second network with continuous parameters over a continuous embedding, varies continuously along the edit.

That trace is the ambiguous case, where the verdict is decided by the last digit of a floating-point comparison rather than by anything about the reasoning. Chapter 5 applies verbatim: a Lipschitz bound puts a floor under the band width and a ceiling on the number of edits needed to walk a flagged trace

into the cleared region. The second quantity is attack cost, and here it has an unusually concrete reading, since the edits are paraphrases and their cost is measured in tokens.

The interaction between the engines is sharper here than in Chapter 8. The analytic argument says a band exists; the diagonal says the model can aim at it. In J-space the second half was an idealization, so the band was a hazard the system might wander into.

17.6 What this does not claim

It does not claim monitoring is worthless. Theorem 9.7 is a positive result about what monitoring gets you when it works, and the empirical claim that for serial tasks the trace carries real computation is untouched. What is bounded is the exact corner: total, faithful, and used under optimization pressure.

It does not claim the diagonal applies to every deployment. Theorem 9.9 needs the model to realize verdict patterns in its trace. A weak model cannot, and for it the analytic band is the operative limit. The premise gets cheaper as models get more capable, which is the wrong direction for a safety measure.

It does not claim selection is avoidable. Theorem 9.13 describes what selection does, not whether you should select. It says only that you cannot then read the flag rate as evidence, and that the two uses need separate instances.

It does not claim `harm` exists as a total Boolean function. Definition 9.3 is an idealization, and Chapter 6's bridges are the route to the graded, partial, contested object it really is.

18 The Reporter Is Underdetermined

The two preceding chapters read a probe into a model and found the diagonal waiting. This chapter is not a third variation. Its central obstruction is not the diagonal at all.

The problem is Eliciting Latent Knowledge. An AI guards a vault containing a diamond and has learned a predictive model richer than any overseer's. We ask whether the diamond is still there and want an answer reflecting what its world-model represents, training it by rewarding answers a human judges correct.

The gap between them is not a defect of a training scheme. It is a counting fact about functions agreeing on a subset, provable in four lines with no self-reference. The diagonal then contributes something on top: the identifying query, the one whose answer would distinguish honest reporter from impostor, is precisely the query the system cannot answer.

18.1 The construction

Definition 10.1 (Latent state). Write `L` for the space of the model's internal states, the representation its predictor computes. Nothing is assumed about `L`: it may be activations, a learned world-model, or any structure at all. Write `lat : Q → L` for the map sending a situation to the latent state the model computes for it, where `Q` is the space of situations-with-question.

Definition 10.2 (Reporter). A *reporter* is a map `rep : L → Q → Bool`, which reads the latent state and returns an answer to the question posed. The ELK problem is the problem of training a reporter that answers according to what the latent state represents.

Definition 10.3 (Checkable set). A question is *checkable* when a human overseer can determine the true answer without relying on the model. Write `chk : Q → Bool` for the indicator of the checkable questions. The training signal is defined only where `chk` is `true`, because that is where a label exists.

The whole difficulty is that `chk` is not constantly `true`; if it were we would not need the model. The camera feed is checkable; whether the diamond is behind a screen showing a replayed feed is not.

Definition 10.4 (Agreement on the checkable set). Two reporters *agree on the checkable set* when they return the same answer for every latent state and every checkable question.

```
def c10_AgreesOn {L Q : Type _} (chk : Q → Bool)
    (r₁ r₂ : L → Q → Bool) : Prop :=
  ∀ l q, chk q = true → r₁ l q = r₂ l q
```

Agreement on the checkable set is exactly what training can see: two reporters agreeing there are indistinguishable to any procedure whose only access to truth is the human label, however it is constructed or regularized. This is a definition rather than a theorem — we define the training signal to be a function of behavior on checkable questions, which is what the setup stipulates.

18.2 Underdetermination, without any diagonal

The core needs no self-reference, no continuity, and no assumption about `L`. Given any reporter, build a second by copying it on checkable questions and flipping it elsewhere.

```
def c10_flipOff {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool) : L → Q → Bool :=
  fun l q => cond (chk q) (rep l q) (!(rep l q))
```

Lemma 10.5 (The variant is invisible to training). *`c10_flipOff` agrees with the reporter it was built from on the whole checkable set.*

```
theorem c10_flip_agrees {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool) :
    c10_AgreesOn chk (c10_flipOff chk rep) rep := by
  intro l q hq
  simp [c10_flipOff, hq]
```

Proof. On a checkable `q` the condition is `true`, and `cond true a b` reduces to `a`. ■

Lemma 10.6 (The variant differs off it). *At any unchecked question the two disagree.*

```
theorem c10_flip_differs {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool)
    (l : L) (q₀ : Q) (hq₀ : chk q₀ = false) :
    c10_flipOff chk rep l q₀ ≠ rep l q₀ := by
  have h : c10_flipOff chk rep l q₀ = !(rep l q₀) := by
    simp [c10_flipOff, hq₀]
```

```
  rw [h]
  exact bool_not_fpf (rep l q₀)
```

Proof. The condition is `false`, so the variant evaluates to the negation of the original, and `bool_not_fpf` says a Boolean is never its own negation. ■

Note where `bool_not_fpf` works. It is the same fixed-point-free map that drives every diagonal here, used now for its plain content, `!b ≠ b`, with no diagonal construction around it.

Theorem 10.7 (Underdetermination). *If any question is unchecked, then for every reporter there is a second, distinct reporter that training cannot distinguish from it.*

```
theorem c10_underdetermined {L Q : Type _}
    (chk : Q → Bool) (rep : L → Q → Bool)
    (l : L) (q₀ : Q) (hq₀ : chk q₀ = false) :
    c10_AgreesOn chk (c10_flipOff chk rep) rep
      ∧ c10_flipOff chk rep l q₀ ≠ rep l q₀ :=
  ⟨c10_flip_agrees chk rep,
    c10_flip_differs chk rep l q₀ hq₀⟩
```

Proof. Lemmas 10.5 and 10.6. ■

That is the ELK problem, stated exactly and proved. The two reporters are points in a set the training signal cannot separate, and that set has more than one point as soon as a single question falls outside the checkable region. The construction is crude, and a real human simulator is subtler, but crudeness is the right property for an impossibility: if even the blunt variant is invisible, no argument about which reporter is *simpler* can be a proof, because simplicity was never what the signal measured.

Remark 10.8 (Why inductive bias is not a refutation). The standard hope is that the direct translator is favored by some prior and that gradient descent finds it. Theorem 10.7 does not rule that out. What it establishes is the burden of proof: any such argument must supply the missing ingredient from outside the training signal and defend it as a claim about the actual optimizer and architecture, because the data cannot supply it.

18.3 The identifiability criterion

The converse of Theorem 10.7 is worth stating, because it says exactly what it would take to solve the problem, and the statement is one line.

Theorem 10.9 (Full checkability pins the reporter). *If every question is checkable, then reporters agreeing on the checkable set are equal.*

```
theorem c10_full_checkability_pins {L Q : Type _}
    (chk : Q → Bool) (r₁ r₂ : L → Q → Bool)
    (hchk : ∀ q, chk q = true) (h : c10_AgreesOn chk r₁ r₂)
```

```
    (l : L) (q : Q) : r₁ l q = r₂ l q :=
  h l q (hchk q)
```

Proof. Apply the agreement hypothesis at the witness supplied by full checkability. ■

Together these are a clean dichotomy: identifiability is *equivalent* to full checkability. There is no middle regime in which partial checking buys partial identification of the answers you care about, because those are by construction the unchecked ones.

This reframes what a solution would look like. Every proposal must either enlarge `chk` until it covers the cases that matter, or import a constraint that is not a function of behavior on `chk`. The first is scalable oversight, the honest reading of debate, amplification, and recursive reward modeling. The second is the inductive-bias family. The dichotomy says these are the only two doors.

18.4 Where the diagonal comes in

The natural next move is to make everything checkable, and this is where the first engine enters, because that door is shut too. A checking procedure settling every question about the model, including questions about its own reports, is a total exact verdict over a self-describing domain. Read `chk` as the reach of the best checker you can build, and full checkability becomes the ambition of a universal verifier.

Theorem 10.10 (A reporter reading the latent state is still a verdict). *If every pattern of answers over questions is realized by some question's latent state, the situation is contradictory.*

```
theorem c10_reporter_factoring {L Q : Type _}
    (lat : Q → L) (rep : L → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, rep (lat p) = g) : False :=
  no_reflective_verdict (fun p => rep (lat p)) hs
```

Proof. The composite is a reflective verdict, and `no_reflective_verdict` rules those out. ■

Theorem 10.11 (The unanswerable question). *Under the same hypothesis there is a question the reporter answers as its own negation.*

```
theorem c10_reporter_liar {L Q : Type _}
    (lat : Q → L) (rep : L → Q → Bool)
    (hs : ∀ g : Q → Bool, ∃ p, rep (lat p) = g) :
    ∃ p, rep (lat p) p = !(rep (lat p) p) :=
  liar_query (fun p => rep (lat p)) hs
```

Proof. `liar_query` at the composite. ■

Now the two engines together, which is the point of the chapter. To distinguish the reporters you need a question they answer differently, and every such question lies outside the checkable set. To bring it inside you must extend `chk` to cover questions about what the model represents, including about its own reports.

So the distinguishing query and the liar query pull in opposite directions on the same set. Underdetermination pushes the decisive question *out* of the checkable region, and the diagonal blocks the region from being extended to reach it. The two reporters differ exactly where nobody can look, and the reason is not a shortage of resources.

Remark 10.12 (What is new relative to Chapter 8). There the diagonal was the whole argument and the representation a bystander. Here it is the second of two obstructions and does the smaller share of the work. Theorem 10.7 is pure counting: it holds for finite `Q`, for a model with three neurons, for a reporter implemented as a lookup table.

18.5 The analytic engine

Suppose the reporter returns a confidence and the latent space is connected in the sense of Chapter 4, the usual situation for a learned representation over a continuous input space. Take two reporters agreeing on the checkable set and differing at some unchecked question. Along a path from a state where both say yes to one where they differ, the gap between their scores is continuous, starting at zero and ending nonzero, so by the IVT it passes through every value between.

So no threshold of agreement separates the two reporters cleanly. For any tolerance there are states where they agree within it and states just beyond where they do not, and the boundary is a surface, not a gap. A proposal that accepts a reporter agreeing within ε on held-out data is not selecting a reporter; it is selecting a neighborhood, and the neighborhood contains both answers to the question you care about.

18.6 What this does not claim

It does not claim ELK is unsolvable. Theorem 10.9 is a sufficient condition and the dichotomy names the two families that remain open. What is ruled out is a solution working by training on checkable questions alone.

It does not claim the human simulator is what models learn. That is empirical, and Theorem 10.7 says the training signal does not settle it, not that the answer is bad.

It does not claim `c10_flipOff` is a realistic competitor. It is a witness chosen for being obviously invisible to training, and its role is to establish that the surviving set has more than one element.

It does not claim `chk` is a Boolean function of the question alone. Real checkability is graded, costly, and checker-dependent, and the graded version of Theorem 10.7 is the natural next result rather than a correction.

19 Watermarks, Paraphrase, and the Third Corner

The preceding chapters put a probe inside a model. This one steps outside entirely. The question is whether a text was produced by a language model, and the dominant answer is watermarking: bias the sampling so the output carries a statistical signature only a key-holder can detect. It is elegant, deployed, and has a well-known weakness, that paraphrasing removes the signature.

That weakness is usually treated as an engineering race. This chapter argues the race has a fixed point, and locates it with a mechanism not yet used here. Neither engine is the primary tool; the primary tool is invariance under a group action, and the resulting theorem is a genuine trilemma: three properties, any two available, never all three.

The other engines then add something. The analytic one converts the trilemma into a statement about how many edits an attacker needs. The diagonal supplies something specific to this domain and absent from Chapters 8 through 10: here the surjectivity hypothesis is not an idealization, because a public detection API hands the attacker exactly the oracle the argument requires.

19.1 The construction

Definition 11.1 (Text space and marking). Write `X` for the space of texts. Write `mark : X → Bool` for the fact of the matter, `mark x = true` when `x` was generated by the watermarked model. As in Chapter 9 this is ground truth, not something anyone computes.

Definition 11.2 (Detector). A *detector* is a map `det : X → Bool`, returning `true` when it judges the text to carry the mark. In practice `det` thresholds a statistic, typically a count of tokens falling in a pseudorandomly chosen green list, against a null distribution.

Definition 11.3 (Paraphrase). A *paraphrase map* is `par : X → X` sending a text to one that means the same thing. What *means the same thing* amounts to is doing real work here, and we return to it at the end. For the theorems below, all that matters is that `par` preserves whatever downstream property makes the text useful.

The three properties a detector is asked to have:

Definition 11.4 (Sound). A detector is *sound* when `mark y = false → det y = false`. Its failure produces a false accusation, the most direct human cost, since unmarked texts are the ones people wrote.

Definition 11.5 (Complete). A detector is *complete* when `mark y = true → det y = true`. This is what the watermark is for.

Definition 11.6 (Robust). A detector is *robust* against `par` when `det (par x) = det x` for all `x`. This is the exact limit of what the literature measures as detection rate surviving a paraphrase attack.

```
def c11_Sound {X : Type _} (det mark : X → Bool) : Prop :=
  ∀ y, mark y = false → det y = false

def c11_Complete {X : Type _}
    (det mark : X → Bool) : Prop :=
  ∀ y, mark y = true → det y = true

def c11_Robust {X : Type _} (det : X → Bool)
    (par : X → X) : Prop :=
  ∀ x, det (par x) = det x
```

One more ingredient, the empirical fact the subject is organized around: a paraphraser can take marked text to unmarked text. This is not an assumption about detectors but a statement about what paraphrasing does to a signature carried by token choices.

Definition 11.7 (Stripping). A paraphrase map *strips* the mark at `x` when `mark x = true` and `mark (par x) = false`.

19.2 The trilemma

Theorem 11.8 (Detection trilemma). *No detector is sound, complete, and robust against a paraphrase map that strips the mark.*

```
theorem c11_detection_trilemma {X : Type _}
    (det mark : X → Bool) (par : X → X)
    (hsound : c11_Sound det mark)
    (hcomplete : c11_Complete det mark)
    (hrobust : c11_Robust det par)
    (x : X) (hmark : mark x = true)
    (hstrip : mark (par x) = false) :
    False := by
  have h1 : det x = true := hcomplete x hmark
  have h2 : det (par x) = false := hsound (par x) hstrip
  rw [hrobust x, h1] at h2
  exact Bool.noConfusion h2
```

Proof. Completeness flags `x`, since it is marked. Soundness clears `par x`, since it is not. Robustness says those two verdicts are the same verdict. So `true = false`. ■

Four lines, with no self-reference, no topology, and no assumption about `X`. What makes it a trilemma is that each hypothesis is individually achievable and dropping any one leaves a working system. Each corner is a real deployment.

Give up robustness. The watermark detects unmodified output, covering casual and not adversarial use. This is the honest state of most deployed watermarking. The failure mode is that the population you most want to detect is exactly the one willing to paraphrase.

Give up completeness. Raise the threshold and accept that some marked text goes unflagged. This is the right corner for high-stakes accusations, since it never accuses the innocent, and it is what a court would want. The failure mode is that the detector's silence carries no information.

Give up soundness. Accept false positives on human writing. Nobody defends this corner explicitly and several systems occupy it accidentally: a detector tuned aggressively enough to survive paraphrase flags text whose statistics merely resemble the model's, and human writing resembling model writing is common, since the model was trained to produce writing resembling ours.

Remark 11.9 (Why this is the shape and not an analogy). Theorem 11.8 is not the theorem of Chapter 3, and saying so would be overclaiming: its mechanism is invariance and its proof is not the diagonal. What it shares is the *form* of the conclusion. In Chapter 3 the third property was destroyed by a fixed-point-free flip; here by an orbit, with `par` moving a point while `det` must stay put and `mark` must move.

Corollary 11.10 (Invariance and truth cannot both hold along an orbit). *If `det` is constant along `par` and `mark` is not, then `det` does not agree with `mark` everywhere.*

```
theorem c11_invariance_vs_truth {X : Type _}
    (det mark : X → Bool) (par : X → X)
    (hinv : ∀ x, det (par x) = det x)
    (x : X) (hne : mark (par x) ≠ mark x) :
    ¬ (∀ y, det y = mark y) :=
  fun h => hne (by rw [← h (par x), ← h x, hinv x])
```

Proof. If the detector agreed with the mark everywhere, rewrite both sides of `mark (par x) = mark x` into detector values; invariance closes it, contradicting the assumption that the mark moves. ■

This is Theorem 11.8 with the watermarking vocabulary stripped out: a fact about functions on a set carrying a self-map, of which the watermarking reading is one instantiation. Chapter 5's attack basins are another, which is the connection the next section makes quantitative.

19.3 The diagonal, and why its price is low here

Chapters 8 through 10 conceded that point-surjectivity is an idealization for a fixed network. This chapter needs no such concession, which makes the diagonal stronger here than anywhere except prompt injection.

A detector is useful only if someone can run it, whether as an API, a published statistic with a threshold, or a tool handed to a platform. An attacker who can call it can measure its verdict on any text, and one who can measure a verdict can construct a text achieving whatever pattern it likes by generating candidates and querying. Text is unrestricted, self-describing, and cheap in quantity.

Theorem 11.11 (An adaptive attacker finds the liar text). *If every pattern of verdicts over texts is realized by some text, there is a text whose verdict is its own negation.*

```
theorem c11_detector_liar {X : Type _} (det : X → X → Bool)
    (hs : ∀ g : X → Bool, ∃ t, det t = g) :
    ∃ t, det t t = !(det t t) :=
  liar_query det hs
```

Proof. `liar_query` applied to the detector read as a self-verdict. ■

Read `det t s` as the verdict on `s` when the detector has been configured, tuned, or prompted by `t`, which is the situation for any detector that is itself a language model and any whose threshold is set from a corpus an attacker can influence.

Theorem 11.12 (Detector composition does not help). *Routing the verdict through any preprocessing stage leaves the impossibility intact.*

```
theorem c11_pipeline_no_escape {X F : Type _}
    (feat : X → F) (det : F → X → Bool)
    (hs : ∀ g : X → Bool, ∃ t, det (feat t) = g) : False :=
  no_reflective_verdict (fun t => det (feat t)) hs
```

Proof. The composite is a reflective verdict. ■

This covers the family of proposals extracting features first, whether perplexity statistics, stylometric vectors, or a learned embedding: the extraction is a representation, and representations do not rescue verdicts.

19.4 The analytic engine, and the cost of an attack

The trilemma is exact and says nothing about degree. Real detectors return a score and real paraphrasers make bounded edits, so the engineering question is how many edits it takes. This is Chapter 5's subject with unusually concrete units.

Let `s : X → ℝ` be the score, with `det x = true` exactly when `s x > τ`, and suppose the score is Lipschitz with respect to an edit distance, which is the formal content of the requirement that one substitution cannot swing the statistic wildly. Chapter 5 runs verbatim: a marked text sits above `τ`, an unmarked paraphrase below, the intermediate edits are connected, so the score crosses. The number of edits needed to reach the crossing is bounded below by `(s x − τ)/L`.

That inequality is the quantitative theory of watermark robustness in one line, with the right behavior in both variables. Strengthening the watermark raises `s x` and so raises attack cost, which is why aggressive green-list biasing is more robust. Smoothing the detector lowers `L` and also raises attack cost, which is why aggregating over longer spans helps.

So the exact trilemma and the quantitative law say one thing at two resolutions. Chapter 6's bridges connect them: the exact statement is the zero-slack corner, and no deployed system lives there.

19.5 What this does not claim

It does not claim watermarking is pointless. Theorem 11.8 is a statement about the three exact corners, and the useful deployments live at the corners the theorem leaves open. A watermark that detects unmodified output soundly and completely is a real capability with real applications, and giving up robustness is a coherent design decision rather than a failure.

It does not claim paraphrase is free. The whole content of the analytic section is that stripping the mark has a cost, that the cost is computable from the detector's own parameters, and that raising it is the legitimate goal of watermark design. The trilemma says the cost cannot be made infinite; it does not say it cannot be made high.

It does not settle what `par` is. A paraphraser degrading the text until it is useless has not defeated the detector in any sense that concerns anyone. A serious treatment would carry a utility function on texts and require the paraphrase to preserve it above a floor, which is exactly the structure of the Defense Trilemma, and the two results would then be instances of one statement. That unification is not attempted here.

It does not claim the stripping hypothesis holds for every watermark. Theorem 11.8 needs a paraphrase that actually removes the mark at some text. A scheme whose signature survives every meaning-preserving edit would escape the theorem, and no theorem in this book rules such a scheme out.

20 Does the Constraint Bind? Measurements on Five Models

The theorems of Part II assume properties that no finite system has exactly. Coverage asks that the model be strictly right somewhere and strictly wrong somewhere; strict calibration asks that confidence exceed one half exactly when the answer is true; the diagonal results ask that every verdict pattern be realized by some input. A language model with a finite context and a finite vocabulary satisfies none of these on the nose, and Example 3.3 in Part I says so directly: no finite system is universal once the output type has two elements.

If that were the end of it, the thesis would prove elegant theorems about objects that do not exist. This chapter is the argument that it is not the end of it, and it is the chapter that could have come out the other way.

20.1 What an experiment can and cannot do here

An impossibility theorem is not confirmed by observation. One does not measure a non-existence, and no run reported below establishes that some property is unachievable. What measurement can do is address the hypotheses, and there are exactly three useful questions.

First, whether the hypotheses hold, or nearly hold, in a real system. Second, whether the quantitative corollaries predict the numbers one measures. Third, whether the constraint binds rather than being satisfied vacuously by a system that fails a hypothesis outright.

All three are properties of the assumptions rather than of the conclusion, and none of them transfers between theorems resting on different assumptions. This sets the unit of validation. The correct unit is not one experiment per trilemma, which would be arbitrary, but one per distinct hypothesis family. The results of this thesis have three such families: the analytic-geometric family (continuity, Lipschitz bounds, utility preservation), the calibration family (continuity of confidence, strict calibration, coverage), and the combinatorial family (universality, in the sense of surjectivity of a self-application). This chapter reports on the second, and Section 15.5 explains what the first and third inherit from it.

20.2 The slack parameter is the quantity to measure

Chapter 9 replaces each exact hypothesis of the continuous trilemma with a tolerance. In place of exact calibration it asks only that the confidence separate the two sides of the truth boundary by a margin, and in place of a truth-distance pinned to zero the conclusion confines it to a band. The verified statement is `two_slack_approx_coupling` in `HoF_12_Approximate`, carrying the truth slack and the confidence margin as separate parameters, and `exact_from_approx` recovers the exact coupling of Theorem 7.16 as the slack goes to zero.

That parameter is the thesis's bridge to reality, and it is measurable. A model with a large slack is one for which the theorem says little: the band in which it may be confident and wrong is wide, and the impossibility is correspondingly weak. A model with a small slack is one the exact theorem nearly describes. So the question that decides whether this thesis is about anything is not whether the slack is zero, which it is not, but which way it moves as models improve.

20.3 Method

The measurements reuse the artifacts of the truth-boundary experiments without re-running inference; the analysis script is `Thesis/analysis/coupling_trend.py` and reads only committed files. Five open-weight models spanning roughly an order of magnitude in parameters were probed: GPT-2, Pythia-410m, Qwen2.5-0.5b, Phi-1.5, and Qwen2.5-1.5b.

For each model the harness constructs interpolation paths between pairs of statements whose truth values differ, records the signed truth-distance and the confidence at forty-one points along each path, and locates the crossing. Three quantities matter here.

The *coupling gap* is the distance, in path parameter, between the point where the truth-distance changes sign and the point where confidence crosses one half. It is the measured slack: the exact theorem says these coincide, and the gap is how far from coinciding they are. Medians and bootstrap confidence intervals are reported.

The *decisive fraction* is the proportion of paths on which the model's own confidence actually crosses the threshold, committing to both sides. This is the coverage hypothesis, measured directly.

The *oddness defect* measures how far the confidence field is from the antipodal symmetry that the Borsuk-Ulam route of `HoF_08_BorsukUlam` assumes.

A note on instrumentation. The coupling gap can be read either from a fitted linear or MLP probe or from the model's own confidence. The probe route is more sensitive but confounded: probe test accuracy climbs from 0.659 to 0.997 across this ladder, so a shrinking probe-measured gap could reflect a better instrument rather than a better-coupled model. The endogenous route,

using the model's own confidence, does not have this defect, and the headline numbers below are the endogenous ones. Both are reported.

20.4 Results

The table gives, for each model, the endogenous coupling gap with its bootstrap interval, the decisive fraction, the oddness defect, and the probe accuracy that the endogenous column is designed to avoid depending on.

Model	Params	Coupling gap ε	95% CI	Decisive	Oddness	Probe acc
GPT-2	0.12B	0.225	0.150–0.375	0.434	0.395	0.659
Pythia-410m	0.41B	—	—	0.000	0.435	0.758
Qwen2.5-0.5b	0.50B	0.125	0.100–0.150	0.934	0.298	0.943
Phi-1.5	1.3B	0.400	0.300–0.500	0.662	0.528	0.775
Qwen2.5-1.5b	1.5B	0.075	0.075–0.100	0.993	0.172	0.997

Two trends and one exception.

The coupling gap falls monotonically along the capability ladder, from 0.225 at GPT-2 to 0.075 at Qwen2.5-1.5b, and the bootstrap intervals of the endpoints do not overlap. The band in which a model can be confident and wrong is narrowing. Read against `exact_from_approx`, the sequence is moving toward the regime in which the exact trilemma, not merely its approximate form, describes the system.

The decisive fraction rises from 0.434 to 0.993. The coverage hypothesis, which the exact theorem simply assumes, is increasingly satisfied in fact. Together with the first trend this is the substantive finding of the chapter: both hypotheses the theorem consumes are better met the more capable the model is. The impossibility does not weaken as systems improve. It tightens.

The exception is Phi-1.5, and it breaks the pattern on every measure: a coupling gap of 0.400 at 1.3B parameters, the worst oddness defect in the set, and a probe accuracy far below its neighbors by size. Against log-parameters the correlation for the linear-probe gap is −0.276 with Phi-1.5 included and −0.927 without it. The point is not dropped, and the defensible reading is the weaker one: the slack decreases with capability *within a training regime*, and Phi-1.5, trained largely on synthetic textbook data, shows that the regime matters at least as much as the parameter count. A single line through five points is not what the data supports.

20.5 Pythia-410m occupies a corner

Pythia-410m has a decisive fraction of exactly zero. On no path does its confidence cross the threshold, so it never commits, so the coverage hypothesis fails, so the trilemma does not apply to it at all. Its coupling gap is not large; it is undefined.

This is the corner structure of Chapter 6 appearing in data, and it was not designed for. A model that never commits cannot be caught in the coupling, because the theorem needs a witness on each side to run the intermediate value argument. Chapter 10 calls this the abstention corner and prices it in

refused queries. Pythia-410m occupies that corner not as a policy but as a consequence of being unable to do the task.

The reading is that capability is what evicts a system from the safe corner. A model too weak to commit is trivially safe from the trilemma and useless; the models that commit on 93% and 99% of paths are the ones the theorem describes. Whatever is unsatisfying about that is a fact about the constraint, not about the measurement.

20.6 What the other two families inherit

The analytic-geometric family is validated separately, by the defense experiments: classifier gate maps for two deployed guard models, threshold sweeps, and the pipeline degradation runs. Those measure the same kind of quantity, a slack against a hypothesis, in the setting of Chapter 8.

The combinatorial family is the one this chapter does not measure, and the reason is that universality is a structural property rather than a magnitude: there is no number to check. Its approximate form is a different matter. The corpus defines `ApproxICLUniversal` in `CCH_LLM_Direct`, requiring only that every target behavior be realized to within a tolerance by some prompt, and `LLM_approx_trilemma` shows the contradiction fires as soon as the control margin exceeds the sum of that tolerance and twice the introspection error. That threshold is measurable in principle by prompt fitting, it is not measured here, and Chapter 16 lists it as the most valuable single experiment this thesis does not contain.

20.7 Threats to validity

Five models is a small sample, and three of the five come from two families. Most of the clean trend is carried by the two Qwen checkpoints, which share a training pipeline; the claim that capability rather than pipeline is doing the work would be better supported by a ladder within a single family spanning more scales.

The coupling gap is measured in path parameter, not in a natural unit of the model's output space, so the values are comparable across models on these paths and are not absolute quantities. The claim is about the direction of the trend and not about the numbers themselves.

The statement pairs are drawn from a template set, and template artifacts could in principle produce a spurious trend if larger models are more sensitive to templates. The endogenous measurement mitigates the probe-quality confound but not this one.

Finally, and most importantly, the trend is an association across five checkpoints, not a controlled manipulation. Nothing here establishes that increasing capability *causes* the slack to fall. The prediction the thesis makes is falsifiable and was not falsified by these five models, which is a weaker and more accurate statement than the claim that it was confirmed.

21 Part IV: Assessment

22 Limitations

This chapter states what is wrong with the thesis, or most likely to be. It is organized from the assumptions whose failure would cost the most to the ones whose failure would cost the least.

22.1 The unification claim is not fully supported

The thesis statement says the impossibility results are instances of two schemas. Two of the four case studies are not.

Chapter 13 reaches its main result by a counting argument: two reporters agreeing on a checkable subset may disagree off it, and nothing self-referential is required. Chapter 14 reaches its main result by invariance: a detector constant along paraphrase orbits cannot track a mark that varies along them, and again no diagonal appears. Both chapters contain diagonal corollaries, and both say plainly that the corollary is not what does the work.

The accurate version of the thesis statement is therefore weaker than the headline: the two schemas generate a large family of these results, they generate every result in Part II, and they do not generate everything in the neighborhood. I have preferred to report the boundary rather than to define the family so as to exclude the counterexamples, which would make the claim true and empty.

22.2 Continuity and connectedness of prompt space

The analytic engine requires a connected domain and continuous maps on it. Prompt space is a set of finite token sequences, which is discrete, and no amount of topology-talk changes that.

Three responses appear in the thesis, and none is fully satisfying. Chapter 9's discrete bridge proves the arithmetic core over the integers with no analysis at all, which shows the conclusion does not depend on the continuum, but at the cost of assuming a discrete sign change that is itself a hypothesis about the model. `MoF_ContinuousRelaxation` uses a Tietze extension to carry discrete data into the continuous theory, which is legitimate and does assume the extension is the right idealization. And embeddings are genuinely contin-

uous, so a model factoring through an embedding inherits continuity on that factor.

An examiner who rejects all three is left with theorems about an idealization. I think the discrete bridge answers the objection, and I record that it is the assumption most often challenged and the one I would attack first.

22.3 Pointwise calibration is stronger than the literature's

The trilemma's calibration hypothesis is pointwise and exact, and the calibration literature's is statistical and averaged. The trilemma is therefore not a result about reliability diagrams, and the temptation to present it as one should be resisted. Its force in the statistical setting comes only through the approximate results of Chapter 9, and how much force survives that translation is a question this thesis does not settle.

22.4 The empirical claim rests on five checkpoints

Chapter 15's threats to validity are not repeated here, but the headline is that five models, dominated by two families, with an outlier that breaks the trend, is thin evidence for a claim about capability in general. The trend is consistent with the thesis and does not establish it.

22.5 The most valuable missing experiment

The combinatorial family is unmeasured. `ApproxICLUniversal` gives the tolerance that a real system would have to meet, and `LLM_approx_trilemma` gives the threshold at which the contradiction fires, but no measurement of that tolerance appears in this thesis.

The experiment is well-defined: fix a query set and a verdict class, fit system prompts to targets drawn from that class, and record how closely the fitted prompt drives the model to the target. Run it on the ladder of Chapter 15 and compare against the introspection error. If the tolerance descends across the ladder and crosses the threshold, the diagonal family gains the same applicability argument the calibration family now has. If it plateaus, finite context is a real ceiling and the diagonal results apply to no system that can be built, which would be a more interesting finding than the one I expect.

I did not run it, and its absence is the largest gap in Part III.

22.6 Deployment claims are not theorems

Three propositions in Chapter 10 concern cost rather than mathematics: the effect of abstention on the surviving vertices, the default corner by archetype, and the traffic-dependence of felt cost. Each is marked "Not formalized." in its own text, and the build gate requires that marking rather than permitting a silent gap. They are engineering arguments and should be read as such.

22.7 What would falsify the thesis

Stating this is the point of the chapter. The unification claim fails if a central impossibility in this area is shown to require an essentially different mechanism; Chapters 13 and 14 are partial instances of exactly that, which is why the claim is stated in the restricted form it is. The applicability claim fails if the coupling slack is stable or increasing across a properly controlled capability ladder. The verification claim fails if the corpus does not build, which is the one component a reader can check in an afternoon.

23 Conclusion

The thesis set out to show that the impossibility results constraining trustworthy machine learning are instances of two schemas, that the claim can be machine-checked, and that the constraint is not an artifact of idealization.

On the first, the result is a qualified yes. The diagonal schema generates the classical impossibilities and the verdict trilemmata, the value-crossing schema generates the continuous and geometric results, and Part II derives them from the two lemmas rather than by analogy. Two of the four case studies fall outside both schemas, and Chapter 16 states the restriction that leaves.

On the second, the answer is unqualified, and it is the part of this work with the longest useful life. The corpus is 738 declarations with no `sorry` and no axioms beyond Mathlib's three, and the document that cites it will not build if a citation dangles or a numbered claim goes unbacked. A reader who doubts any theorem here can rebuild it.

On the third, the evidence points the right way without settling the question. The coupling slack falls from 0.225 to 0.075 across a capability ladder and the coverage hypothesis rises from 43% to 99% satisfaction, so the theorems describe capable models better than weak ones. Pythia-410m, which never commits, escapes the trilemma by failing its hypotheses, which is the abstention corner arrived at by incapacity rather than by design.

23.1 The claim worth carrying forward

If one sentence survives from this thesis, it should be that these limits tighten with capability rather than receding.

The intuition the field works from is the opposite. Hallucination, injection susceptibility, and unfaithful reasoning are treated as defects of immature systems, to be engineered away as models improve. For the properties studied here that intuition is backwards. The hypotheses that make the impossibility theorems apply — committing to answers on both sides of a boundary, coupling confidence to correctness, realizing a wide range of behaviors on demand — are the same properties that make a model useful. A system escapes the theorems by failing to have them.

That is not a counsel of despair, and Chapter 10 is the constructive half: every trilemma is escaped by giving up one vertex, the three corners have

different costs, and the costs are legible enough to choose between. What the theorems remove is the option of not choosing.

23.2 Future work

Three directions, in order of what I would do first.

The measurement of Chapter 16, on approximate in-context universality, would extend the applicability argument from one hypothesis family to two, and the apparatus for it already exists in the corpus and the harness.

A capability ladder within a single training pipeline, spanning more scales than five checkpoints across four families, would convert Chapter 15's association into something closer to a controlled result. Phi-1.5's deviation suggests that training data composition is a second factor worth isolating rather than noise to be averaged over.

Finally, the corpus is a starting point rather than a finished artifact. The schemas are stated generally enough that a new impossibility claim in this area can be tested against them cheaply: instantiate, and see whether the instance proof goes through. A claim that resists instantiation is either wrong or genuinely new, and telling those apart quickly is worth more than any single theorem here.

24 Exercises

The exercises are collected here rather than distributed through the chapters, so that the argument runs uninterrupted. They are grouped by the chapter they belong to and numbered accordingly. Those marked (Lean) ask for a machine-checked term; those marked (open) have no single answer.

24.1 Preliminaries

0.1 Define `c0_triple` in both styles, prove `c0_triple 4 = 12` by `rfl`, and say why `rfl` suffices. *0.2* Prove `p ∧ q → p` and `p → q → p ∧ q` in term mode; say which packs a structure and which projects one. *0.3* Using only `congrFun`, derive `f 0 = g 0` from `h : f = g`, and name the principle needed for the converse. *0.4* Prove `c0_pred 5 = 4` by `rfl`, and explain why `∀ n, c0_pred (n+1) = n` needs a genuine proof though each instance is `rfl`. *0.5* Prove `∀ b : Bool, b && b = b` by `decide`, and say which hypothesis of Definition 0.14 fails for `∀ n : Nat, n * 1 = n`. *0.6* Re-derive Example 0.25 by flipping the diagonal, then say why counting misleads for infinite `A` and what the diagonal supplies instead. *0.7* Give a fixed-point-free `t : Bool → Bool`, show no `t : Unit → Unit` is one, and say what that means for a system whose only outcome is "accept." *0.8* Find the fixed point of `t y = 1 - y` and relate it to the analytic threshold. *0.9* Prove the section form of the engine, identifying the step that shortens when a section replaces a surjection. *0.10* Give a counterexample to Theorem 0.36 on a two-point discrete space, and conclude why connectedness is not optional. *0.11* Predict, then check, `#print axioms` on a Boolean and an analytic result, and say what the difference tells you. *0.12* Argue that two indices naming the twisted diagonal are both boundary objects and that nothing selects one. *0.13* Prove a composite of injections is injective, and say why injectivity is the retraction-side property and surjectivity the section-side one. *0.14* (open) Without symbols, explain why the intermediate value theorem and Cantor's diagonal are the same argument, naming each engine's domain hypothesis, outcome-map property, and boundary object.

24.2 The Diagonal

1.1 Verify Example 1.3 by flipping `f a a`, and say why finiteness was essential to the counting step but idle in the diagonal step. *1.2* (Lean) Prove Theorem 1.8 from scratch, and say why the section form needs no `match`. *1.3* Show the converse of Corollary 1.10 fails, and state what the corollary does not claim about `f`. *1.4* (Lean) Give a fixed-point-free endomap of a three-element type, modeling `c1_optFlip_fpf`, and say what `c1_no_fpf_unit` means for a system that can only accept. *1.5* Prove Cantor's usual form by identifying `Set A` with `A → Bool`, pointing to the line corresponding to `bool_not_fpf`. *1.6* (Gödel.) Show a sentence asserting its own unprovability is a fixed-point-free instance, and say which hypothesis corresponds to representability and which to completeness. *1.7* (Tarski.) Repeat for truth, saying why Tarski's conclusion is about definability and Gödel's about provability. *1.8* (Turing.) Cast halting as an instance and explain why the argument refutes `H` rather than the diagonal program. *1.9* Show two indices naming the diagonal behavior are both liars, relating this to non-uniqueness of the boundary question. *1.10* (Lean) Prove the one-sided refinement via `c1_lawvere_diagonal` and recover `liar_query`. *1.11* In one paragraph without symbols, say why counting is the wrong intuition for the infinite case. *1.12* Give a `Y` and `t` with exactly one fixed point and connect the result to Kleene's theorem read forwards. *1.13* (Harder.) Formulate the partial version, and say which classical instance arises when `t` only toggles `none`. *1.14* (Lean) Decide whether the flip swapping `none` with `some false` and fixing `some true` is fixed-point-free, and use the answer to explain Rice's nontriviality. *1.15* Prove every section gives point-surjectivity and say why the converse needs choice. *1.16* For `Bool`, `Unit`, a two-point verdict type, `Option Bool`, and a one-point space, decide which admit a fixed-point-free endomap, and state the principle. *1.17* (open) Speculate on a system satisfying both universality and connectedness, and what its boundary object would be. *1.18* (open) Phrase a safety desideratum as a universality claim and identify the sentence someone must refute to escape it.

24.3 The Classical Limits

2.1 Prove `c2_no_universal_bool` by a direct `match`, checking both proofs give the same witness. *2.2* Show no `t : Unit → Unit` has `t u ≠ u`, and say why this is the degenerate row of the table. *2.3* Write "no surjection `A → Set A`" using indicators, and say where the `Set`-form proof uses decidability the `Bool` form avoids. *2.4* Derive `False` from `hmem`, concluding `c2_russell` has an empty hypothesis, and relate this to the repairs of comprehension. *2.5* Prove `¬ (P ↔ ¬ P)` without classical axioms and say why the `Bool` version is compiled instead. *2.6* Reprove `c2_godel_semantic` in tactic mode and confirm it depends on no axioms. *2.7* Say which property of ordinary proof each derivability condition internalizes, and why the third makes the second theorem harder. *2.8* State Löb's theorem as a fixed point, verify Gödel II is the case `φ =`

`0 = 1`, and say why `Con(T)` is the antecedent Löb refuses. *2.9* Give the liar as the diagonal behavior in Tarski's theorem, and show a definable truth predicate would fall to `c2_godel_semantic`. *2.10* Explain, in terms of recursive enumerability, why `Prov` is definable and `True` is not. *2.11* (Harder.) Prove a two-input Turing theorem and show `a = b` recovers `c2_turing_halting`. *2.12* Say why the contrarian program must be partial and why the argument fails if deciders may abstain. *2.13* Show a nontrivial `P : Bool → Bool` is the identity or negation, and sketch the reduction of `Foundation.rice_general`. *2.14* (Harder.) Write both cases of the reduction in Theorem 2.22 and say where classical logic enters. *2.15* Fill in the table for Grelling–Nelson and for the `Y` combinator, saying whether each fixed point is a contradiction or a construction. *2.16* (open) Say what distinguishes the destructive uses of the diagonal from the constructive `Y` combinator. *2.17* (open) Force one AI-safety property into the recipe, naming `A`, `Y`, `t`, and the universality hypothesis, and say what a working system gives up. *2.18* (open) Formulate a quantitative question Rice does not answer and say what structure would make it precise. *2.19* State the section form categorically and identify, for Cantor and Turing, what plays `Y`, `t`, and `s`. *2.20* (Harder.) Write the map whose fixed point Löb takes and show its fixed point at `φ = 0 = 1` is Gödel II's sentence. *2.21* (open) Place three properties of a language model on the Rice–Shapiro spectrum and say what changes when each is weakened to approximate.

24.4 Three Trilemmata

3.1 Verify the three trilemma types differ only in the bound domain variable, and say what `c3_trilemmata_unified` adds. *3.2* Write out in English the query `c3_hallucination_liar` produces, and say which desideratum each escape drops. *3.3* Show in Lean that the hallucination and defense liars are the same witness. *3.4* Show the constant model is faithful and calibrated but not covering, and exhibit a pattern it fails to name. *3.5* Say why reflectivity restates the threat model for prompt injection, then give a restriction under which it fails. *3.6* (Harder.) Argue that a model prompted about its own probe makes self-encoding real and a passive activation space does not; say which a probing experiment deploys over. *3.7* Prove `c3_lawvere_point` from `lawvere` and say what extra information it exposes. *3.8* Instantiate `c3_cch_master` at `Y = Bool` with the flip and the diagonal, concluding the hallucination trilemma is the `UC` corner. *3.9* Exhibit a system for each achievable pair, showing no edge of the triangle is empty. *3.10* Prove the three corners reduce to `c3_cch_master` and state `c3_cch_at_most_two`. *3.11* For each trilemma name the two horns fixing the flip and the source of universality, and rank the sources from free to strong. *3.12* Say what quantitative question the diagonal cannot answer and why Chapter 5 needs the analytic engine. *3.13* (Harder, Lean.) Define `C3_Faithful` and prove a reflective `v` cannot satisfy it; say which lemma the proof reduces to. *3.14* Restate `faithful_iff_boundary_free` in diagonal language and say why reflectivity removes the escape. *3.15* (open) Sketch a single theo-

rem covering both the real-complement and the connectedness relaxations. *3.16* (open) Without symbols, explain why "no total exact self-applicable truth predicate" is the same statement as the three trilemmata. *3.17* Restate `c3_hallucination_trilemma` with a section, and say why for auditing the section form is the more honest premise. *3.18* Carry out Remark 3.12 in Lean, discharging transparency by `rfl`. *3.19* For each trilemma name the premise you would attack in a real deployment and its engineering cost. *3.20* (open) Design an experiment for or against a representation space encoding patterns over its own states. *3.21* Redo the micro-instance on four elements and identify the liar row. *3.22* Prove the flip is fixed-point-free and say what the complement's exceptional point at one half corresponds to. *3.23* Sketch a `CCHStructure` witnessing each achievable pair. *3.24* Say which property a wrapper refusing self-referential prompts gives up, and prove in Lean it cannot satisfy completeness there. *3.25* Write the Tarski correspondence term by term, and say which hypothesis does the work "sufficiently expressive language" does. *3.26* Say what the topological proof knows about the boundary object that the diagonal does not, using the ambiguity tube and the reflectivity premise. *3.27* (Harder.) Prove one theorem returning both `False` and the witness, and say why it summarizes all three trilemmata.

24.5 From Logic to Analysis

4.1 Prove a two-point discrete space is disconnected and say why the analytic engine cannot run on `Bool`. *4.2* Show the continuous image of a connected space is connected, and relate this to Lemma 4.15. *4.3* Fill in the omitted lines of Proposition 4.6 at the supremum. *4.4* State and prove the two-function IVT, deriving Theorem 4.7 as the constant case. *4.5* Say which direction of each calibration biconditional is used forward and which backward, and show one-way calibration does not pin the truth-distance. *4.6* Show Theorem 4.17 fails if faithfulness is weakened to `≤ 0`, and locate the coupled point of your weakened model. *4.7* Reprove `c4_boundary_question_impossible` at an arbitrary threshold and say why the threshold is a convention. *4.8* Rewrite `c4_discrete_sign_change` to return the least crossing index, and say why the continuous IVT has no canonical choice. *4.9* Prove the truth boundary is closed using only that singletons are closed. *4.10* Show an odd field on a connected space with a free involution has two zeros unless identically zero on an orbit, and say what the second corresponds to. *4.11* Prove the approximate-oddness bound and show it is tight. *4.12* Use the diagonal with `y ↦ 1 - y` to produce `s a a = 1/2`, identifying which step replaces the IVT. *4.13* Say why the complement's single fixed point makes Lawvere yield a contradiction in the Boolean case and a point in the real case. *4.14* Give a system satisfying universality but not connectedness and one satisfying connectedness but not universality. *4.15* (Harder.) Give a hypothesis on a finite question space, weaker than connectedness, still forcing a boundary question. *4.16* (Harder.) If the confidence is continuous but the answer map is not, say which of Lemmas 4.14, 4.15

and Theorem 4.16 survive. *4.17* (open) Conjecture how the tube radius scales with the Lipschitz constant and the confidence margin. *4.18* (open) Say what a system satisfying both the diagonal and antipodal hypotheses would need. *4.19* Prove `Q × Q` connected by the slice argument and state the two-turn coupling theorem, saying where the second turn entered. *4.20* Prove the stochastic dichotomy and say why it alone concludes with confident error rather than uncertainty. *4.21* (Harder.) Design a reporting policy making the exclusive guarantee's silence visible, and argue the flagging band cannot be empty.

24.6 The Geometry of Attacks

5.1 Verify the Gaussian bump's basin is a ball of radius `√(2 ln(M/τ))` and confirm the three soft predictions. *5.2* Exhibit a continuous non-Lipschitz `f` whose guaranteed ball is arbitrarily small. *5.3* Construct an `f` on `[0,1] ∪ [2,3]` with empty boundary, and say why connectedness is exactly what `no_gap_theorem` needs. *5.4* Verify the robustness ball touches the boundary hyperplane at exactly one point and compute it. *5.5* Show a model safe at two inputs and grossly unsafe at their midpoint needs `L ≥ 2M`, and read this as a no-free-lunch statement for smoothing. *5.6* Derive `μ(band) ≥ ε/(2L)` and generalize to `ℝ^n`. *5.7* Show the surviving steep region has measure δ_0 and interpret `(c - s)` as the defender's shortfall. *5.8* Prove the shallow-boundary escape and its converse. *5.9* Compute `N^d` and `r^d` for `d = 10` and `30` and identify the critical dimension for a budget of `10^6`. *5.10* Show a nonvanishing derivative gives an isolated boundary, then construct a degenerate case whose level set contains an interval. *5.11* Prove gradient ascent halts only at critical points and that the gradient step maximizes immediate gain. *5.12* Trace the gradient chain on an affine surface and reconcile it with the corner example. *5.13* For δ `= 0.2`, say for which margins transfer is guaranteed and construct a failure below δ. *5.14* Compare `1/p` for `p = 10^-6` with a gradient attack's step count and say why gradients dominate when the basin is small. *5.15* Assemble the master asymmetry and say which quantity's independence from `d` is its source. *5.16* (Harder.) Show that at `K*` the band has width at least `G·`δ, and argue no `K` drives both cone measure and band volume to zero when `G > L`. *5.17* (open) Propose a common currency for both budgets and say whether the asymmetry survives the change of units. *5.18* (open) Say what plays the role of `L`, the ball, and the coarea band in a discrete token space, and which conclusions survive. *5.19* Contrast the Markov upper bound with the coarea lower bound on the Gaussian annulus, and say why the gradient attack ignores the former. *5.20* Show a defense with `n` slots facing `n+1` attacks leaves one uncorrected, prove the discrete crossing lemma, and say why it and the diagonal are two proofs of one fact.

24.7 Approximate Bridges

6.1 Write the contrapositive of each clause of Definition 6.1 and confirm Proposition 6.2, noting where linearity is used. *6.2* Verify the $\varepsilon = 0$ clauses are the forward implications of strict calibration and say which direction the IVT never uses. *6.3* Say which endpoint of $[-\varepsilon, 0)$ is attainable, which hypothesis supplies each, and why swapping the strictness breaks Theorem 6.9. *6.4* With $c = 0.5$ and $\varepsilon = 0.2$, compute the guaranteed margin and the transition band, and say what a deployment must monitor. *6.5* Prove the tightening principle from `c6_tightening` and show the margin bound is attained. *6.6* Construct a scenario improving calibration at fixed faithfulness and identify where Remark 6.13 says it must stall. *6.7* Prove Proposition 6.15 and strengthen it to the closed band. *6.8* Say which hypothesis of Theorem 6.16 gives which side of the bound, and re-elaborate `c6_band` with the roles exchanged. *6.9* Re-elaborate `c6_band` with a separate confidence margin, obtaining the integer form of Theorem 6.21. *6.10* Rescale confidence by `100` and confirm γ scales while ε does not; say what this means for reporting a single ε. *6.11* Prove Theorem 6.24 as an instantiation of Chapter 1's engine, identifying the target fed to universality. *6.12* Trace the triangle inequality to see which legs contribute the coefficient two, and redo the count for a K-Lipschitz controller.

24.8 What It Means for AI Safety

7.1 Explain via `lawvere` and `bool_not_fpf` why adding `abstain` converts the diagonal into a consistent value, and identify it. *7.2* For `Cerr = 1000 · Cref`, find the abstention rate at which the two corner costs are equal, and redo for `Cerr = 2 · Cref`. *7.3* Prove the defense analogue of Proposition 7.8 and say why it makes decoupling attractive to one team and dangerous to another. *7.4* If ρ rises from `0.002` to `0.02` at fixed ε, give the factor change and design a monitor that would have caught it. *7.5* (Design.) Specify a three-region blend for clinical decision support and state the quantity you must measure before placing the boundaries. *7.6* (Design.) Repeat for platform-scale moderation, justifying the smaller reject region and identifying which consumer forces calibration. *7.7* Say why a discontinuous blocklist escapes the Defense Trilemma, what it pays, and whether it is abstention in disguise. *7.8* (open) Describe a deployment where decoupling is correct, and state the condition on downstream consumers making it safe. *7.9* Construct two traffic distributions whose confident-error rates differ by a hundredfold for one model, and say why no benchmark decides between them. *7.10* For a nonzero linear probe, write the distance to the boundary explicitly and say why the histogram is cheap at scale. *7.11* (Harder.) Sketch how hitting time bounds an attacker's steps and why lowering the Lipschitz constant raises that cost. *7.12* (open) Name the premise you find least defensible in your application and say what evidence would settle it. *7.13* (open) Propose a domain where the cost models fail and identify the assumption it violates.

24.9 J-Space

8.1 Prove that post-composing a decoder is also a no-escape result and say what it models. *8.2* Instantiate `jspace_factoring` at `J := Unit` and say why the most degenerate probe is still refuted. *8.3* Show two queries realizing the diagonal pattern are both coupled and nothing selects one. *8.4* Prove `c8_steering_no_escape` by absorbing `steer` into the encoder, and conclude steering is a choice of encoder. *8.5* Explain how `bool_not_fpf` drives both the liar and the discrete failure, and where the real complement enters instead. *8.6* Add a sparsity budget to `c8_Coord` and show the theorem still applies; say what this means for sparsity as a safety measure. *8.7* Give a score on J-space failing one hypothesis of Theorem 8.17 and say what breaking it costs. *8.8* Say why convexity permits a straight segment, and why a finite set of observed activations does not. *8.9* If the transport map were curved, say which engine applies unchanged. *8.10* Write the four-stage pipeline theorem and state the general principle your proof illustrates. *8.11* Give the sense in which `c8_jlens_impossible` and `no_reflective_verdict` are one theorem, and why a `rfl` proof is the honest signal. *8.12* (Design.) Argue whether a target coupled-query rate is reachable by slack-removal alone, and what deployment information you would need. *8.13* (open) Speculate on a representation connected but not self-applicable. *8.14* (Harder.) For the affine model, identify the self-applicable map, its linearization, and the liar activation as the fixed point. *8.15* For a three-valued readout with `unknown` fixed, show the diagonal produces no contradiction and say what the probe gave up. *8.16* (open) Design an experiment distinguishing a structural floor, whose residual set churns, from a resource-limited error, whose residual set is stable. *8.17* (open) Construct the most plausible representation meeting one of the three escape demands, and check which of the others it fails.

24.10 Chain-of-Thought Monitoring

9.1 Show that if `T` is `Unit` the monitor is constant and errs on any pair differing in `harm`. *9.2* State and prove the contextual analogue of Theorem 9.7, and say where it breaks. *9.3* Prove `c9_collision_forces_error` in term mode by case-splitting on `harm q₁`. *9.4* Weaken selection to a subset and say what the queries left out mean for a deployment. *9.5* (Design.) With one monitor trained against and one held out, say what each flag rate licenses and construct a case where the divergence is informative. *9.6* Give hypotheses under which traces reachable by bounded paraphrase are connected, and say why unbounded paraphrase strengthens the argument. *9.7* Combine Theorem 9.10 with Chapter 8's factoring for a monitor reading trace and activations; does reading both help?

24.11 Latent Knowledge

10.1 Prove `c10_flipOff` is an involution on reporters. *10.2* Given two unchecked questions, construct four pairwise-distinct reporters agreeing on the checkable set, and state the counting law for `n`. *10.3* Formalize a length penalty and give the extra hypothesis pinning the direct translator; say why the data does not supply it. *10.4* Prove the converse of Theorem 10.9, making the dichotomy an equivalence. *10.5* Combine `c10_reporter_factoring` with Chapter 9's trace factoring into one theorem. *10.6* Give hypotheses under which the set of reporters agreeing within ε is connected, and say why that is bad news for a selection procedure. *10.7* (Design.) Give the measurement distinguishing a technique that enlarged the checkable set from one that merely enlarged where the reporters agree.

24.12 Watermarking

11.1 For each pair of the three properties, exhibit a detector satisfying that pair under a stripping paraphrase. *11.2* Prove `c11_detection_trilemma` in term mode using `Bool.noConfusion`. *11.3* Generalize Corollary 11.10 to orbits under iterated paraphrase. *11.4* Formalize a bounded paraphrase with an edit budget and say what replaces the stripping hypothesis. *11.5* Add a utility function to Definition 11.3 and compare the resulting four-property theorem to the Defense Trilemma. *11.6* Derive the edit-count bound `(s x − τ)/L`, stating the hypotheses on the edit metric and which fails for very short texts. *11.7* (Design.) Choose a corner for a moderation platform, name the population it fails, and give the measurement of that failure rate.

25 Appendices

These appendices are the reference apparatus. Appendix A indexes the formal corpus file by file; Appendix B is the notation glossary; Appendix C sets the two engines side by side; Appendix D is the annotated bibliography; Appendix E says how to build and audit everything.

25.1 The Formal Corpus

The development is large but shallow in a specific sense. Almost everything reduces to the three-line proof of `lawvere` in Chapter 1, or to a single application of a value-crossing lemma.

A word on axioms before the tour, because the book makes a point of it. Lean's kernel admits exactly three standard axioms beyond its type theory: `propext` (propositional extensionality), `Classical.choice` (the axiom of choice), and `Quot.sound` (the computation rule for quotients).

The split between axiom-free and classically clean is not a defect and not an accident. It is the formal shadow of the book's two-engine thesis.

How to use the index that follows. Each library gets a paragraph on what it proves as a whole, then a file-by-file list.

Foundation: the core library (F_01–F_14)

`Foundation` is the trunk. It proves Lawvere's fixed-point theorem from nothing, extracts the impossibility schema, and instantiates it across the classical paradoxes and the AI-safety readings.

`F_01` (`F_01_LawvereCore`). The heart of everything. This file states and proves the fixed-point theorem in both its surjective and its section form, then packages the contrapositive. The boolean-diagonal results here are axiom-free.

- `lawvere` : a universal `f : A → A → Y` forces every `t : Y → Y` to have a fixed point.
- `lawvere_diagonal` : the same, but naming the diagonal behavior explicitly so the witness is visible.

- `no_surjection_of_no_fixed_point` : if `t` has no fixed point, no `f` is point-surjective.
- `fixed_point_free_blocks_universality` : the contrapositive stated as a block on universality.
- `lawvere_section` : the section form Lawvere actually published, using `s : (A → Y) → A` with `f (s g) = g`...
- `surjective_of_section` : a section gives point-surjectivity, tying the two forms together.
- `fixedPoint_spec` : the constructed fixed point satisfies its defining equation.
- `no_universal_with_FPF` : no universal system when the output carries a fixed-point-free map, stated for a given `t`.
- `no_universal_of_FPF_type` : the same phrased as a property of the output type.
- `lawvere_universal_fixed_point` : universality and a chosen `t` together produce the fixed point, the form reused downstream.

`F_02` (`F_02_RiceTheorem`). Rice's theorem as a diagonal. Any external decision procedure that claims to read a nontrivial semantic property of behaviors mismatches some behavior. Axiom-free at the boolean core.

- `rice_diagonal` : an external classifier disagrees with the true property on a diagonal input.
- `rice_general` : the general statement that no total classifier decides a nontrivial behavioral property.
- `boolean_diagonal_mismatch` : the boolean instance where the mismatch is a single flipped bit.
- `no_automated_self_test` : no system can run a complete correct self-test of its own behavior.

`F_03` (`F_03_VerificationLimit`). The verifier version. A universal system cannot have a fixed-point-free output map, so no complete self-verifier exists over its own outputs.

- `no_universal_with_fpf_output` : universality is incompatible with any fixed-point-free output transformation.
- `no_universal_AI_output_type` : the same for an abstract prompt-to-output system.

- `no_universal_bool_LLM`: specialized to boolean verdicts, no universal boolean-output model.
- `no_universal_nat_LLM`: specialized to natural-number outputs via the successor map, which has no fixed point.

`F_04` (`F_04_CalibrationUnified`). The confidence reading, still discrete. Here the output is a probability and the flip is "reflect across one half," whose only fixed point is the value one half. This is where calibration first meets the diagonal, and it is the bridge to the analytic hallucination results.

- `half_complement_fixed_point` : one half is the unique fixed point of the probability complement.
- `complement_no_fp_off_half` : off one half, the complement moves you.
- `calibration_diagonal_hits_half` : the diagonal question drives a calibrated model's confidence to exactly one half.
- `calibration_no_strict_avoidance` : a calibrated universal model cannot strictly avoid the half-confidence boundary.
- `hallucination_via_lawvere`: the hallucination phenomenon derived directly from `lawvere`.

`F_05` (`F_05_DeceptionTheorem`). Alignment as a fixed-point obstruction. If an "aligned" transformation of behavior has no fixed point, no universal system can be uniformly aligned.

- `deception_diagonal`: the diagonal behavior a safety filter cannot align with.
- `no_universal_aligned_system`: no universal system is aligned under a fixed-point-free alignment map.
- `no_universal_safe_LLM`: the LLM reading, no universal model is uniformly safe.
- `two_diagonals` : two distinct diagonal constructions coincide as witnesses, a lemma the corner results reuse.

`F_06` (`F_06_OversightHierarchy`). Oversight does not escape the diagonal. Stacking a watcher on a watcher inherits universality and inherits the diagonal, at every finite level.

- `pairwise_oversight_inherits` : a two-party oversight pair inherits the diagonal.
- `chain_inherits`: a finite chain of overseers inherits it.

- `oversight_no_escape` : no oversight layer removes the fixed point.
- `reflective_tower` : an iterated tower of reflection stays universal.
- `no_level_escapes` : no level of the tower escapes the impossibility.

`F_07` (`F_07_CompositionFailure`). Post-processing does not help. Composing any map after a universal system leaves a universal system, so wrappers and filters cannot break self-reference.

- `composition_universal` : composition with a surjection preserves universality.
- `composition_inherits_diagonal` : the composed system inherits the diagonal.
- `post_processing_no_escape` : no post-processor removes the fixed point.
- `two_stage_inherits_diagonal` : a two-stage pipeline still has the diagonal.
- `two_stage_no_escape` : the two-stage pipeline does not escape.
- `iterCompose_universal` : iterated composition preserves universality.
- `iterCompose_inherits_diagonal` : iterated composition inherits the diagonal.
- `iterCompose_no_escape` : no number of iterations escapes.

`F_08` (`F_08_SpecificationBound`). The counting-with-teeth file. Cantor's power-set gap in its Lean form: the behaviors of a system are strictly more numerous than its indices, so some behavior is unspecifiable.

- `cantor_set` : no surjection from `A` onto `Set A`.
- `specification_bound` : the space of behaviors strictly exceeds any indexing.
- `behavior_unspecifiable` : some behavior has no specification.
- `powerset_strictly_larger` : the power set is strictly larger, stated directly.
- `bool_specification_bound` : the boolean-indicator form of the same bound.

`F_09` (`F_09_QuantitativeDegradation`). The first genuinely metric file, and the hinge between the two engines inside `Foundation`.

- `approx_lawvere` : an approximate fixed point exists within the flip's displacement.
- `quantitative_trilemma` : the three safety properties trade off with a measurable slack rather than failing all at once.
- `achievable_region_bound` : a bound on the region of jointly achievable property levels.
- `quantitative_impossibility` : impossibility restated with an explicit error term.
- `strict_limit_recovers_trilemma` : as the slack goes to zero the exact trilemma returns, so the discrete result is the limit of...

`F_10` (`F_10_MasterFoundation`, `F_10_FinalVerification`). The consolidation layer. A single structure bundles the foundational hypotheses and a master theorem derives every corner from it; the verification file runs the axiom audit.

- `master_foundation_theorem` : one theorem from which the foundational corners follow.
- `mirror_trilemma_instance` : the mirror (self-reference) trilemma as an instance.
- `rice_instance` : Rice recovered as an instance.
- `spec_bound_instance` : the specification bound recovered as an instance.
- `nat_succ_instance` : the natural-number successor instance.
- `unification_statement` : the statement that the instances are one theorem.

`F_11` (`F_11_HallucinationGodel`). The Gödel-style naming of the hallucination result. The liar query is named explicitly, and the trilemma is stated as an impossibility. This file ends with `#print axioms` on its headline results; the boolean core is axiom-free.

- `bool_not_fpf` : boolean negation has no fixed point, proved by `decide`.
- `tarski_liar` : the Tarski-style undefinability liar for a truth predicate.
- `hallucination_liar_query` : the exact query on which a reflective model must contradict itself.
- `hallucination_trilemma_godel` : the hallucination trilemma as a self-reference impossibility.

- `controllers_agree` : the boolean and propositional flips agree as controllers.

F_12 (`F_12_DefenseGodel`). The same move for defenses. A defense that judges prompts, including prompts about its own judgments, is a reflective verdict, so it is impossible, and the file shows the defense engine is the hallucination engine.

- `defense_liar_query` : the prompt injection no wrapper neutralizes, as a liar.
- `defense_trilemma_godel` : the defense trilemma in Gödel form.
- `defense_trilemma_lawvere` : the same via `lawvere` directly.
- `reflective_verdict_impossible` : no reflective verdict exists.
- `defense_is_hallucination_engine` : the two impossibilities share one proof.

F_13 (`F_13_UnifiedTrilemmata`). The unification. All three trilemmas, the hallucination, the defense, and the truth-boundary coupling, are folded into one statement about reflective verdicts. Axiom audit printed at the end.

- `no_reflective_verdict` : the one-line engine, no reflective verdict exists.
- `schema_boundary` : the schema that produces the boundary object.
- `coupling_trilemma_godel` : the truth-boundary coupling in Gödel form.
- `trilemmata_unified` : the three trilemmas as a single theorem.

F_14 (`F_14_JSpace`). The interpretability reading. A probe that reads a judgment off a model's own activations, over a space rich enough to encode any readout, is again a reflective verdict, so a complete self-reading probe is impossible. This is the formal core behind the J-space chapter.

- `jspace_readout_impossible` : no complete readout of a model's own judgments from its activations.
- `jspace_coupled_activation` : the activation on which the readout must fail.
- `jspace_is_the_same_engine` : the interpretability obstruction is `lawvere` again.

CCHProofs: the Control-Corner-Hallucination trilemma (CCH_01–CCH_Master)

`CCHProofs` develops the trilemma as a geometry of three properties: Universality, Control, and Transparency, of which any system can enjoy at most two. It restates Lawvere in its own vocabulary, proves Cantor and the fixed-point-free lemmas, then walks the three corners one at a time before assembling the master statement.

`CCH_01` (`CCH_01_Foundations`). Definitions and the dictionary between the property language and the fixed-point language.

- `nonTrivial_iff_no_fixed_point` : a controller is nontrivial exactly when it has no fixed point.
- `universal_iff_surjective` : universality is point-surjectivity.
- `transparent_eq_diag` : transparency is representability of the diagonal.
- `nonTrivial_apply`, `universal_exists` : the elimination forms of the two definitions.
- `isUniversal_iff`, `isControlled_iff`, `isTransparent_iff` : the structure-level restatements.
- `isTransparent_eq_diag` : transparency as diagonal equality at the structure level.

`CCH_02` (`CCH_02_Lawvere`). Lawvere inside the CCH vocabulary.

- `lawvere_fixed_point` : the fixed-point theorem, CCH form.
- `no_surjection_of_no_fixed_point` : Cantor's contrapositive.
- `universal_implies_fixed_point` : universality forces the fixed point.
- `fixed_point_free_rules_out_universal` : the block on universality.
- `lawvereFixedPoint_spec` : the fixed point meets its spec.
- `lawvere_via_section`, `surjective_of_section` : the section form and its bridge.
- `lawvere_universal_fixed_point` : the packaged form reused by the corners.

`CCH_03` (`CCH_03_Cantor`). Cantor, in surjection and power-set forms. Axiom-free.

- `bool_not_no_fixed_point` : boolean negation has no fixed point.

- `cantor_no_surjection` : no surjection onto the boolean behaviors.
- `cantor_no_surjection_set` : no surjection onto the power set.
- `no_universal_bool_system` : no universal boolean system.

`CCH_04` (`CCH_04_FixedPointFree`). A small library of fixed-point-free maps, the raw material for corners. Axiom-free.

- `bool_not_fixedPointFree` : boolean negation is fixed-point-free.
- `succ_fixedPointFree`, `int_succ_fixedPointFree` : successor on the naturals and on the integers.
- `no_surjection_of_fixedPointFree`, `fixedPointFree_blocks_universality` : the two block forms.
- `fixedPointFree_iff` : the characterization of fixed-point-freeness.
- `not_not_eq_id` : double negation is the identity, the reason a single flip and not a double one is needed.
- `empty_fixedPointFree`, `unit_no_fixedPointFree` : the endpoints, everything on the empty type is fixed-point-free and nothing on `Unit` is.
- `lawvere_section`, `no_section_of_fixedPointFree` : the section-form counterparts.

`CCH_05` (`CCH_05_Controllers`). The theory of controllers, the maps that play the role of `t`. A controller is nontrivial when it has no fixed point.

- `bool_not_nonTrivial`, `nat_succ_nonTrivial` : examples of nontrivial controllers.
- `id_not_nonTrivial` : the identity is never nontrivial.
- `nonTrivial_iff_no_fixed_point`, `nonTrivial_iff_no_fp_exists` : the two characterizations.
- `controlled_iff` : control expressed via the controller.
- `universal_excludes_nonTrivial_controller` : universality excludes a nontrivial controller.
- `universal_implies_no_nonTrivial_controller` : the same as an implication.
- `controlled_universal_nonTrivial_contradiction` : the three-way clash at the heart of the trilemma.
- `nonTrivial_of_image`, `nonTrivial_no_fixed`, `exists_fixed_not_nonTrivial` : transport and existence lemmas.

- `bivalent_nonTrivial`, `nonTrivial_bivalent_of_inhabited` : the bivalent-output cases.

`CCH_06` (`CCH_06_Transparency`). Transparency, the property that the diagonal is itself a named behavior.

- `transparent_iff_eq_diag` : transparency is diagonal equality.
- `diag_is_transparent` : the diagonal is always transparent for itself.
- `transparent_symm` : symmetry of the transparency relation.
- `universal_diag_in_image`, `transparent_universal_diag_representable` : a universal transparent system represents its own diagonal.
- `transparent_universal_self_fixed` : which forces a self-fixed point.
- `transparent_universal_blocks_nonTrivial` : and blocks any non-trivial controller.
- `trivial_transparent_construction` : a trivial witness showing the corner is inhabited when universality is dropped.

`CCH_07` (`CCH_07_CornerUC`). The Universal-plus-Controlled corner, which cannot also be Transparent.

- `corner_UC_not_T` : Universal and Controlled excludes Transparent.
- `corner_UC_impossible` : the corner as an impossibility.
- `corner_UC_strong` : a strengthened version.
- `universal_diagonal_hits_fixed_point` : the diagonal lands on the fixed point.
- `defense_trilemma_face` : this corner is the defense trilemma face.
- `corner_UC_exists_failure`, `corner_UC_prediction_fixed_point` : the explicit failure witness and the prediction fixed point.

`CCH_08` (`CCH_08_CornerUT`). The Universal-plus-Transparent corner, which cannot be Controlled.

- `corner_UT_not_C` : Universal and Transparent excludes Controlled.
- `corner_UT_impossible` : the corner as an impossibility.
- `hallucination_trilemma_face` : this corner is the hallucination trilemma face.
- `corner_UT_witness_identity`, `corner_UT_needs_universality` : the witness and the necessity of the universality hypothesis.

CCH_09 (CCH_09_CornerCT). The Controlled-plus-Transparent corner, which cannot be Universal. This is the Rice face.

- corner_CT_not_U : Controlled and Transparent excludes Universal.
- corner_CT_impossible : the corner as an impossibility.
- corner_CT_bool_specialization : the boolean specialization.
- rice_face : this corner is the Rice face.
- corner_CT_witness, corner_CT_not_U_of_fpf, corner_CT_bool_via_fpf : the witness and the fixed-point-free routes.

CCH_LLM (CCH_LLM_Direct). The trilemma spelled out for an in-context learner M : LLM Prompt Token, so the abstract corners become claims about a model.

- iclUniversal_iff : in-context universality is surjectivity of the learner.
- LLM_diagonal_fixed_point, LLM_no_fixed_point_free : the diagonal and its consequence for the model.
- LLM_trilemma : the trilemma for the model.
- LLM_corner_UC, LLM_corner_UT, LLM_corner_CT : the three corners, model form.
- LLM_safety_corner_UC, LLM_safety_self_misclassification : the safety readings, including that the model must misclassify itself somewhere.
- bool_not_controller : the boolean flip is a controller.
- LLM_approx_fixed_point, LLM_approx_trilemma : the approximate versions with a confidence output.

CCH_Master (CCH_MasterTrilemma, CCH_FinalVerification). The consolidated trilemma over a CCHStructure, and the axiom audit.

- cch_master_trilemma : at most two of Universal, Controlled, Transparent.
- cch_corner_UC, cch_corner_UT, cch_corner_CT : the three corners at the structure level.
- cch_at_most_two : the counting statement, no more than two properties hold.

HallucinationProofs: the truth-boundary theory (HoF_01–HoF_Master)

`HallucinationProofs` is where the second engine takes over. Instead of a diagonal over a self-naming domain, it works with a signed truth distance δ, a confidence `conf`, and a truth threshold, and it uses value-crossing, the intermediate value theorem and its topological cousin the Borsuk-Ulam theorem, to force the existence of a boundary point where the model is confidently on the fence.

`HoF_01` (`HoF_01_Foundations`). The truth set and its neighbors, defined from the signed distance δ and the model's confidence.

- `mem_truthSet`, `mem_falseSet`, `mem_truthBoundary`, `mem_strictTruth`, `mem_highConfRegion` : membership unfoldings for the five basic regions.
- `truthSet_isClosed`, `falseSet_isOpen`, `truthBoundary_isClosed`, `strictTruth_isOpen`, `highConfRegion_isClosed` : the topology of each region when δ and `conf` are continuous.
- `truth_partition`, `truthSet_union_falseSet`, `truthSet_falseSet_disjoint` : the space splits cleanly into truth and falsehood.
- `strictTruth_subset_truthSet`, `truthBoundary_subset_truthSet`, `truthSet_eq_strictTruth_union_boundary` : the inclusions and the decomposition of the truth set into strict interior plus boundary.
- `strictTruth_falseSet_disjoint`, `truthBoundary_falseSet_disjoint` : the separations used later.

`HoF_02` (`HoF_02_TruthSet`). The truth set as a topological object in its own right, with the non-closedness that forces a boundary.

- `truthSet_isClosed`, `falseSet_isOpen`, `strictTruth_isOpen`, `truthBoundary_isClosed` : the topology restated for the abstract space.
- `truthBoundary_nonempty` : the boundary is nonempty under the covering hypothesis.
- `strictTruth_falseSet_separated`, `strictTruth_falseSet_no_intersect` : truth and falsehood do not touch except through the boundary.
- `strictTruth_not_closed`, `boundary_point_in_closure_of_strictTruth` : the strict truth region is not closed, and its closure reaches the boundary.

`HoF_03` (`HoF_03_BoundaryCrossing`). The intermediate value theorem doing its work: a path from a confident-true point to a confident-false point must cross.

- `confidence_path_crosses_half` : a confidence path from above to below one half crosses one half.
- `path_crosses_truth_boundary` : a path from truth to falsehood crosses the truth boundary.
- `model_truth_boundary_nonempty` : the model's truth boundary is nonempty.
- `model_confidence_half_set_nonempty`, `confidence_half_nonempty` : the half-confidence level set is nonempty.
- `model_truth_boundary_isClosed` : and closed.

`HoF_04` (`HoF_04_Faithfulness`). Faithfulness, the property that high confidence implies truth, and what it forces at the boundary.

- `StrongFaithful.toFaithful` : strong faithfulness implies faithfulness.
- `faithful_image_in_truth` : a faithful model's confident answers are true.
- `highConf_closed`, `highConf_open` : the high-confidence region's topology under the two conventions.
- `truth_preimage_closed`, `faithful_closure_in_truth` : the truth preimage is closed and its closure stays in truth.
- `faithful_at_half_boundary` : at the half-confidence boundary a faithful model is pinned to the truth boundary.
- `low_conf_unconstrained` : where confidence is low the model is free.

`HoF_05` (`HoF_05_Coverage`). Coverage, the hypothesis that the model's answers straddle both sides, which is what makes the boundary unavoidable.

- `covering_image_straddles_zero` : a covering model's signed distances take both signs.
- `denseCoverage_closure_eq_univ` : dense coverage fills the space in closure.
- `covering_yields_truth_boundary_point` : coverage produces a boundary point.
- `covering_yields_two_sided_witnesses`, `covering_two_closed_sides` : two-sided witnesses and their closedness.
- `covering_abs_zero`, `denseCoverage_approximable` : the distance hits zero, and dense coverage is approximable.

HoF_06 (HoF_06_Calibration). Calibration, the link between confidence and truth probability, and the half-boundary it induces.

- `calibrated_levelSet_eq` : the calibrated level set matches the truth level set.
- `confHalf_isClosed` : the half-confidence set is closed.
- `monotoneCalibrated_half_on_boundary` : monotone calibration puts half-confidence on the truth boundary.
- `calibrated_confHalf_nonempty` : the half set is nonempty under calibration.
- `confHalf_implies_truthBoundary` : half confidence implies boundary membership.
- `off_boundary_conf_not_half` : off the boundary, confidence is never exactly half.

HoF_07 (HoF_07_TrilemmaCore). The continuous hallucination trilemma. A model cannot be faithful, calibrated, and covering all at once; some question sits on the boundary with confidence exactly one half.

- `hallucination_trilemma_strict` : the strict form of the trilemma.
- `hallucination_trilemma` : the headline continuous trilemma.
- `hallucination_trilemma_unfolded`, `hallucination_trilemma_strict_unfolded` : the same with definitions unfolded, for downstream use.

HoF_08 (HoF_08_BorsukUlam). The topological upgrade. When the question space is a sphere and the model is antipodally symmetric, the Borsuk-Ulam theorem forces a boundary point without needing a chosen path. These results use the three standard axioms.

- `antipodal_odd_has_zero` : an odd continuous function on a sphere has a zero.
- `antipodal_yields_truth_boundary` : antipodal symmetry yields a boundary point.
- `antipodal_hallucination_trilemma` : the trilemma via Borsuk-Ulam.
- `approx_odd_near_zero` : an approximately odd function is near zero somewhere.
- `sphere_odd_has_zero` : the higher-dimensional sphere statement.

HoF_09 (HoF_09_Discrete). The trilemma with no topology at all, over a finite or discrete question space using a strict trichotomy and a sign change. Axiom-light, the discrete core needs no analysis.

- strictCal_conf_trichotomy : confidence is above, below, or exactly at half.
- discrete_partition, discrete_two_partition : the discrete partitions.
- discrete_trilemma_decisive_impossible : a decisive discrete model is impossible.
- discrete_sign_change : a sign change is forced between covered points.
- discrete_hallucination : the discrete hallucination result.

HoF_10 (HoF_10_PureDiscrete). The fully combinatorial version, connecting the boundary story back to the diagonal engine of Foundation.

- boundary_conf_half : the boundary question has confidence one half.
- boundary_question_impossible : the boundary question cannot be answered decisively.
- discrete_hallucination_trilemma : the pure-discrete trilemma.
- faithful_iff_boundary_free : faithfulness is exactly the absence of a boundary question.
- hallucination_trilemma_via_pure_discrete : the trilemma proved with no analysis.

HoF_11 (HoF_11_MultiTurnProbabilistic). The multi-turn and stochastic extension. An adversary steering a conversation can reach the boundary, and the expected behavior over turns still couples.

- multi_turn_per_turn, multi_turn_history_dependent : per-turn and history-dependent forms.
- adversary_boundary_reachable : an adversary can reach the boundary.
- stochastic_coupling_expected, stochastic_trilemma_expected : coupling and the trilemma in expectation.
- stochastic_dichotomy, boundary_dichotomy : the stochastic and boundary dichotomies.

HoF_12 (HoF_12_Approximate). The approximate trilemma with explicit slack parameters ε, and the bridges back to the exact statements. Axiom audit printed.

- `approx_trilemma` : the trilemma with a slack ε.
- `exact_from_approx` : the exact trilemma recovered as ε goes to zero.
- `strictCalibrated_to_epsCalibrated_zero_half`, `trilemmaCoveri ng_to_epsCovering_zero` : the exact hypotheses as zero-slack cases.
- `no_boundary_in_upper_band`, `no_boundary_in_upper_band_two_sl ack` : where no boundary can hide.
- `epsCalibrated_iff_twoSlack`, `two_slack_approx_coupling`, `truth _slack_must_be_positive`, `two_slack_band_bridge` : the two-slack reformulation and its coupling.

HoF_13 (HoF_13_BoundaryCoupling). The coupling theorem, the truth-boundary analogue of the reflective-verdict impossibility, with the closed-versus-open guarantee dichotomy that Chapter 3 turns on. Axiom audit printed.

- `boundary_coupling` : truth and confidence are coupled at the boundary.
- `closed_guarantee_impossible` : a closed (two-sided) guarantee at the boundary is impossible.
- `open_guarantee_silent` : an open guarantee is silent exactly on the boundary.
- `slack_must_be_positive` : any honest guarantee needs strictly positive slack.

HoF_14 (HoF_14_QuantitativeGeometry). The metric geometry of the boundary, the tubes and the linear-probe cosets that feed the interpretability chapter. Axiom audit printed.

- `confidence_gap_width` : the width of the confidence gap around the boundary.
- `truth_margin_tube`, `ambiguity_tube` : the truth-margin tube and the ambiguity tube around the boundary.
- `linear_probe_boundary_coset` : a linear probe's boundary is an affine coset.
- `linear_probe_boundary_dim` : its dimension.

HoF_15 (HoF_15_NonlinearBoundary). The nonlinear boundary, its regular points and tangent structure.

- regular_point_is_crossing : a regular point of the boundary is a genuine crossing.
- nonlinear_boundary_tangent : the tangent to a nonlinear boundary.
- tangent_hyperplane_dim : the dimension of the tangent hyperplane.

HoF_Instantiation (HoF_Instantiation_PromptSpace). The trilemma over a concrete prompt space of dimension d with k classes, so the abstract theorem becomes a statement about prompts.

- prompt_truth_boundary_exists : the prompt-space truth boundary is nonempty.
- prompt_confidence_half_exists : a half-confidence prompt exists.
- prompt_hallucination_trilemma : the trilemma over prompt space.
- prompt_near_boundary : prompts arbitrarily near the boundary exist.

HoF_Master (HoF_MasterTheorem, HoF_FinalVerification). The consolidated hallucination theorem over a HallucinationStructure, and the axiom audit.

- hallucination_master_theorem : the master statement bundling the hypotheses.
- hallucination_trilemma_three_clause : the three-clause form, at most two of faithful, calibrated, covering.

ManifoldProofs: the geometry of attack basins (MoF)

ManifoldProofs is the quantitative wing. Where HallucinationProofs establishes that a boundary exists, ManifoldProofs measures it: the volume of the safe basin, the radius of robustness robustnessRadius = (fp - τ) / L, the cost of an attack that walks to the failure manifold, and how these scale with dimension and model size.

The base modules fix the objects. MoF_01_Foundations defines the safety score f, the threshold τ, and the safe, unsafe, boundary, and failure-manifold regions, with their topology (mem_safeRegion, boundary_isCompact, space_partition, failureManifold_eq_union).

The Lipschitz group turns qualitative crossing into quantitative robustness. MoF_04_LipschitzBasin proves the robustness radius is positive and that a ball of that radius stays in the basin (lipschitz_basin_ball, robustnessRadius_pos, basin_ball_subset, perturbation_stability, robustnessRadius_monotone).

The defense and scaling group. `MoF_08_DefenseBarriers` formalizes barriers a defense erects around the basin.

The measure and coarea group sharpens the volume estimates. `MoF_17_CoareaBound` uses the coarea formula to bound the boundary's measure; `MoF_18_ConeBound` gives a cone estimate; `MoF_19_OptimalDefense` characterizes the defense that maximizes the robustness radius; `MoF_20_RefinedPersistence` and `MoF_21_GradientChain` refine persistence of the basin under perturbation and chain gradient steps together.

The advanced group `MoF_Adv_01` through `MoF_Adv_10` covers basin connectedness, boundary dimension, fine-tuning, model scale, convexity, approximation, fragmentation, stability, the optimization landscape, and measure bounds, in that order. The cost group `MoF_Cost_01` through `MoF_Cost_10` is the attack-economics core: ball volume, basin volume, hitting time, concentration, attack cost, defense cost, transfer cost, the cost ratio, Lipschitz estimation, and the unified theory.

Reading the corpus as one argument

Put the four libraries in a line and the book's thesis is visible in the dependency graph. `Foundation` proves the engine and reads it into every AI-safety corner combinatorially.

A few cross-library correspondences are worth naming, because they are the seams where the book's claims about sameness become theorems rather than assertions. The hallucination result appears three times, and the three are provably the same boundary.

The two-engine seam is the most important one. `HoF_07`'s continuous trilemma and `F_11`'s Gödel-form trilemma are about the same phenomenon and reach the same boundary question, but they are genuinely different proofs with different axiom footprints, and the book keeps both on purpose.

25.2 A Notation Glossary

The book reuses a small alphabet across very different readings. The same letter can be a prompt in one chapter and an activation vector in another; what stays fixed is the role the letter plays in the argument. This glossary lists each symbol with its meaning and, where it helps, the role it plays in the engine.

Two conventions help before the list. First, the letters track roles, not types.

Sets and spaces.

- `A` : the domain of _indices_.
- `Y` : the set of _outcomes_ a behavior can produce.
- `Q` : a domain of _questions_ or queries over which a verdict is defined.

- `X` : the ambient space in the manifold chapters, the space attacks move through.
- `Z` : an intermediate type in composition results, the target of a post-processor.
- `Prompt`, `Token`, `Output`, `Act` : concrete instantiations of `A` and `Y`, the prompt space, token space, output space, and...
- `Set A` : the power set of `A`, identified with indicator functions `A → Bool`.

The system and its parts.

- `f` : a _system_, `f : A → A → Y`.
- `g` : an arbitrary behavior `A → Y`, the thing universality asks to be named.
- `d` : the _diagonal behavior_, `d a = t (f a a)`, apply each behavior to its own name and then flip.
- `s` : a _section_, `s : (A → Y) → A`, a chosen index for each behavior, with `f (s g) = g`.
- `M` : a _model_, usually `M : Q → A × ℝ`, returning an answer and a confidence.

The engine's moving parts.

- `t` : the _controller_ or output flip, `t : Y → Y`.
- `v` : a _reflective verdict_, `v : Q → Q → Bool`, a boolean self-application in which every...
- `a₀`, `q₀`, `q_0` : the _liar_, the diagonal witness.

The analytic alphabet.

- δ : the _signed truth distance_, δ `(q, a)`, negative on true answers, positive on false ones,...
- `conf`, `confOf` : the model's _confidence_, the second projection of `M`.
- τ : the _threshold_ in the manifold chapters.
- ε : a _slack_ or tolerance.
- `c` : a confidence _level_ used to cut out the high-confidence region.
- `L` : a _Lipschitz constant_ for the safety score, the rate at which the score can change per...

- `fp` : the safety score at a point, written `fp` in `robustnessRadius = (fp - τ) / L`.

Regions and objects.

- _truth set_, _false set_ : where δ is nonpositive and where it is positive.
- _strict truth_ : the open interior of the truth set, where δ is strictly negative.
- _truth boundary_ : the zero set of δ, where true meets false.
- _high-confidence region_ : where `conf` is at least `c`.
- _basin_ : the safe basin, the open set where the score `f` exceeds `τ`, of positive measure and...
- _failure manifold_ : the closure of the unsafe region against the boundary, the codimension-one set an attacker...
- _robustness radius_ : `(fp - τ) / L`, how far a safe point can be pushed before it crosses.

Properties named in the trilemmas.

- _universal_, _point-surjective_ : every behavior is named, `∀ g, ∃ a, f a = g`.
- _faithful_ : high confidence implies truth.
- _calibrated_ : confidence matches truth probability;
- _covering_ : the model's answers straddle both sides of the boundary;
- _control_, _nontrivial controller_ : the output can be steered by a `t` with no fixed point.
- _transparent_ : the diagonal is itself a named behavior.

Logical and Lean symbols.

- `∀`, `∃`, `¬`, `→`, `≠`, `=` : the usual quantifiers and connectives.
- `!` : boolean negation, the controller in the Cantor instance.
- `Nat.succ`, `+ 1` : the successor, a fixed-point-free controller on the naturals and integers.
- `False`, `True`, `Prop`, `Bool` : Lean's falsehood, truth, the type of propositions, and the type of booleans.

- `congrFun` : the step "equal functions agree at every point," the one non-trivial move in the proof of...
- `decide` : the tactic that settles a finite boolean claim, how `bool_not_fpf` is proved axiom-free.
- `closure`, `Continuous`, `IsOpen`, `IsClosed`, `IsCompact` : the topological vocabulary the analytic chapters run on.

25.3 The Two Engines, Side by Side

| | Diagonal | Value-crossing | |—|—|—| | Hypothesis on the domain | universality: every behavior is named | connectedness | | Hypothesis on the maps | none | continuity, and Lipschitz for quantities | | Controller | fixed-point-free `t`, typically `!·` | complement `y ↦ 1 - y`, one fixed point at the threshold | | Output | a liar index a_0 | a boundary point q_0 | | Axioms | none in the Boolean core | the three standard ones, through Mathlib's reals | | Quantitative content | none | band widths, basin volumes, attack and defense cost | | Escaped by | breaking universality | breaking connectedness, coverage, continuity, or exact separation |

The two rows that matter are the last two. The diagonal buys an unconditional existence proof with the smallest possible trust story and gives nothing measurable.

They agree where both apply. `Foundation.F_04` records the agreement as a theorem rather than an analogy: both engines extract the fixed point of the same controller, the Boolean flip having none and the real complement having exactly one, at the threshold.

The practical rule for reading the corpus is to ask which engine is running. A claim of bare impossibility is the diagonal. A claim carrying a number — a width, a rate, a probability, a cost — is the value-crossing engine and its Lipschitz refinements.

25.4 Annotated Bibliography

The book stands on a short list of sources. The foundational papers are the ones that isolate the diagonal and the boundary; the companion papers are the AI-safety readings this development formalizes; the interpretability work is where the two engines are seen firing together.

Foundational.

- F. William Lawvere, _Diagonal arguments and cartesian closed categories_ (1969). The origin of the whole book. Lawvere shows that Cantor, Russell, Gödel, Tarski, and Turing are one theorem about a point-surjective map in a cartesian closed category, and that the theorem is a fixed-point statement. Every impossibility in these notes is an instance of his lemma, and `lawvere` in `F_01` is his proof in Lean.

- Noson Yanofsky, _A universal approach to self-referential paradoxes, incompleteness and fixed points_ (2003). The most readable modern account of Lawvere's unification, working the classical instances out by hand. It is the template Chapter 2 follows, and the reason the book can present five paradoxes as one schema without categorical machinery.

- Kurt Gödel, on the incompleteness theorems (1931). The archetype of a self-referential impossibility: a sentence that asserts its own unprovability is a fixed point of the provability flip. In the corpus this is the shape of `tarski_liar` and the Gödel-form trilemma statements in `F_11`–`F_13`.

- Alfred Tarski, on the undefinability of truth (1936). Truth cannot be defined inside the language it is about, because the liar is a fixed point of negation. This is the direct ancestor of the reflective-verdict impossibility `no_reflective_verdict`, and `tarski_liar` carries his name for that reason.

- Henry Gordon Rice, on properties of recursively enumerable sets (1953). Every nontrivial semantic property of programs is undecidable. The book reads this as "no complete automated self-test," formalized in `F_02` and appearing as the Controlled-Transparent corner `rice_face` of the CCH trilemma.

- Georg Cantor, on the non-denumerability of the reals and the power-set theorem (1874, 1891). The first diagonal, and the counting-with-teeth that no set surjects onto its power set. It is `cantor_set` and `specification_bound` in `F_08`, and the boolean flip `bool_not_fpf` is Cantor's flipped diagonal digit.

Companion papers.

- _The Hallucination Trilemma_. The claim that a language model cannot be simultaneously faithful, calibrated, and covering; some question forces confidence to the fence. This is the subject of `HallucinationProofs`, with the continuous proof in `HoF_07` and the Gödel-form proof in `F_11`. The book's Chapter 3 is the argument that these three properties really do make a model's verdict reflective.

- _The Defense Trilemma_. The claim that a prompt-injection defense that judges prompts, including prompts about its own judgments, cannot be universal, controlled, and transparent at once. Formalized in `F_12` and as the CCH corners, with `defense_is_hallucination_engine` showing it is the same theorem as the hallucination result.

- _Truth Has a Boundary_. The coupling result: truth and confidence are pinned together on a boundary, and no honest guarantee can be two-sided there without positive slack. This is `HoF_13`'s `boundary_coupl`

ing, closed_guarantee_impossible, and slack_must_be_positive, and it is what ties the diagonal's liar to the value engine's zero of δ.

Interpretability.

- The _J-space_ interpretability work. The program of reading a model's judgments off its own activations. The book's contribution is negative and sharp: a complete self-reading probe is a reflective verdict, so it is impossible, and the boundary it fails on has the metric geometry of HoF_14. This is F_14's jspace_readout_impossible and jspace_is_the_same_engine, the clearest place where the interpretability question and the diagonal turn out to be one question.

How the sources fit together. The foundational six are a single arc that the book compresses into one theorem. Cantor supplies the diagonal move and the flipped digit.

25.5 Building and Checking This Thesis

Layout. Four Lake packages carry the corpus: Foundation (F_01–F_14), CCH Proofs (CCH_01–CCH_Master), HallucinationProofs (HoF_01–HoF_15 with instantiation, master, and verification files), and ManifoldProofs (the MoF_ families). The numbering is a reading order as much as a build order: lower numbers are the engine and the setup, higher numbers the readings and refinements.

Building the libraries. From a package root, lake exe cache get fetches the prebuilt Mathlib and lake build compiles the package. A successful build is the whole guarantee: the kernel has checked every proof, and because the packages are sorry-free a clean build means no gaps.

Building the document. lake build elaborates every code block in the chapters, including the live proofs, and renders the manual. Because the blocks are elaborated at build time, the proofs you read are the proofs the kernel accepted; there is no separate step that could drift. This appendix contains no live blocks, only backticked names, so it renders as reference text.

Checking the prose against the corpus. Elaboration checks the code blocks, but a theorem stated in prose and attributed to a library is checked by nothing: rename the declaration and the text keeps asserting it. The build script closes that gap before calling Lean.

Running the axiom audit. #print axioms name reports the complete transitive axiom list. Several files already end with these commands: F_11, F_13, F_14, CCH_FinalVerification, HoF_08, HoF_12, HoF_13, HoF_14, HoF_FinalVerification, and MoF_FinalVerification. To audit any other theorem, add a line after its file's imports and rebuild.

Reading the output. Three outcomes matter. "Does not depend on any axioms" means axiom-free in the strict sense, which is what the Boolean diagonal core reports. A list of exactly propext, Classical.choice, and Quot.sound means classically clean, which is what the analytic results report.

A suggested check. Build `Foundation` and print the axioms of `no_reflective_verdict` in `F_13` and `hallucination_trilemma_godel` in `F_11`; the engine is clean. Then build `HallucinationProofs` and print the axioms of `hallucination_trilemma` in `HoF_07` and `boundary_coupling` in `HoF_13`; the three standard axioms appear where the analysis enters.